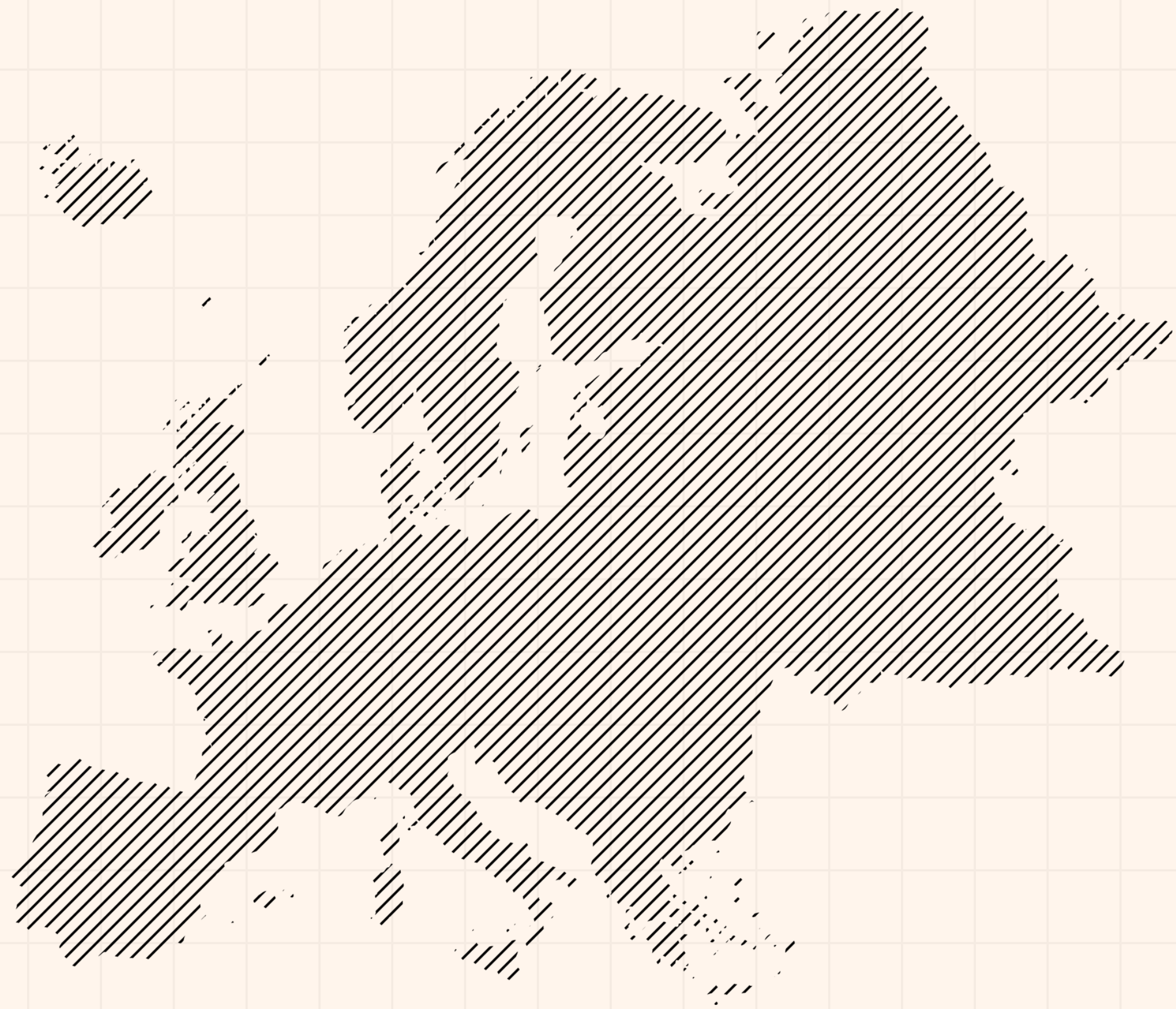




Aprendizaje No Supervisado

SIA-72.27 Grupo 3

1. EUROPA



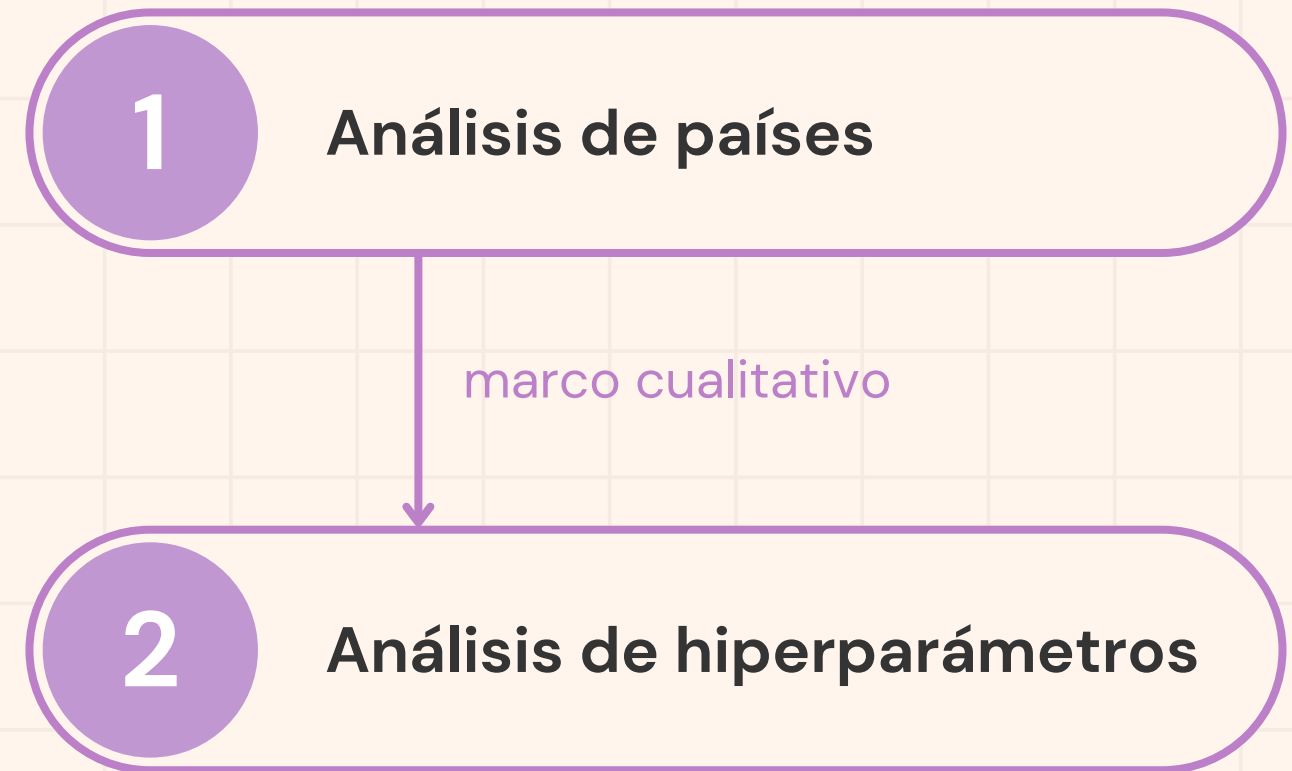
KOHONEN

Mapa autoorganizado 2D

Haykin, *Neural Networks and Learning Machines*, 3rd ed., 2009.

KOHONEN

Mapa autoorganizado 2D



Haykin, *Neural Networks and Learning Machines*, 3rd ed., 2009.

IMPLEMENTACIÓN: SCHEDULES

Tasa de aprendizaje

$$\eta(i) = \max \left(0.01, \eta_0 \times \exp \left(\frac{-i}{1000} \right) \right)$$

Radio del vecindario

$$R(i) = \max \left(1, R_0 \times \exp \left(\frac{-i}{\tau_1} \right) \right)$$
$$\tau_1 = \frac{1000}{\ln(R_0)}$$

k	4
eta_0	0.1
radius_0	k
n_iter	500×k×k
weight_init	samples sin reemplazo
distance	euclidean

PAÍSES POR NEURONA (HIT MAP)

k

4

eta_0

0.1

radius_0

k

n_iter

500×k×k

weight_init

samples

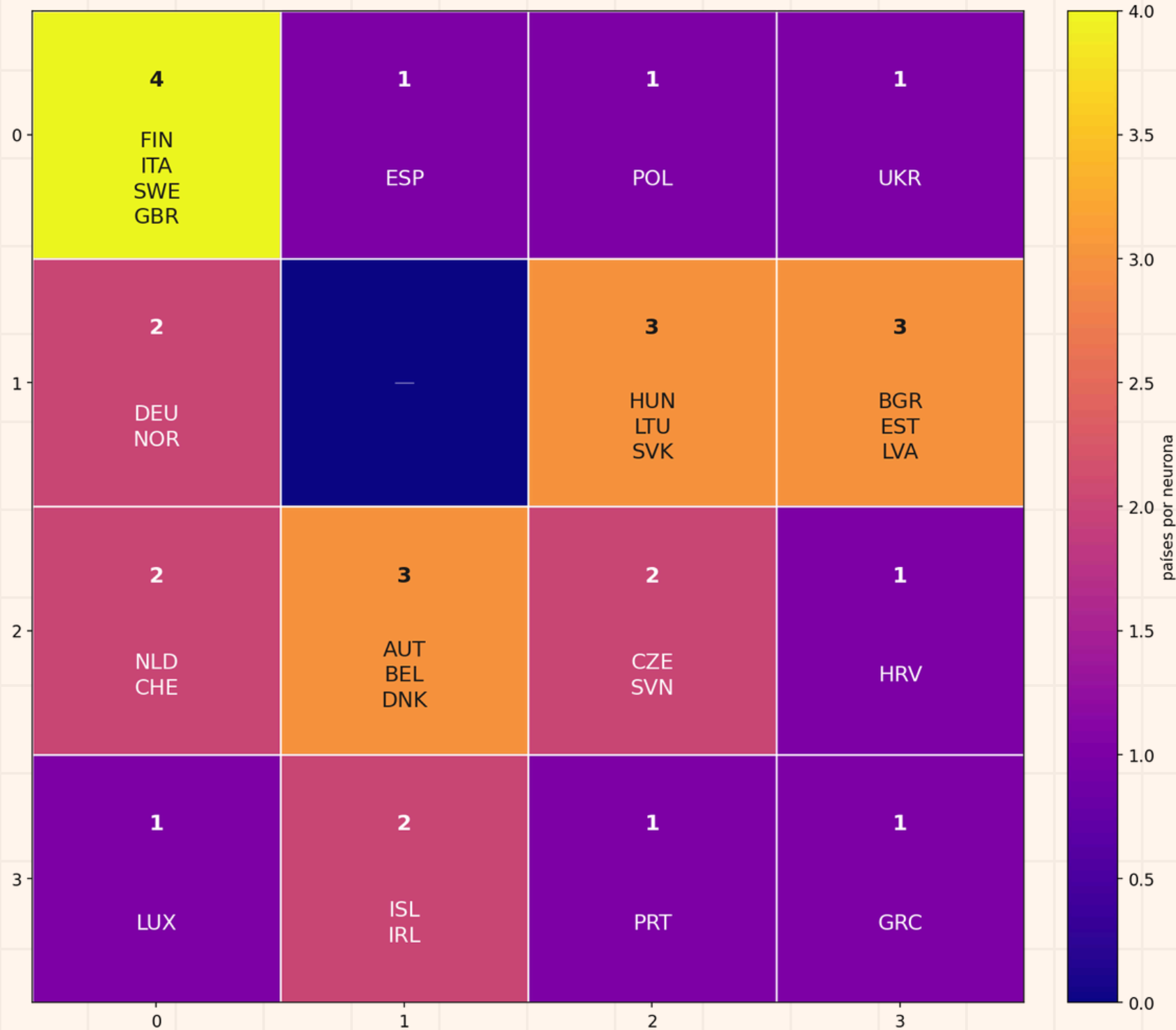
sin reemplazo

distance

euclidean

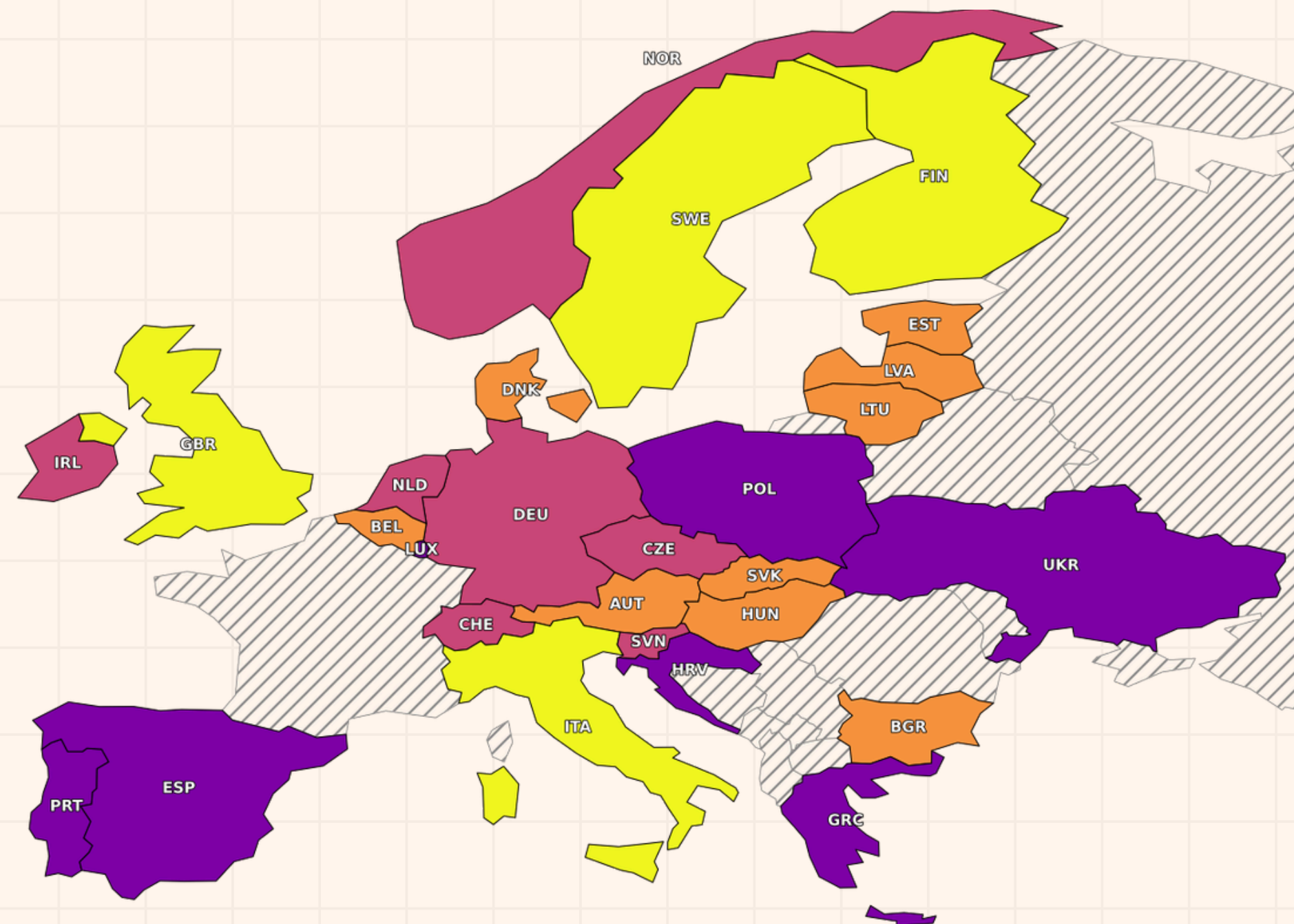
seed_used

6



1

ANÁLISIS DE PAÍSES

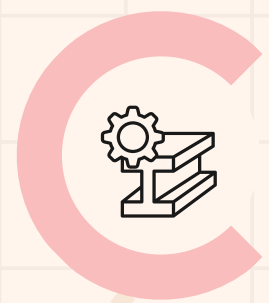




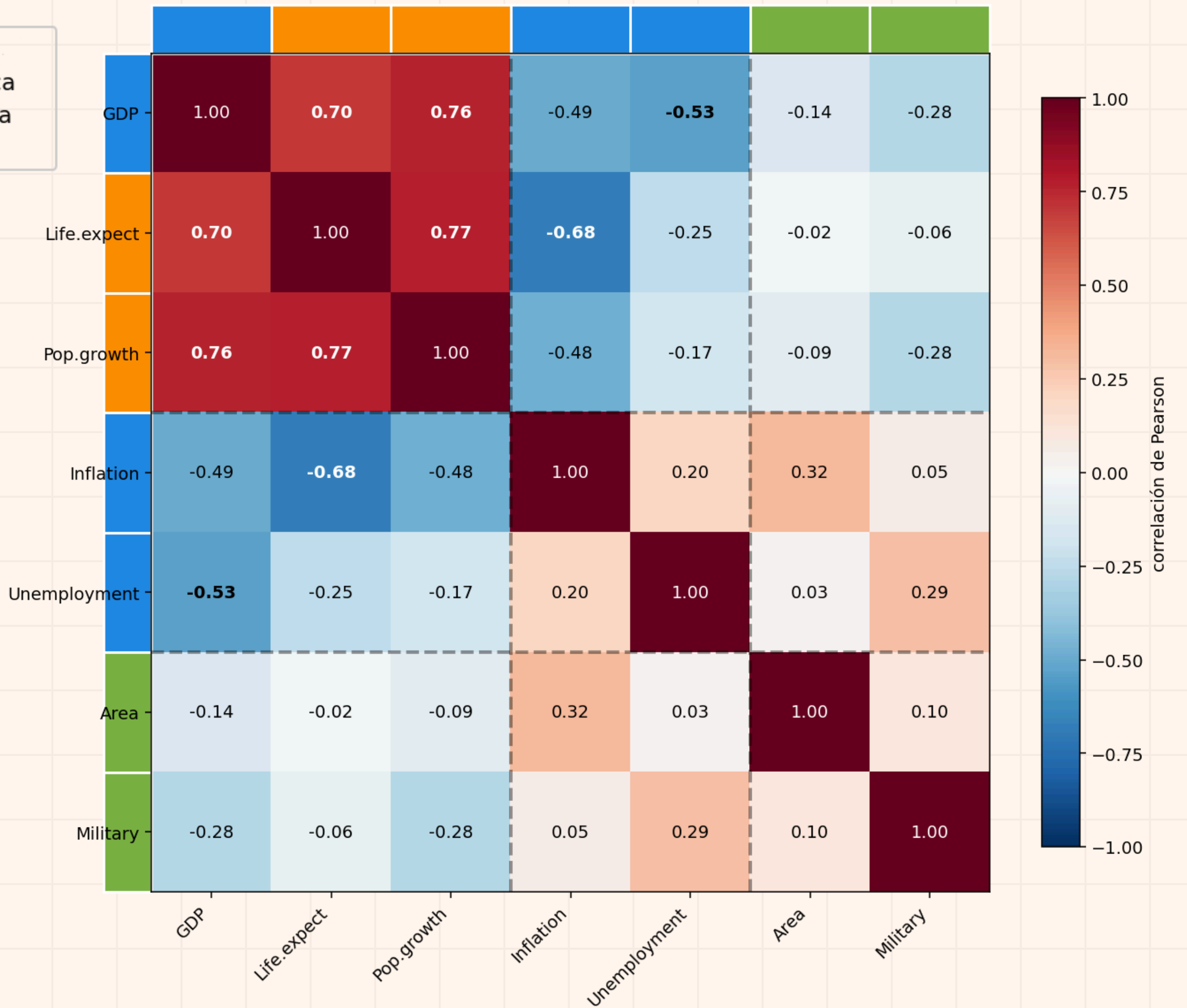
Eje de bienestar (principal)
GDP, **LifeExpect**, **PopGrowth**



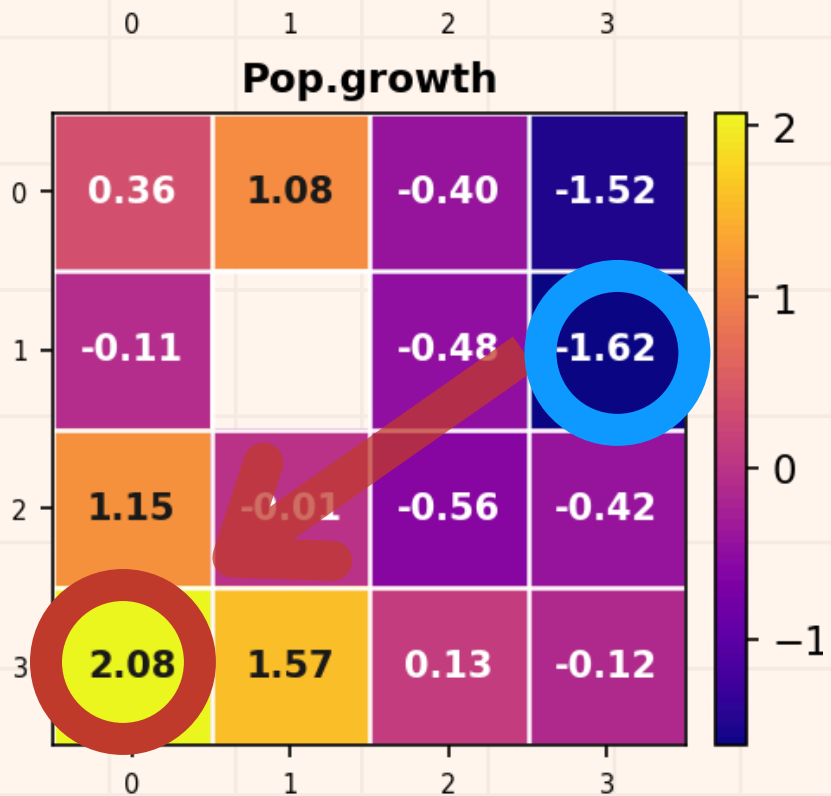
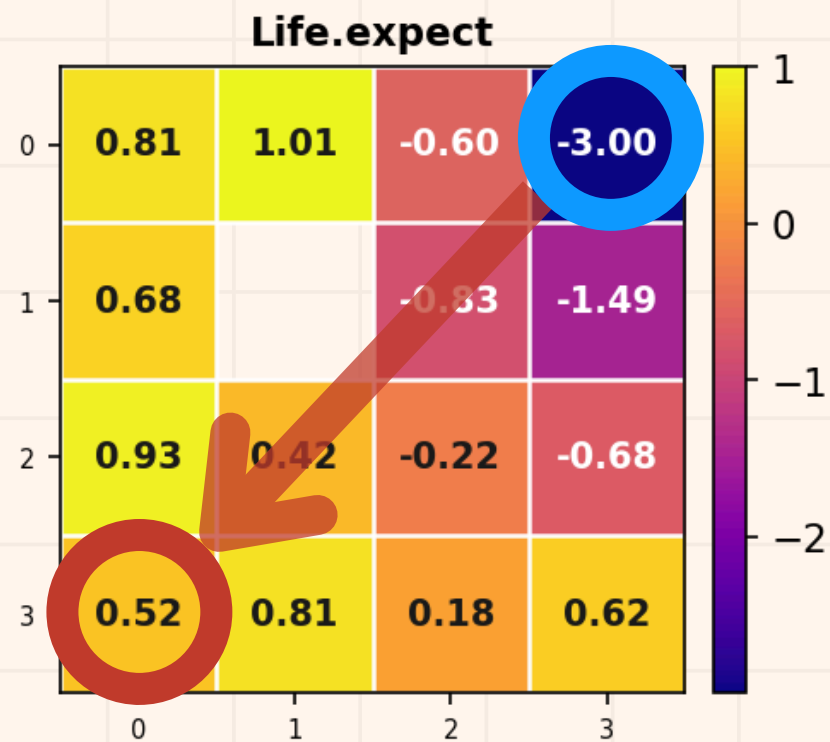
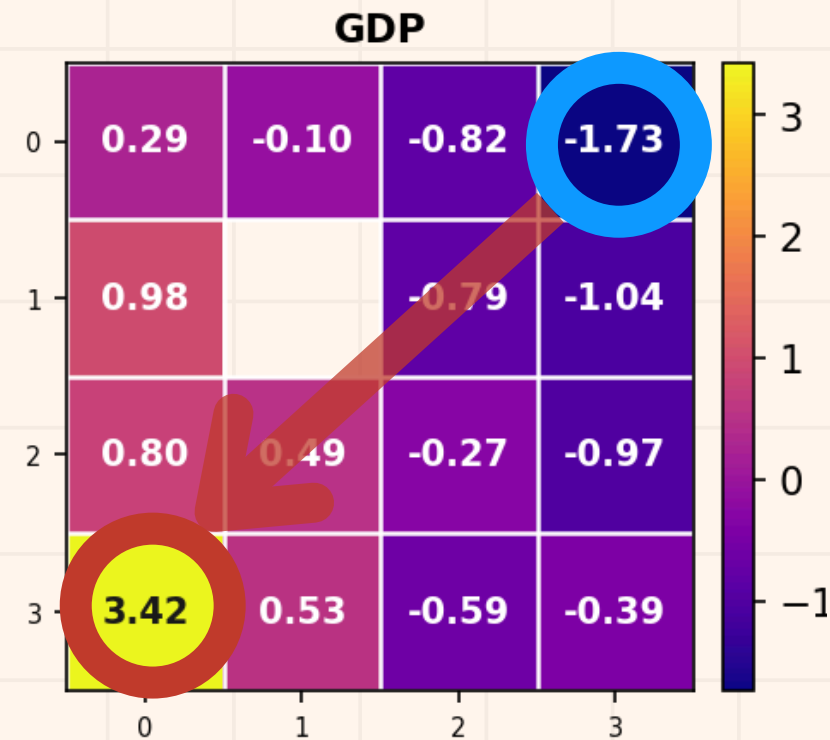
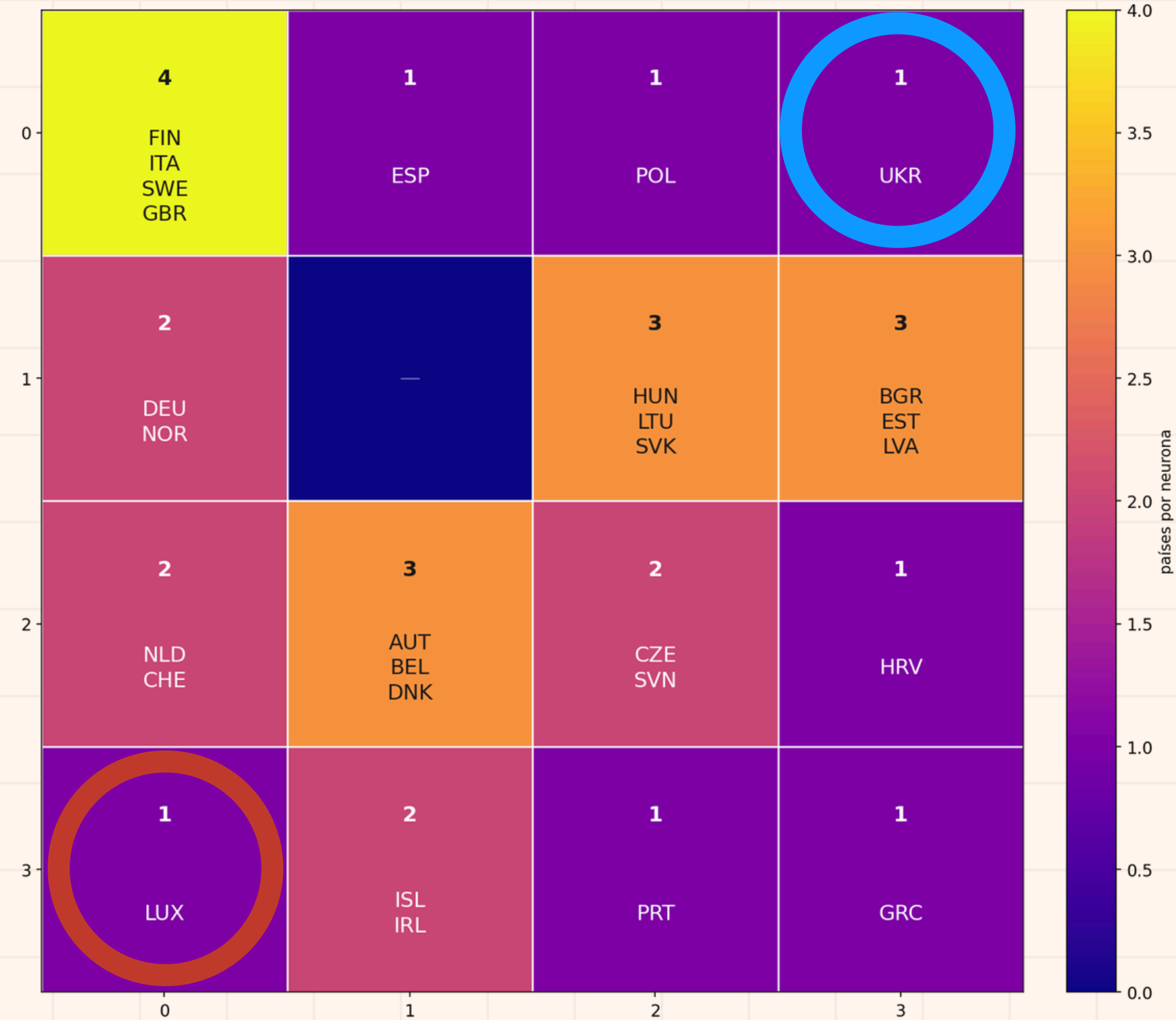
Variables opuestas al eje principal
(Indicadores de crisis)
Inflation, **Unemployment**



Variables independientes
Area, **Military**



ANÁLISIS POR VARIABLE: EJE DE BIENESTAR



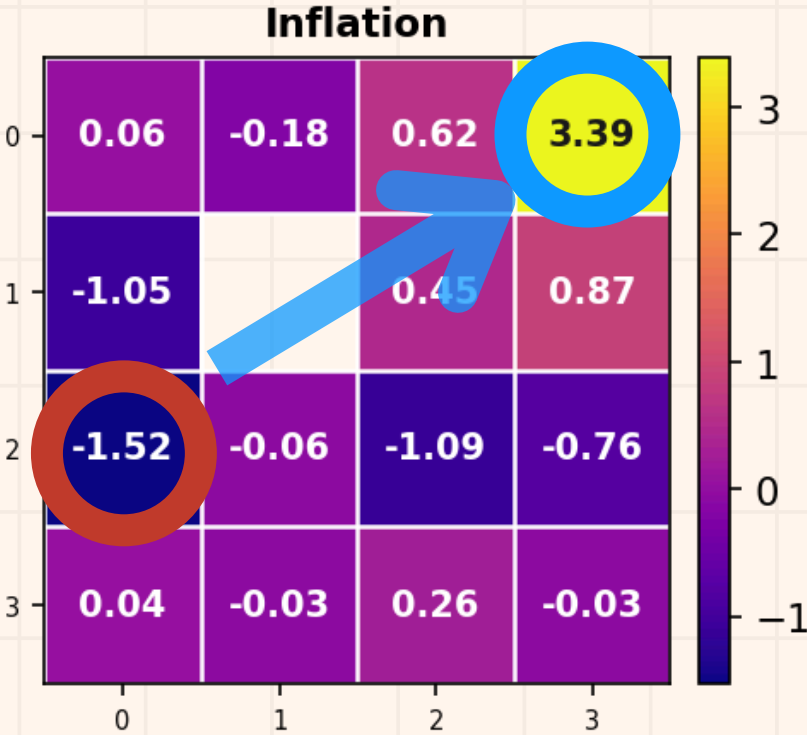
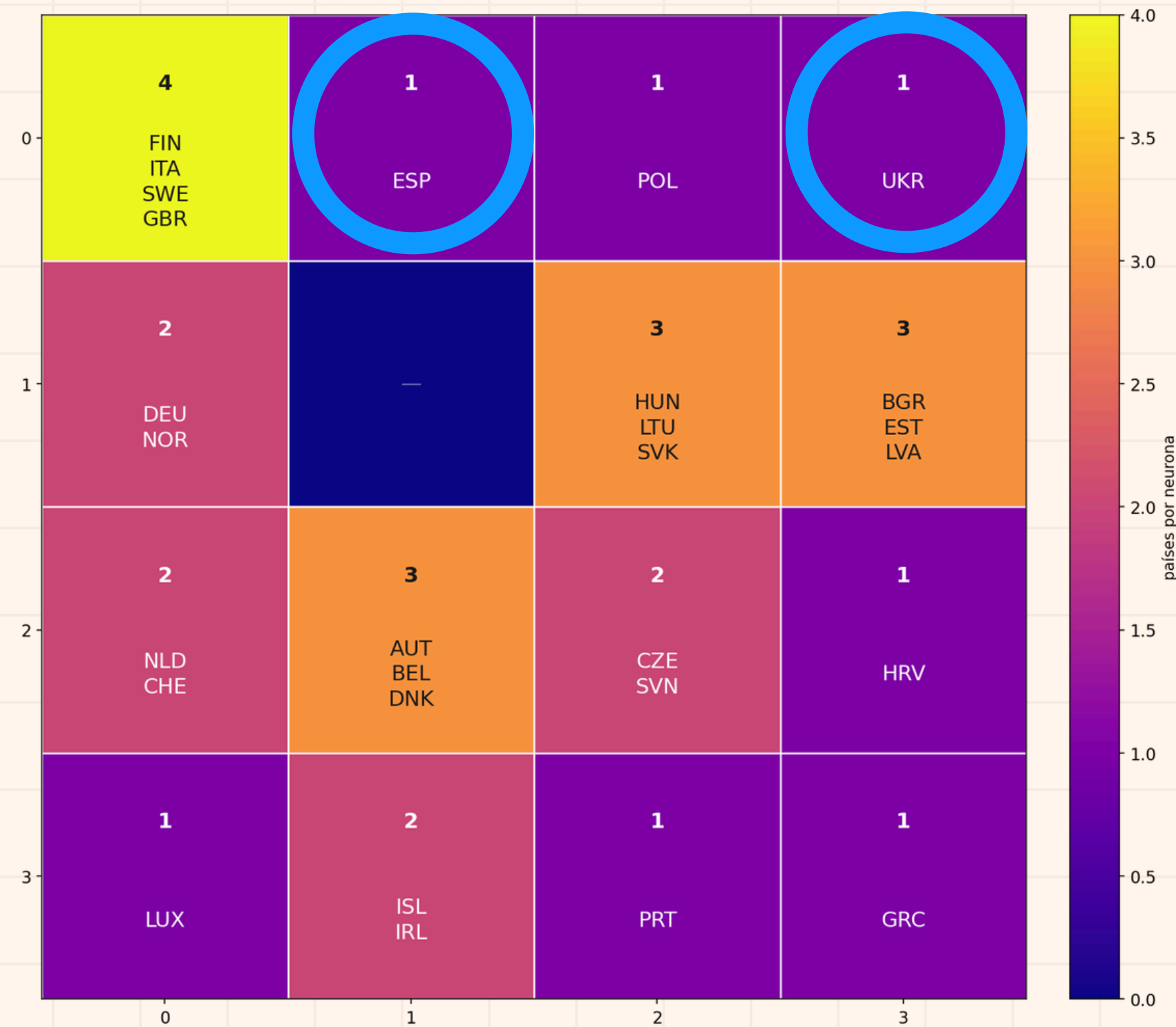
3 variables comparten el mismo gradiente: bajo a derecha, alto a izquierda,

Eje diagonal del mapa codifica bienestar

ANÁLISIS POR VARIABLE: INDICADORES DE CRISIS

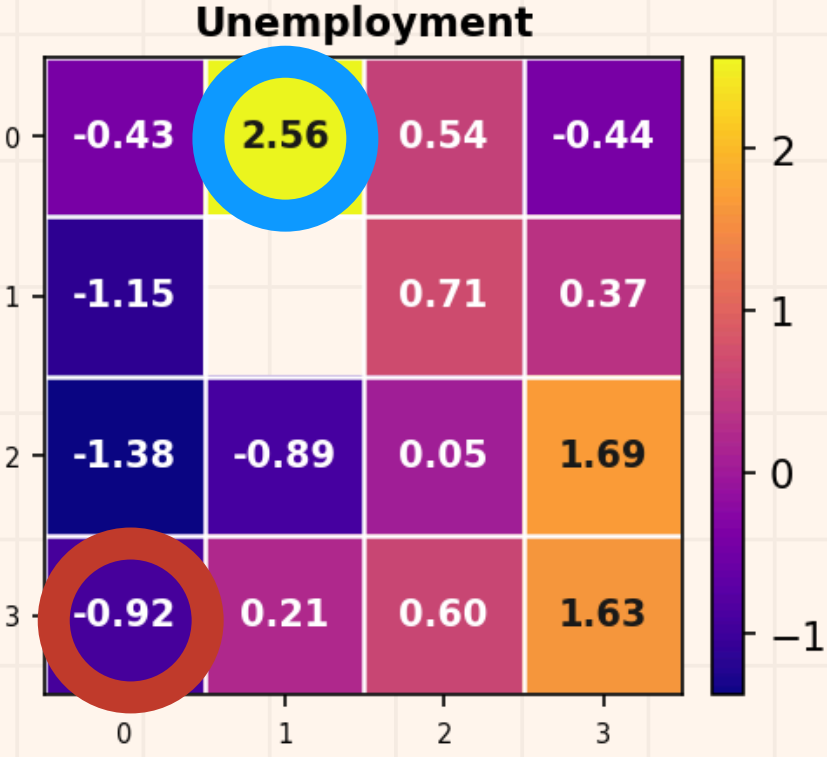


Países agrupados por la red de Kohonen (4x4)

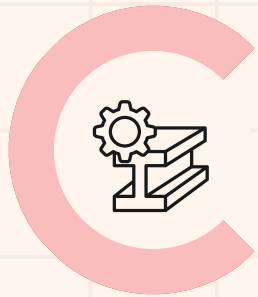


Tiende en sentido opuesto al eje de bienestar aunque con ruido

Unemployment es más ruidoso, consistente con su baja correlación con el eje principal



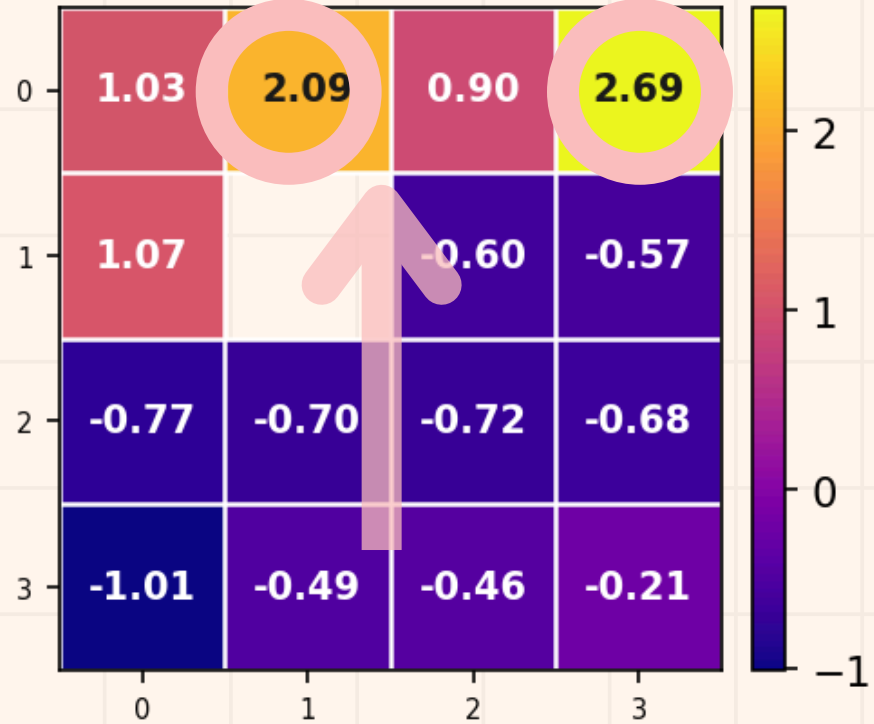
ANÁLISIS POR VARIABLE: INDEPENDIENTES



Países agrupados por la red de Kohonen (4x4)

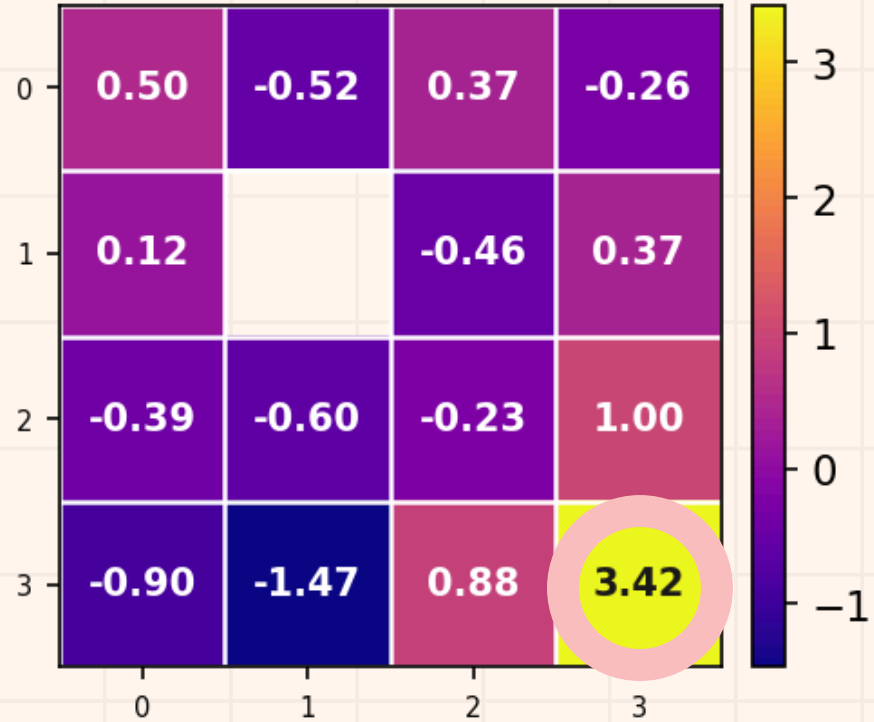


Area



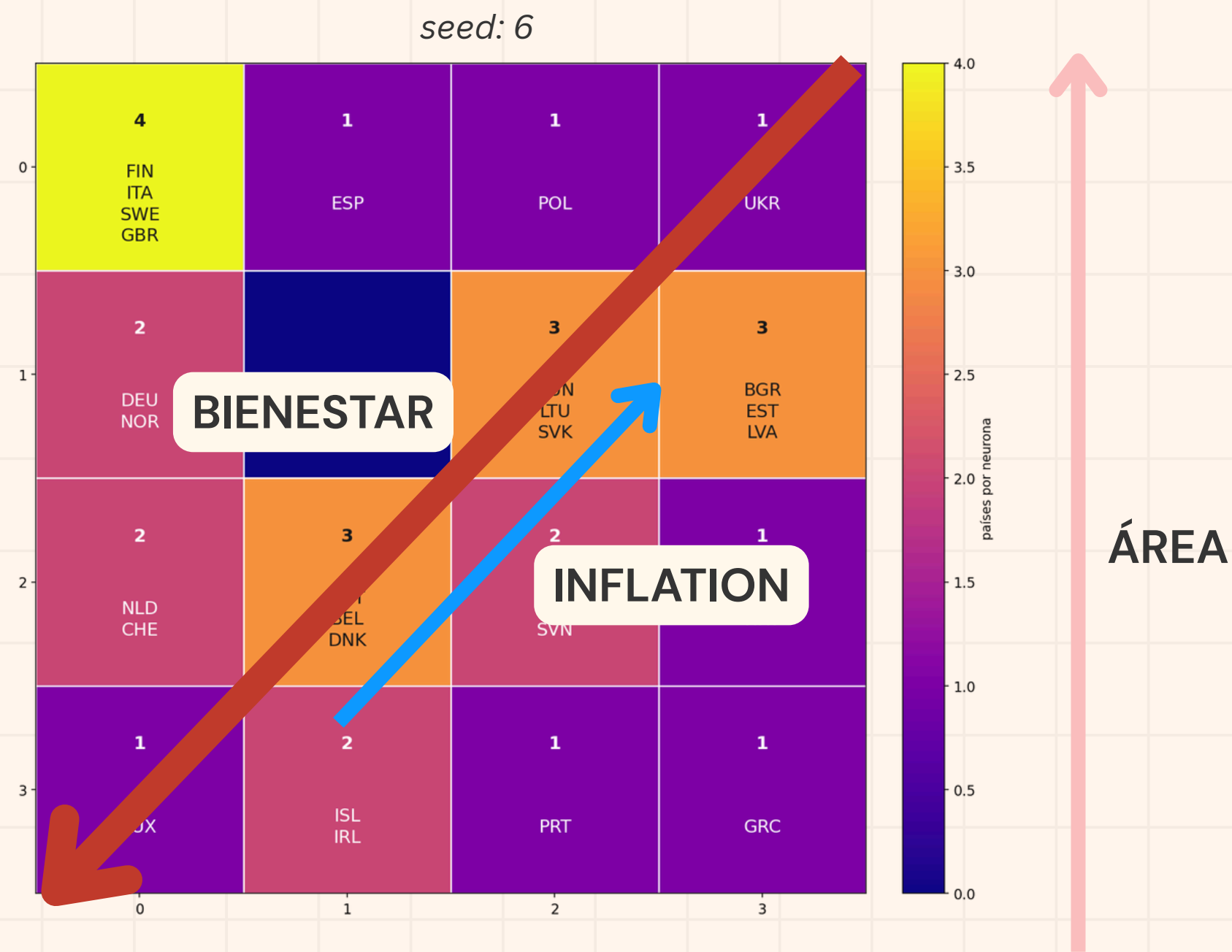
Crece de abajo a arriba.

Military

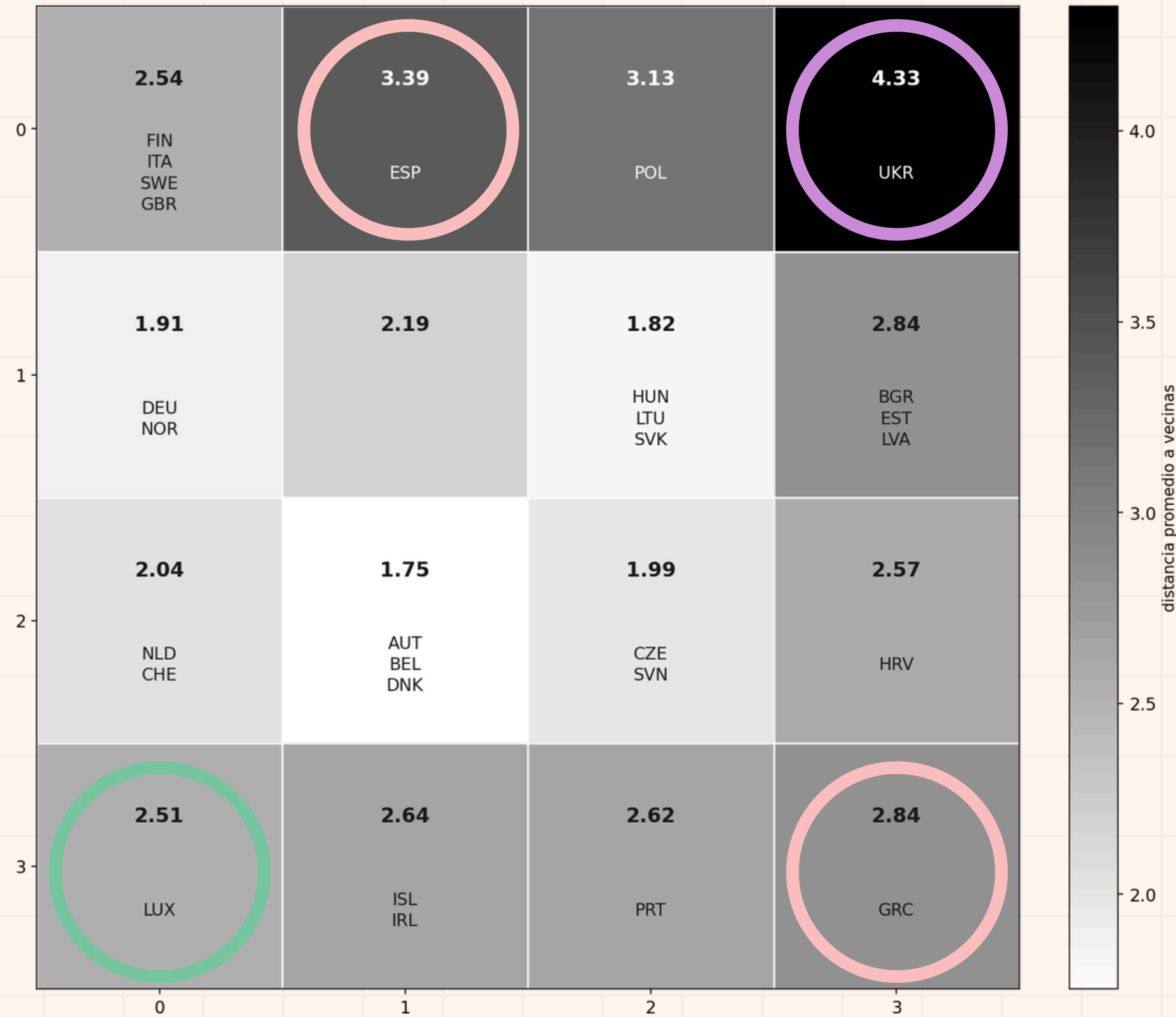


No se ordena en ninguna dirección.

Kohonen proyectó 7D en dos direcciones interpretables



U-MATRIX



Aislamiento alto

- **Ucrania** (extrema en **Inflación**, **Area**, **LifeExp**) → outlier multivariable

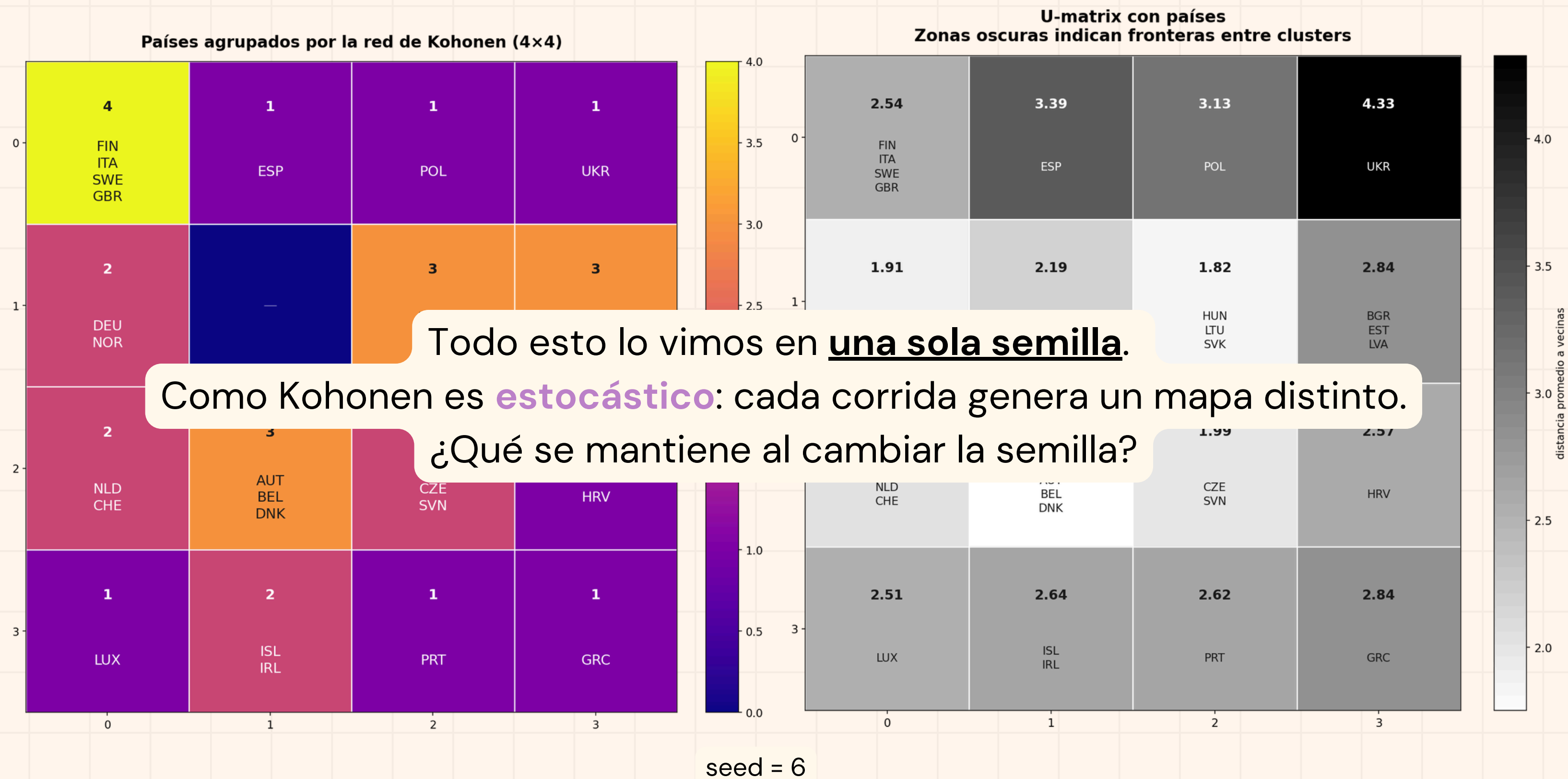
Aislamiento moderado

- **España**: extrema en dos (**Unemployment** y **Area**)
- **Grecia**: extrema en una sola (**Military**)

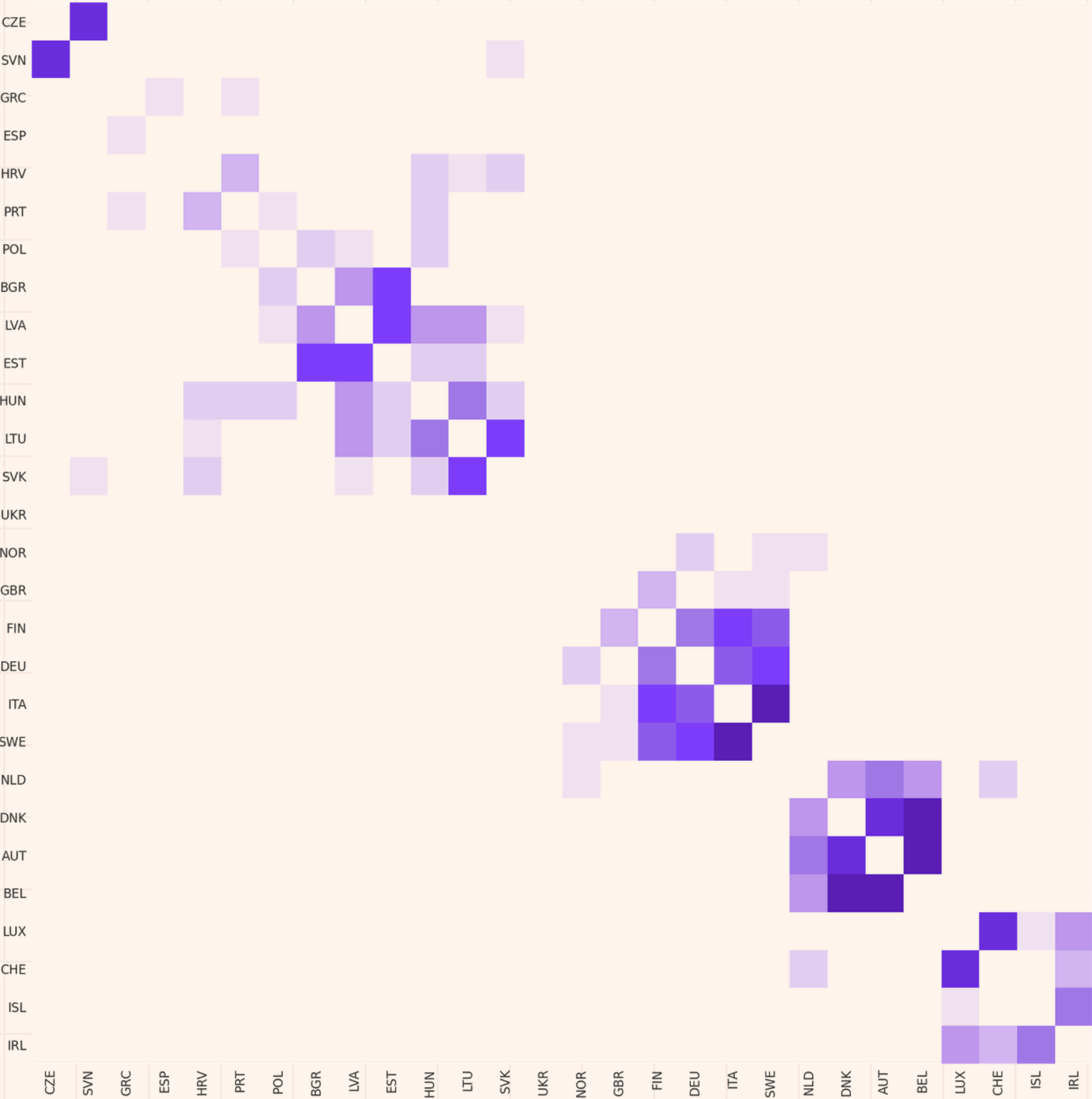
Aislamiento bajo

- **Luxemburgo**: extrema en **GDP**, **PopGrowth** pero rodeado de economías ricas

La U-matrix es local (no global): un extremo no implica frontera si se parece a las vecinas



Co-ocurrencia entre países • 10 semillas

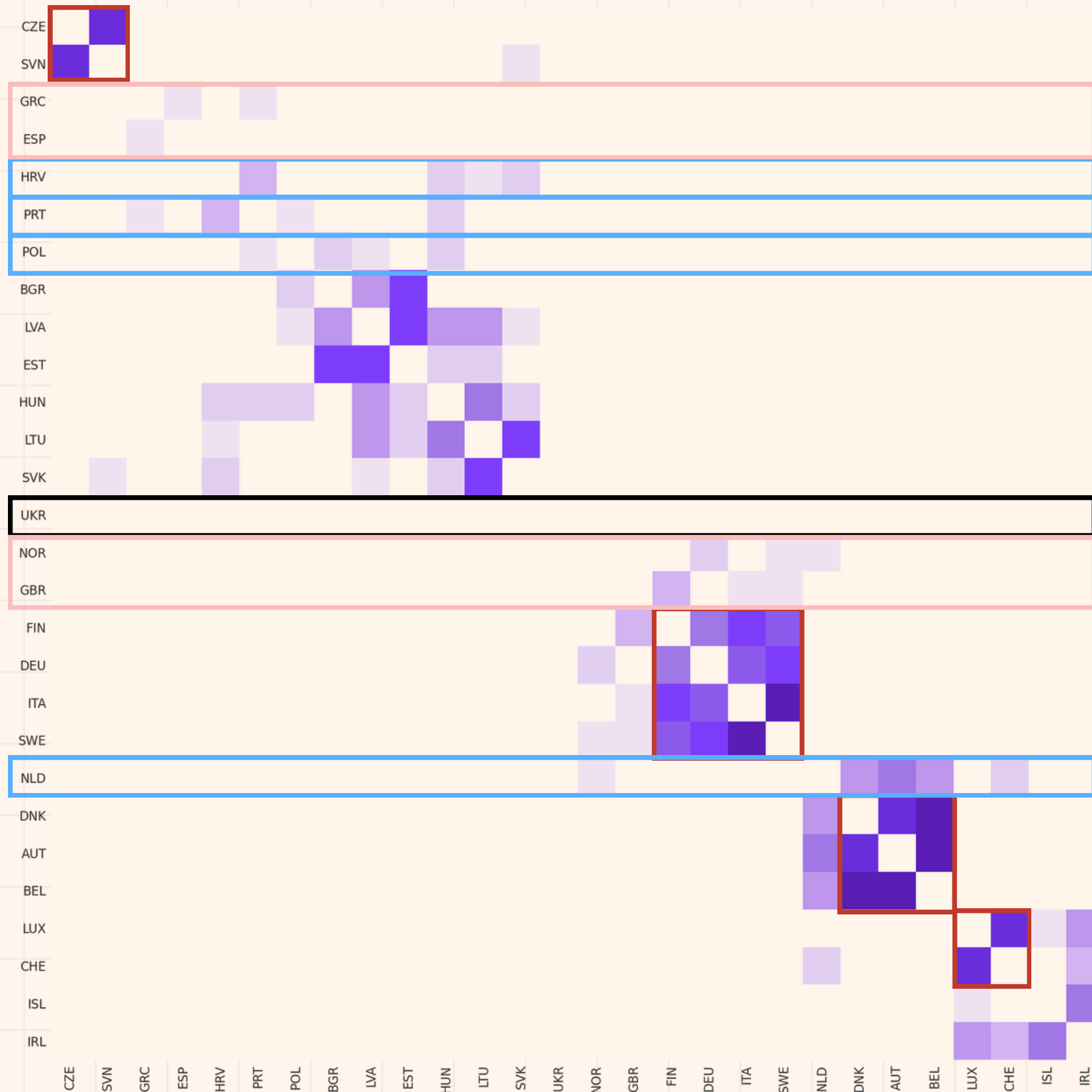


¿QUÉ SE REPITE ENTRE SEMILLAS?

- Cada semilla produce un mapa distinto; las posiciones (i,j) no son comparables entre corridas
- La **matriz de co-ocurrencia** cuenta cuántas veces cada par cae junto sobre las 10 semillas

**Ver apendice para ver la matriz con números*

Co-ocurrencia entre países · 10 semillas

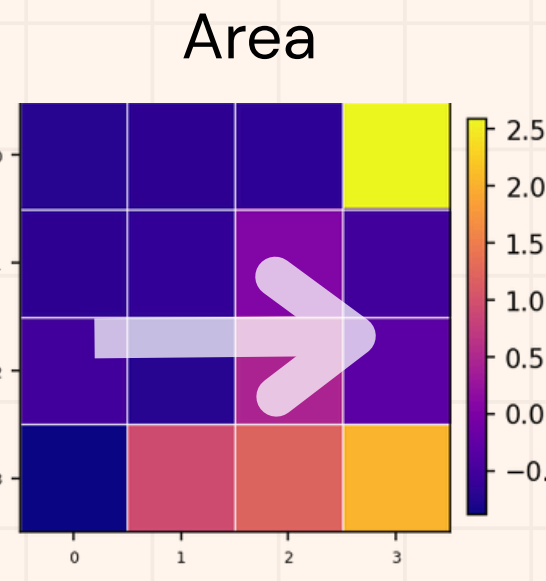
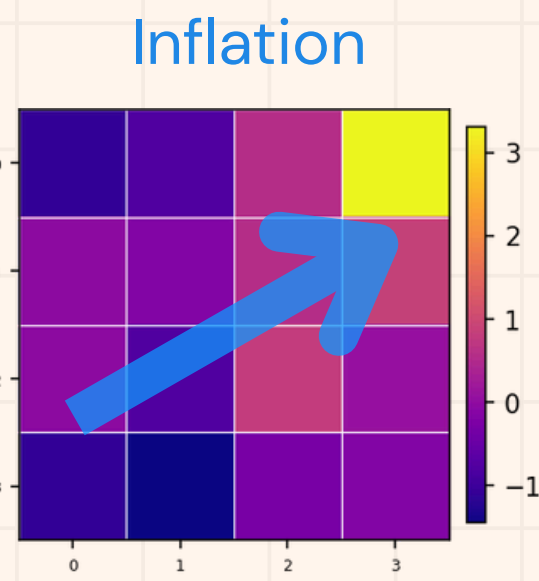
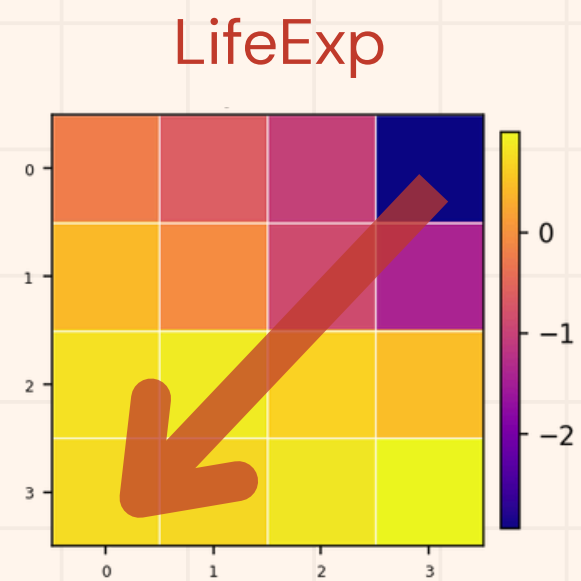
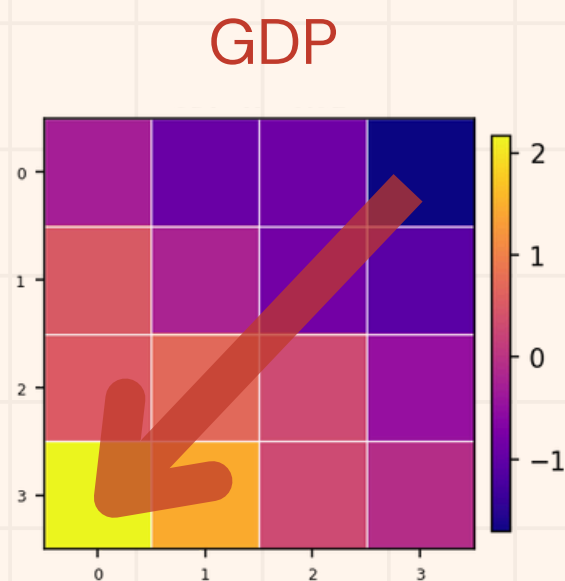


GRUPOS ESTABLES Y PAÍSES AMBIGUOS

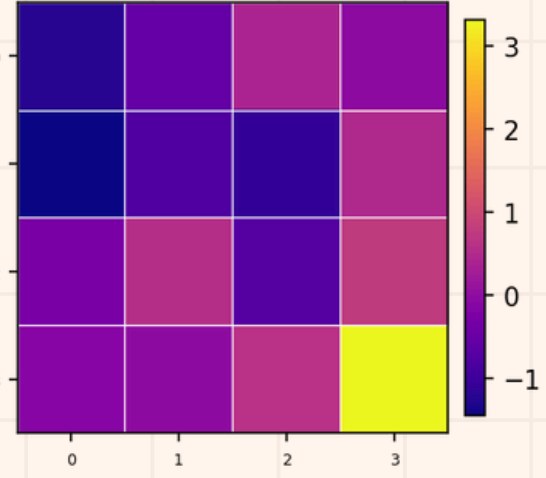
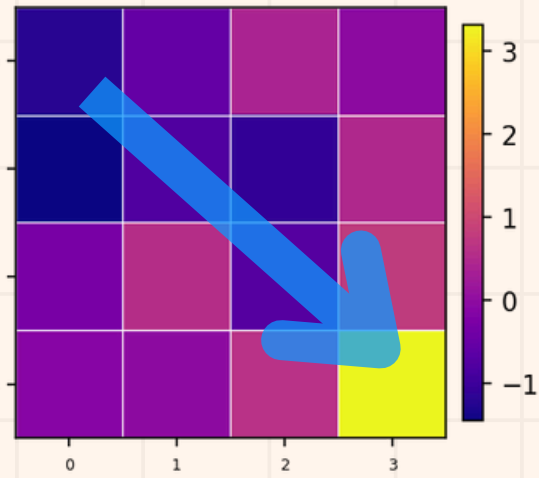
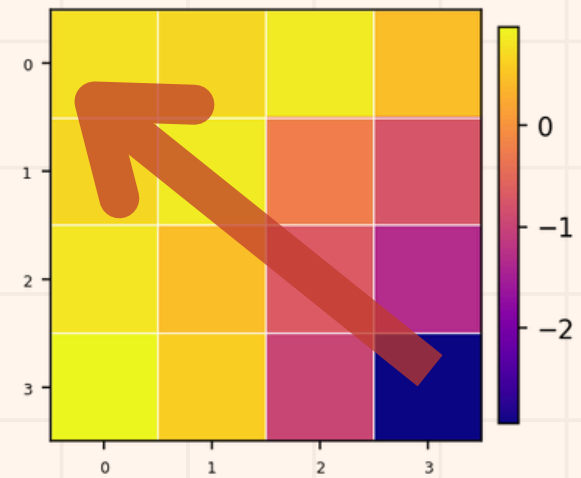
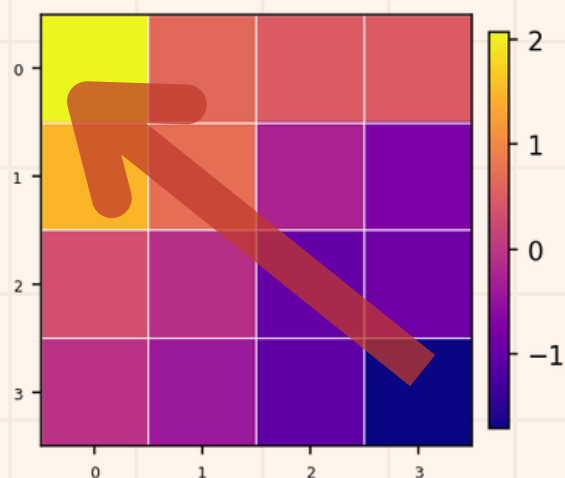
- **Bloques oscuros:** grupos que reaparecen entre semillas
- Celdas claras dispersas: **países de perfil ambiguo** que cambian de grupo según la semilla
- Ucrania aislada en **todas** las semillas
- **Outliers débiles** (casi siempre solas)

Grupos estables y outliers reaparecen entre semillas; lo que varía es a qué grupo se asignan los países de perfil ambiguo

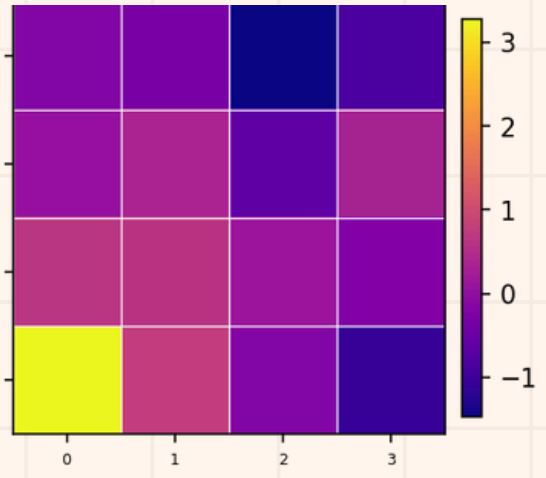
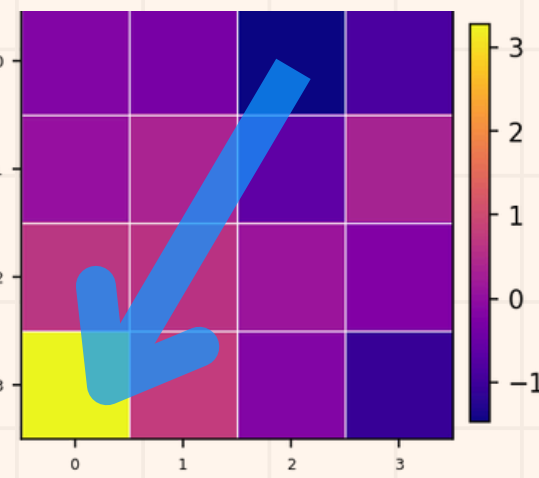
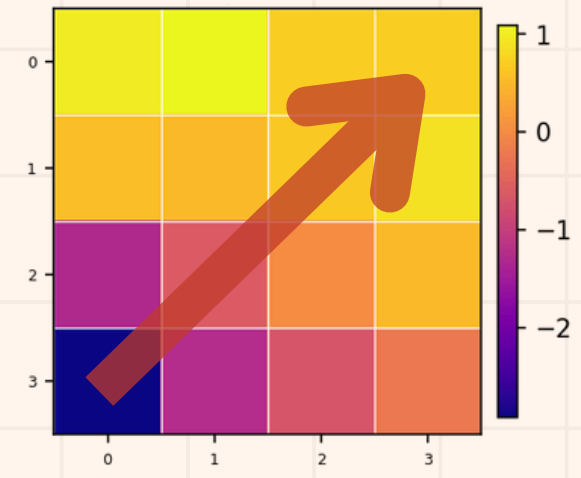
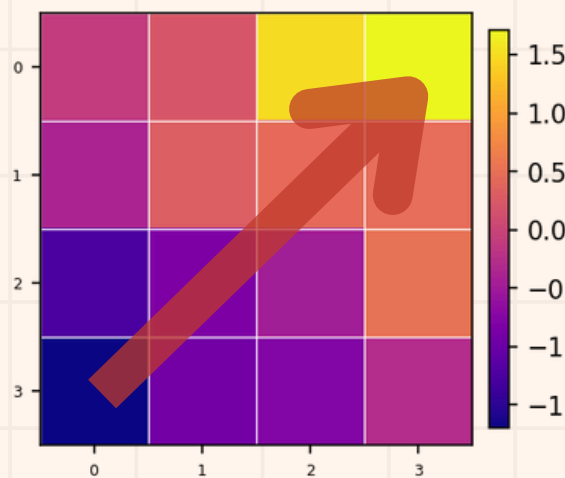
seed: 2



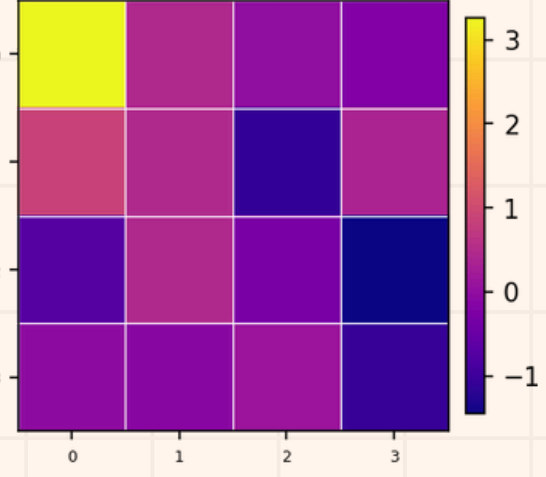
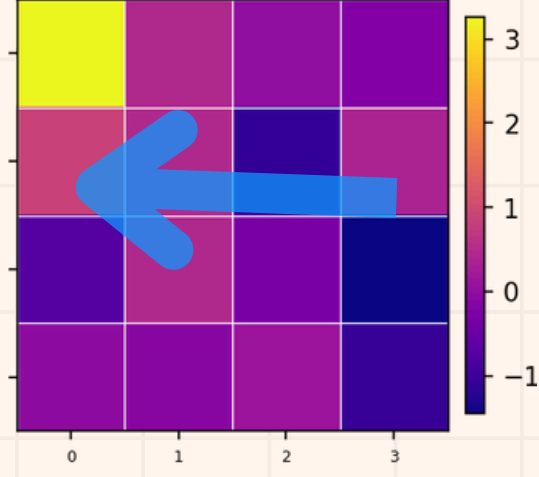
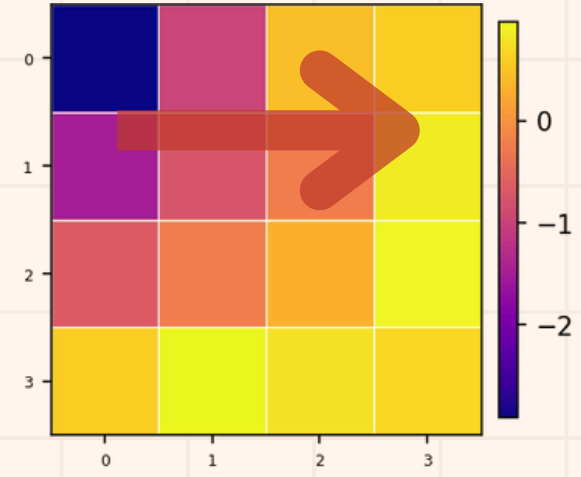
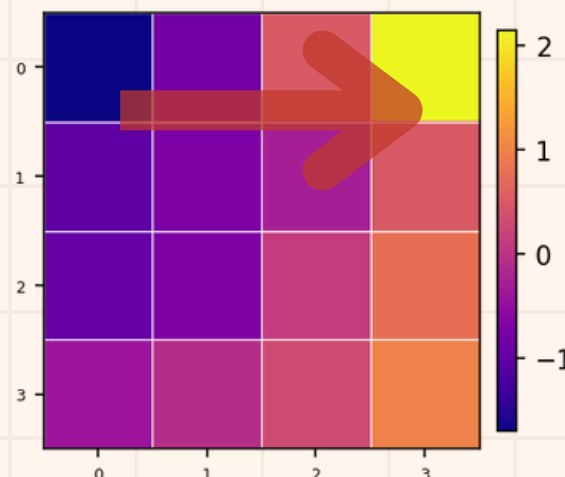
seed: 3



seed: 4



seed: 5



¿Y LOS EJES?

- Variables de bienestar (GDP, LifeExp, PopGrowth) co-varían y forman un eje estable entre semillas
- Inflation tiende al eje opuesto pero con más ruido
- Area es inconsistente entre semillas
- Military, Unemployment no forman gradiente

Las relaciones entre variables se preservan entre semillas; la orientación geométrica del mapa, no.

CONCLUSIONES

Sobre Kohonen y los países

Kohonen proyecta las 7 variables a una grilla 2D donde países con perfiles parecidos quedan en neuronas vecinas.

- Lo que es estable entre semillas: los grupos de países que reaparecen, los outliers, y las relaciones entre variables (qué co-varía, qué es independiente).
- Lo que varía según la semilla: la geometría del mapa (orientación, posiciones absolutas, asignación de los países de perfil ambiguo)

Esto es esperable: Kohonen aprende vecindades, no posiciones absolutas.

2

ANÁLISIS DE **HIPERPARÁMETROS**

CRITERIOS DE SELECCIÓN

¿Cómo encontrar la mejor K / hiperparámetros?

“No hay un criterio demostrado para [elegir el mejor tamaño de grilla]” – Cátedra SIA

MÉTRICAS CUANTITATIVAS

- Tamaño del cluster más grande
- Neuronas muertas
- % singletons
- Máxima distancia entre neuronas
- Quantization Error (**QE**): distancia promedio país–neurona ganadora (mide qué tan bien los pesos se ajustan a los datos)
- Topographic error (**TE**): % de muestras cuyas dos BMU no son vecinas en la grilla (mide preservación de topología)

CRITERIOS CUALITATIVOS

- **Coherencia interna sobre clusters/aislamiento de outliers:** ¿países agrupados comparten perfil económico–social?

**Ver apéndice para fórmulas de QE y TE*

COMPARACIÓN DE TAMAÑO DE GRILLA



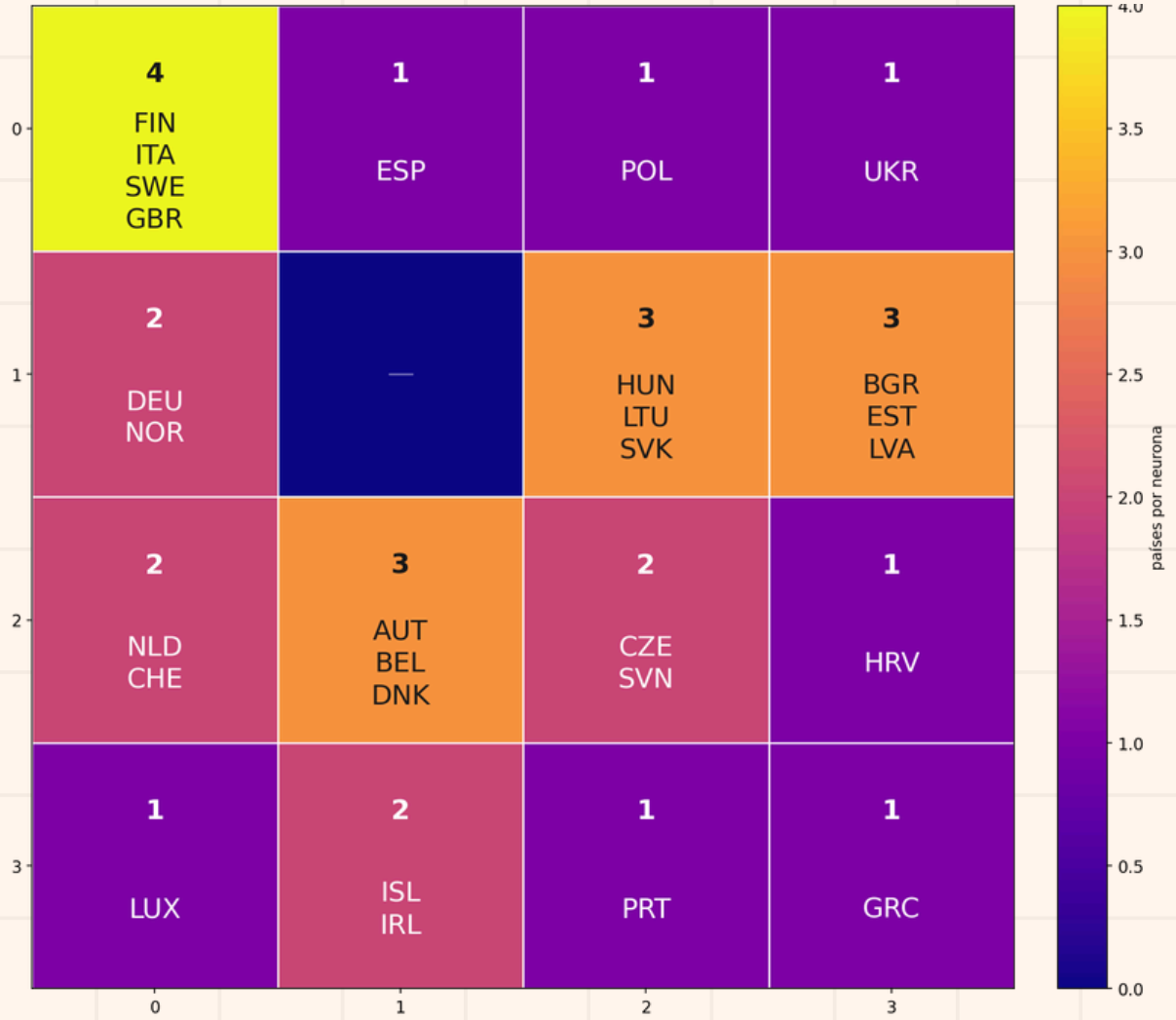
k	??
eta_0	0.1
radius_0	k
n_iter	500×k×k
weight_init	samples sin reemplazo
distance	euclidean
# de seeds	10

3x3



Sub-resuelto

4x4



5x5



Sobre-fragmentado

Métrica	k=3	k=4	k=5
QE	1.14 ± 0.06	0.74 ± 0.05	0.41 ± 0.06
Dead neurons	0.0 ± 0.0	0.7 ± 0.5	4.3 ± 0.9
Max cluster	6.1 ± 0.5	4.1 ± 0.3	2.9 ± 0.3
Singleton %	24% ± 12%	44% ± 6%	69% ± 5%

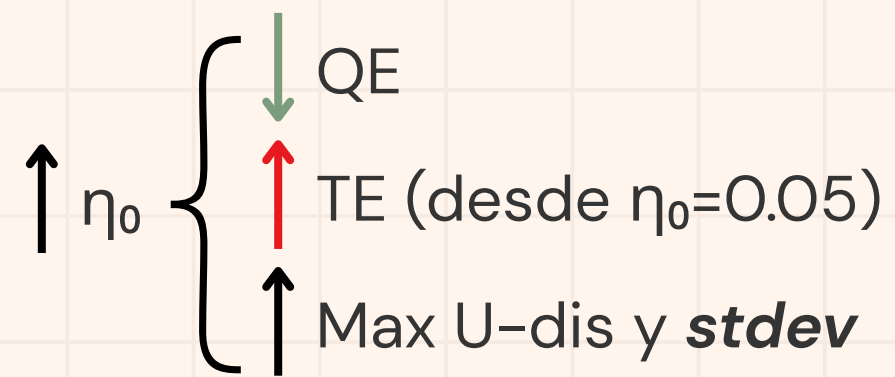
COMPARACIÓN DE **TASA DE APRENDIZAJE INICIAL**



k	4
eta_0	??
radius_0	k
n_iter	500×k×k
weight_init	samples sin reemplazo
distance	euclidean
# de seeds	10

COMPARACIÓN DE TASA DE APRENDIZAJE INICIAL

Métrica / η_0	0.01	0.05	0.1	0.5	0.9
QE	0.76 ± 0.05	0.77 ± 0.06	0.74 ± 0.05	0.56 ± 0.06	0.57 ± 0.05
TE	0.10 ± 0.08	0.08 ± 0.04	0.12 ± 0.08	0.16 ± 0.08	0.20 ± 0.05
Max U-dist	4.35 ± 0.21	4.37 ± 0.11	4.53 ± 0.15	4.76 ± 0.49	5.41 ± 0.62
Neuronas muertas	0.8 ± 0.7	0.8 ± 0.6	0.7 ± 0.5	0.4 ± 0.7	0.5 ± 0.8



Neuronas muertas ruidosa (*stdev* alta en todas) → no es buen criterio para elegir η_0

Dos regímenes:

- $\eta_0 \leq 0.1$ preserva topología (TE bajo, Max U-dist bajo)
- $\eta_0 \geq 0.5$ mejora QE pero rompe topología

Elegimos $\eta_0=0.05$: mínimo TE, dentro del régimen de topología preservada

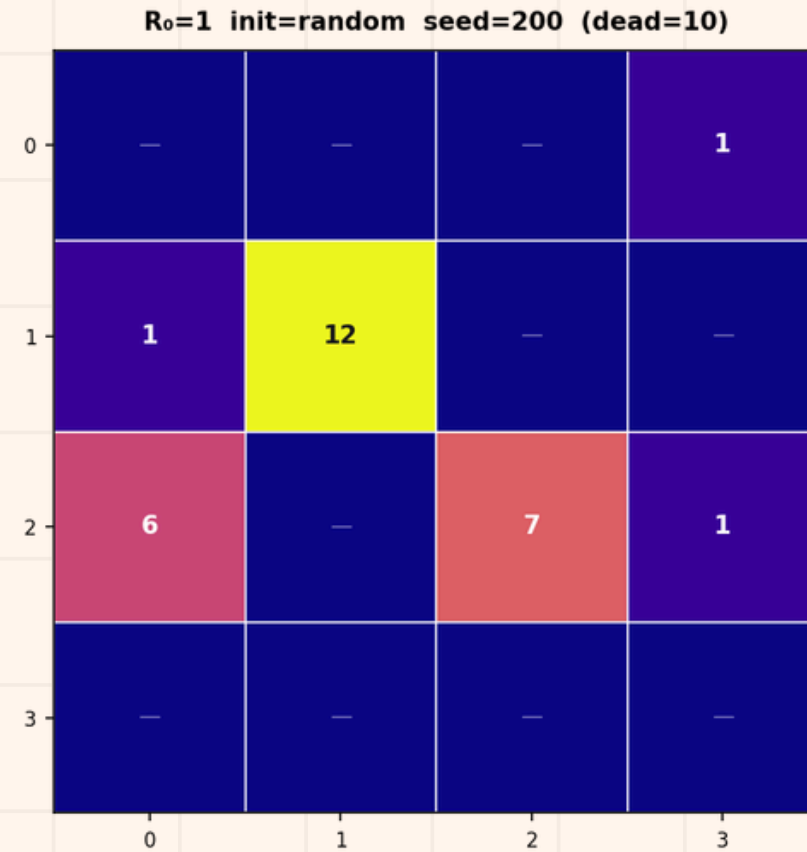
COMPARACIÓN DE INICIALIZACIÓN DE PESOS Y RADIO INICIAL



```
k      4
eta_0   0.05
radius_0 ??
n_iter  500×k×k
weight_init ??
distance euclidean
# de seeds 10
```

COMPARACIÓN DE INICIALIZACIÓN Y RADIO INICIAL

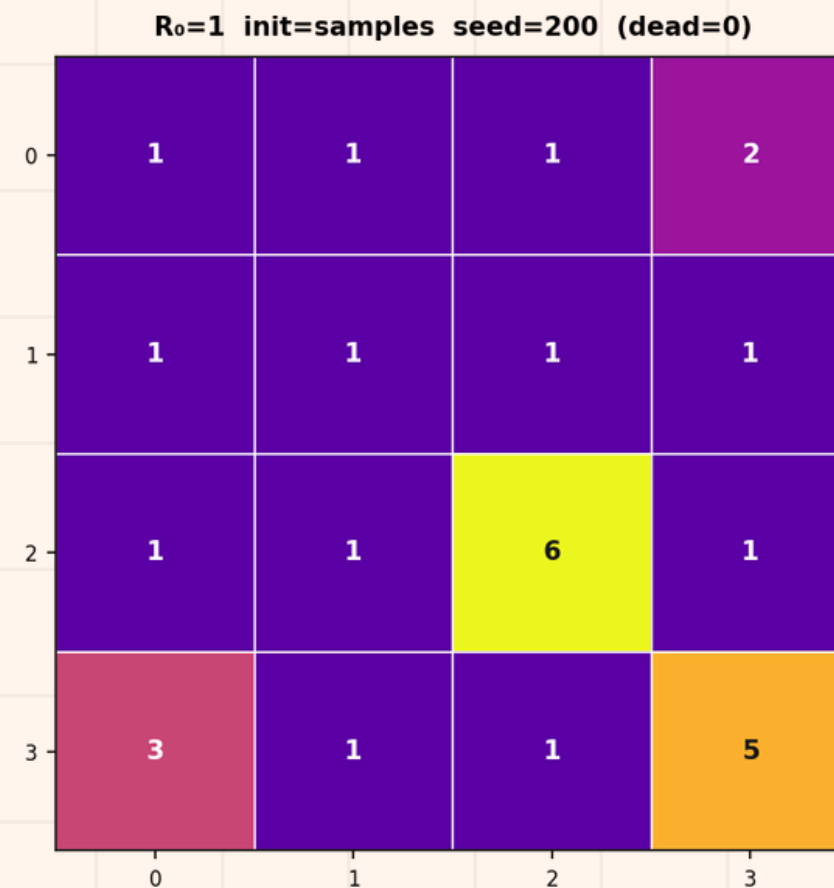
Random



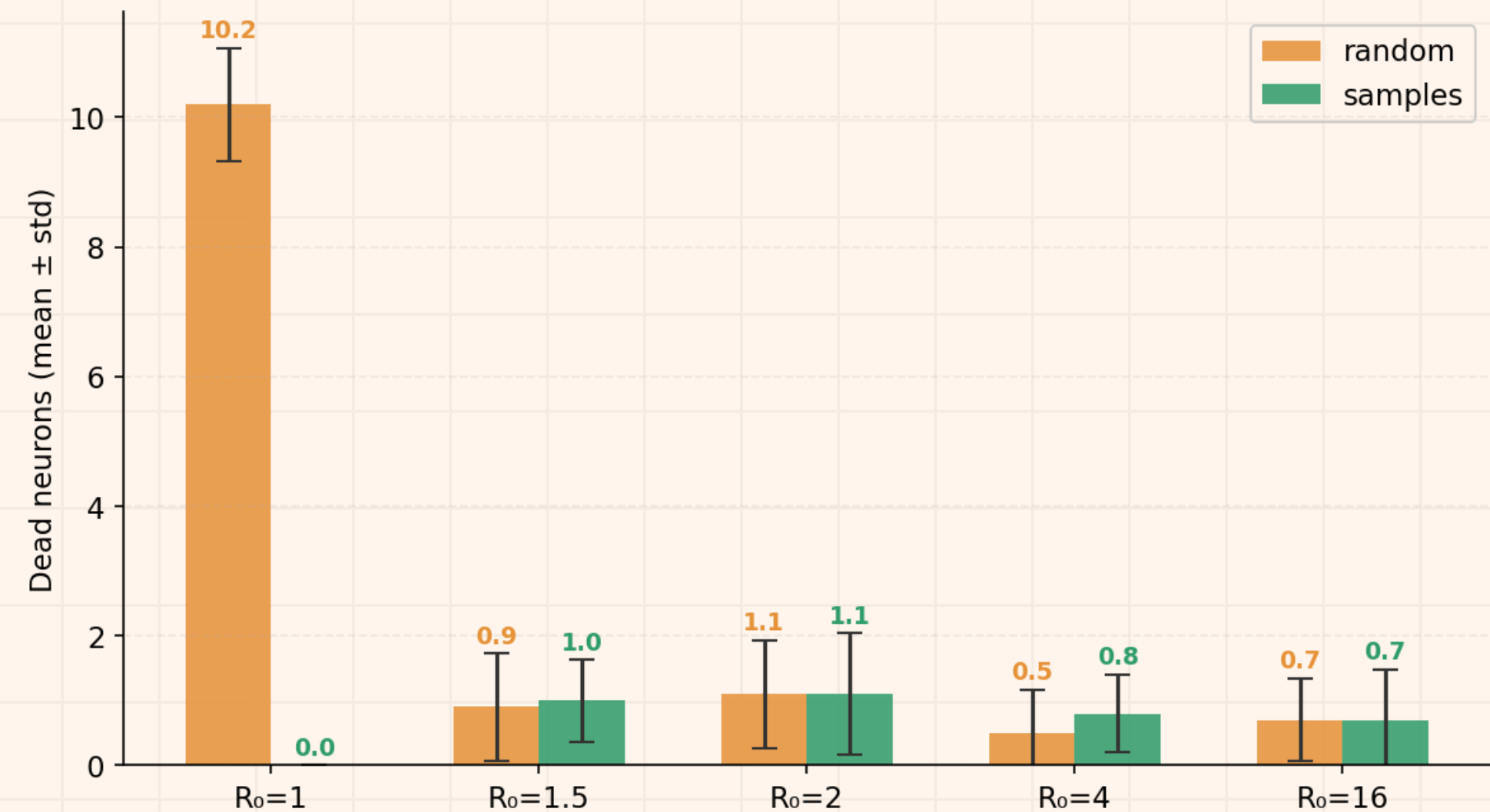
- Una neurona absorbe mayoría de países
- Unidades que arrancan lejos de los datos nunca ganan y mueren

0 neuronas muertas
Cada peso inicial ya es un país y la neurona final se queda con su muestra

Samples



Dead neurons vs R_0 — $k=4$, $\eta_0=0.05$, $n_{\text{iter}}=8000$, haykin
Mean $\pm$ std over seeds [1, 2, 3, 4, 5, 6, 42, 69, 100, 200]



COMPARACIÓN DE INICIALIZACIÓN Y RADIO INICIAL

Samples

Métrica / R_0	$R_0=1$	$R_0=1.5$	$R_0=2$	$R_0=4$	$R_0=16$
QE	0.64 ± 0.04	0.76 ± 0.06	0.77 ± 0.07	0.77 ± 0.06	0.74 ± 0.05
TE	0.69 ± 0.14	0.19 ± 0.09	0.11 ± 0.08	0.08 ± 0.04	0.05 ± 0.04
Neuronas muertas	0.0 ± 0.0	1.0 ± 0.6	1.1 ± 0.9	0.8 ± 0.6	0.7 ± 0.8
Max U-dist	6.36 ± 0.72	4.60 ± 0.37	4.48 ± 0.21	4.37 ± 0.10	4.28 ± 0.08

- $R_0=1$ rompe topología (TE alto)
- A mayor R_0 , menor TE
- QE y muertas no discriminan

Schedule exponencial converge a $R=1$ hacia $i \approx 1000$. Al entrenar 8000 iteraciones, la mayoría del entrenamiento es idéntico entre $R_0 \geq 1.5$, lo que explica la zona robusta

**Ver apéndice para la tabla de Random y Samples para los distintos radios, y para ver los Hit Maps / U-matrix (seed 6)*

CONCLUSIONES

Sobre los hiperparámetros

- **Tamaño:** 3×3 comprime países heterogéneos en una misma neurona, 5×5 sobre-fragmenta (singletons 68%). 4×4 equilibra separación, uso de neuronas y cohesión.
- **Learning rate:** con η_0 alto el QE mejora pero TE empeora (el mapa ajusta mejor los pesos individuales a costa de romper la preservación de topología). Se elige $\eta_0=0.05$ que tiene el mejor balance.
- **Radio inicial e Inicialización:** $R_0=1$ falla en ambas inicializaciones: *random* colapsa (mayoría de neuronas muertas), *samples* no preserva topología. A partir de $R_0=1.5$ zona robusta, donde *random* y *samples* son equivalentes.

OJA

Red neuronal con identificación de
componentes principales



TEORÍA

$$X \in R^{N \times p}$$

N=28 países
p=7 variables

Red neuronal

$$O^\mu = \theta\left(\sum_{i=1}^n x_i^\mu w_i^\mu\right)$$

$$x_i \in R^p$$

Vector de características del país i,
con media 0 (estandarizado).

Actualización de pesos

$$w(n+1) = w(n) + \Delta w$$

Hebb puro:

$$\Delta w = \eta O x_i^n$$

Oja

$$\Delta w = \eta \left(\underbrace{O x_i^n}_{\text{Término Hebbiano}} - \underbrace{O^2 w_i^n}_{\text{Término correctivo}} \right)$$

IMPLEMENTACIÓN & HIPERPARÁMETROS

Tasa de aprendizaje

$$\eta(t) = \eta_0 / (1 + \text{decay} \times t)$$

MSE

Se toma la diferencia cuadrática media entre los scores de los países usando la librería y los obtenidos con Oja

$$MSE = (1/N) \cdot \Sigma(\text{score}_{oja(i)} - \text{score}_{sklearn(i)})^2$$

Var

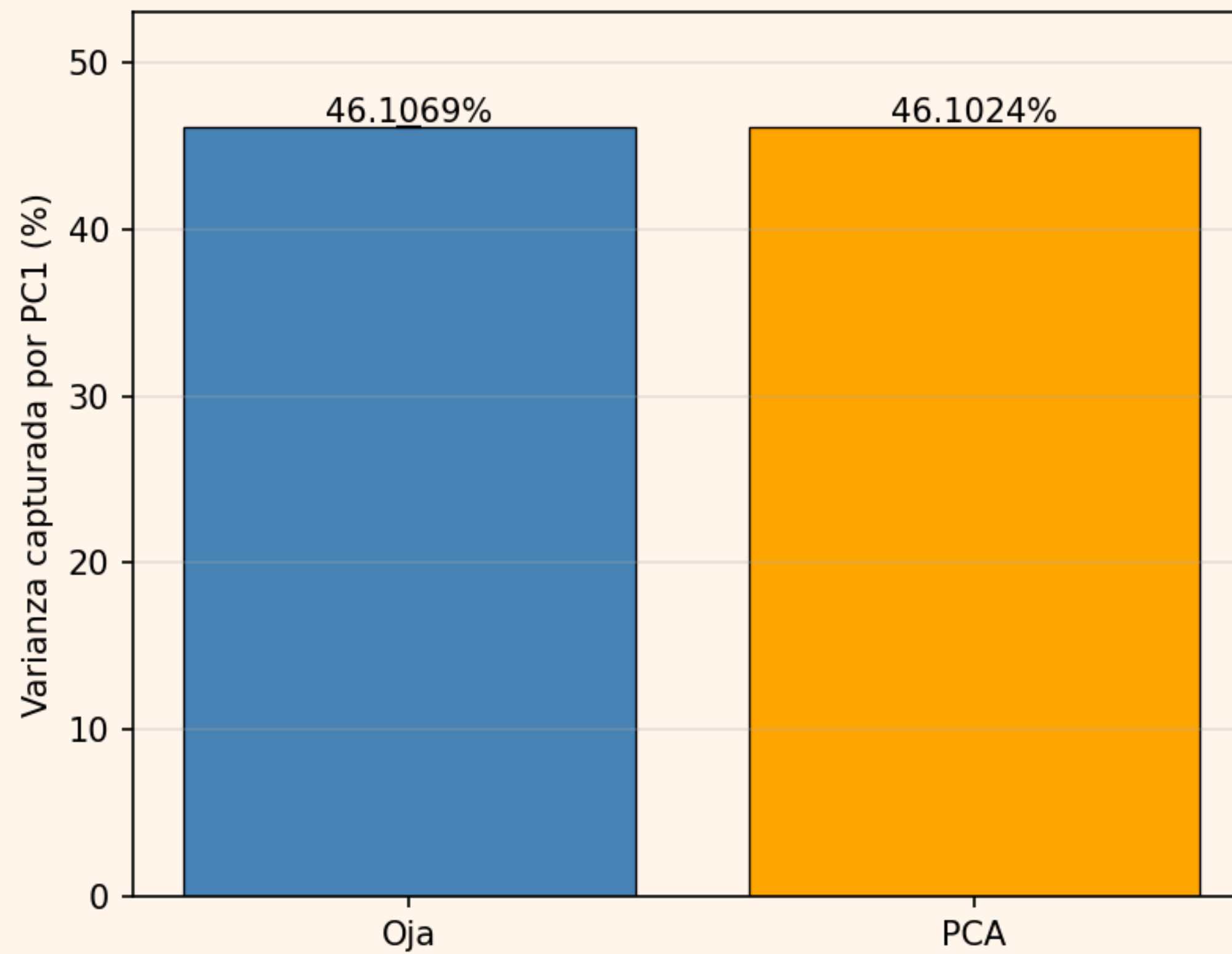
$$\text{Var}(X \cdot w) = (1/N) \cdot \Sigma(\text{score}_i)^2 = w^T C w = \lambda_1$$

si Oja converge bien, $w = v_1$ y la varianza de los scores da λ_1

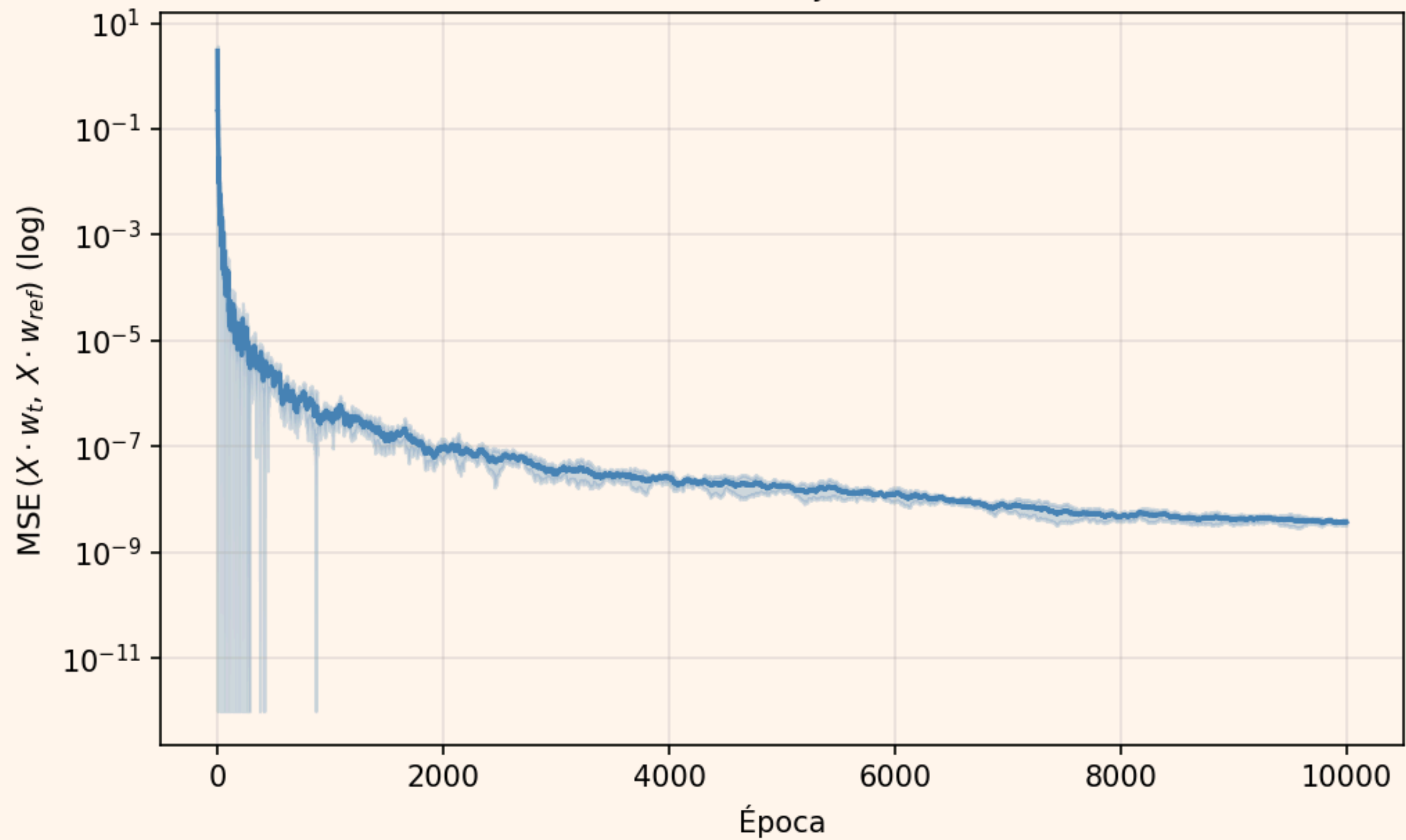


```
eta_0    0.1
decay    0.01
epochs   10000
weight_init  U(0,1)
shuffle   true
normalized_w false
seeds     5
```


Varianza Capturada: Oja vs PCA

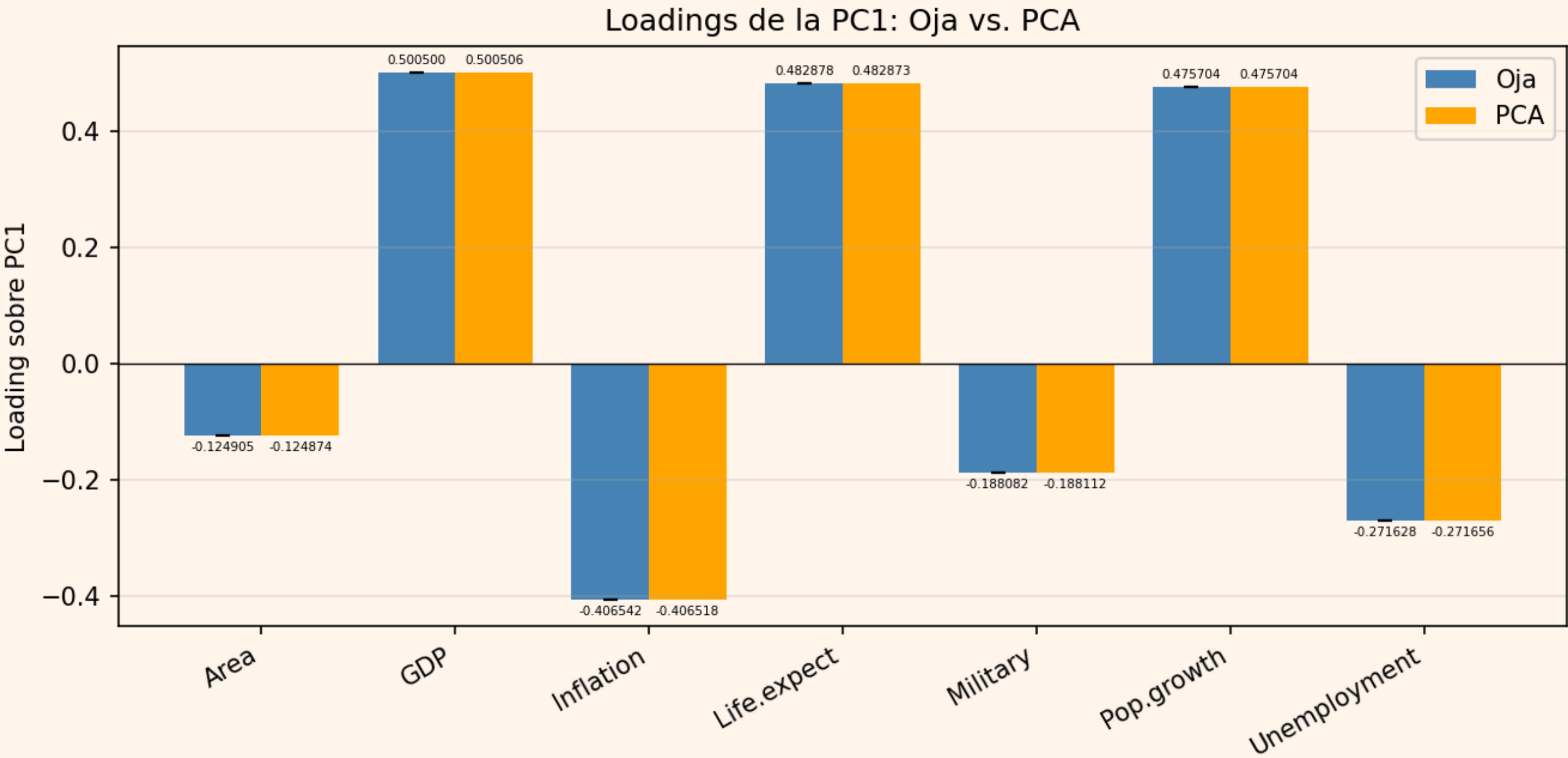


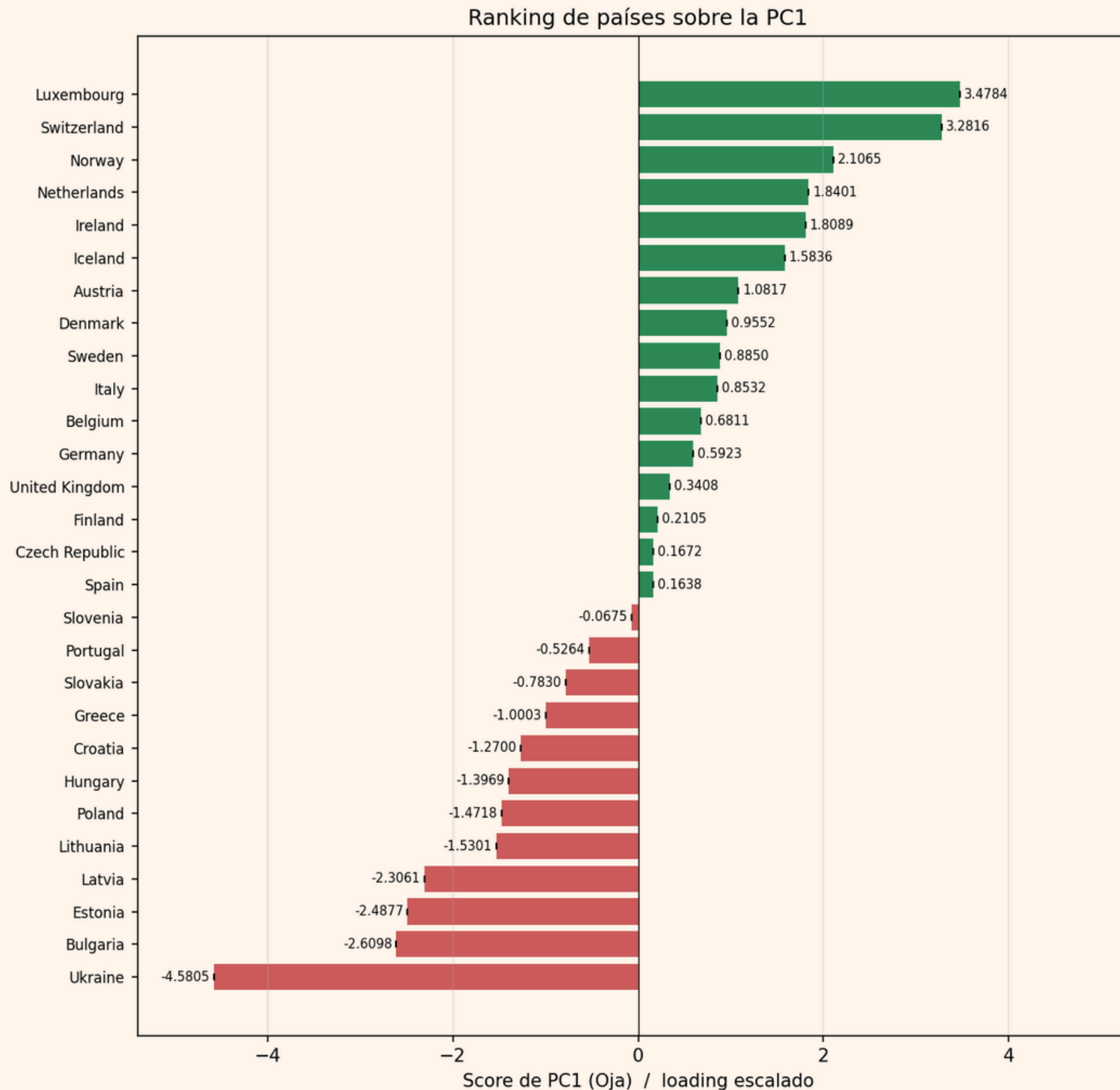
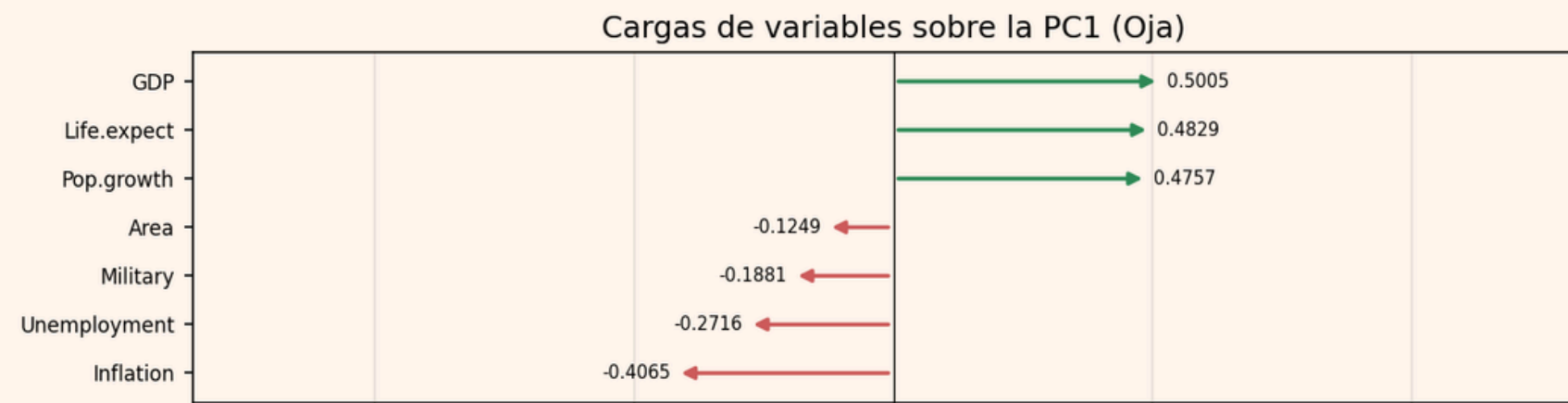
MSD de scores Oja vs. sklearn



Varianza Capturada: 46.1069% (Oja) vs 46.1024% (librería PCA)

Países: $MSE= 1.17 \times 10^{-8}$ $R^2=1.0000$





PC1 es un índice de **prosperidad económica y calidad de vida**.

3 variables con carga positiva fuerte:

- GDP (0.50)
- Expectativa de vida (0.48)
- Crecimiento poblacional (0.48).

Las cargas negativas más importantes:

- Inflación (-0.41)
- Desempleo (-0.27)

Area y Military pesan poco.

PC1 capturando el 46% de toda la variabilidad del dataset.

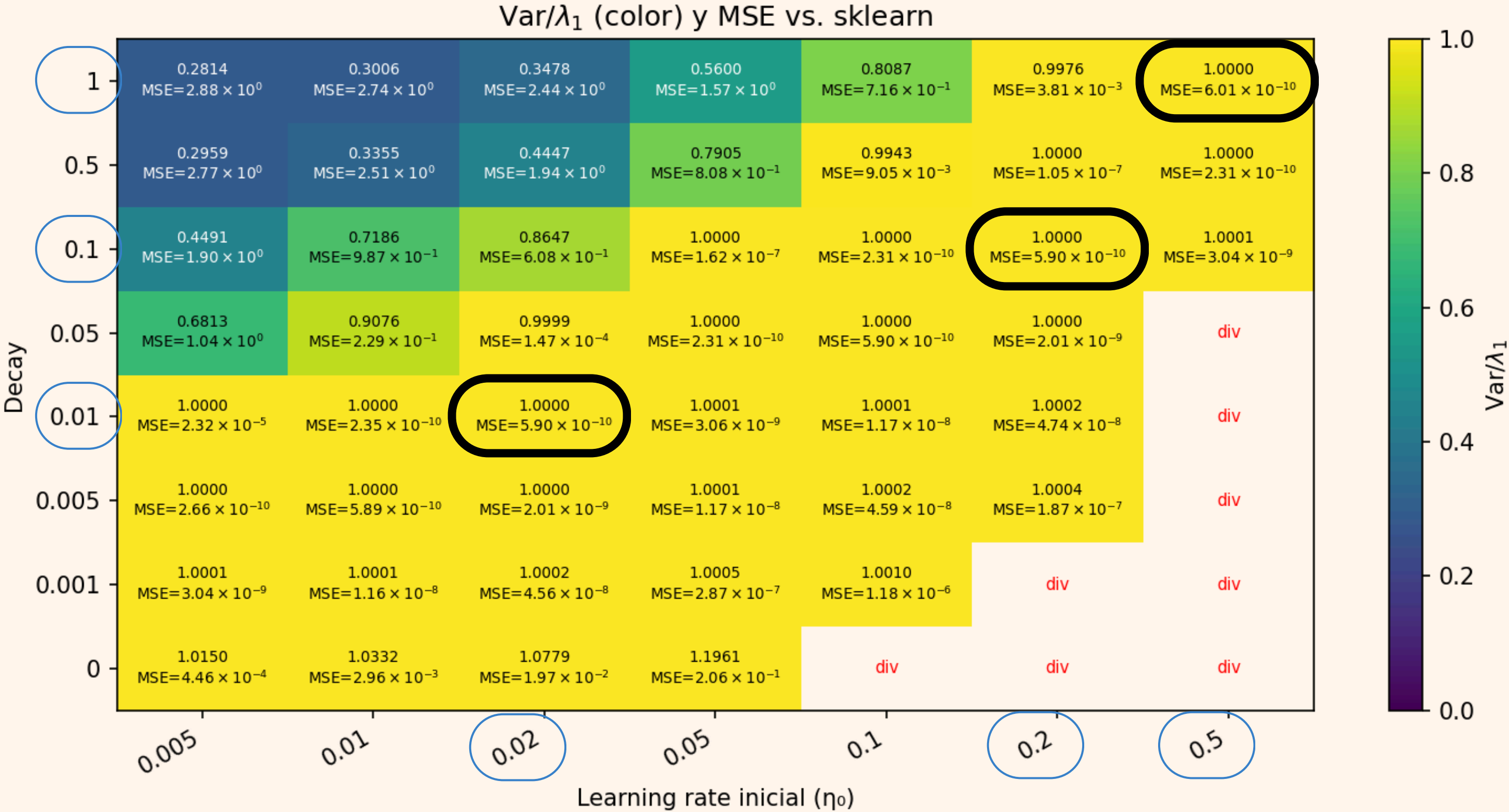
La PC1 separa a los países en tres bloques según su prosperidad socioeconómica:

- Más desarrollados (Luxemburgo, Suiza, Noruega, etc.)
- Intermedios cercanos a 0 (Bélgica a Eslovaquia)
- Menor desarrollo con scores negativos.
- Ucrania aparece como outlier extremo (-4.58).

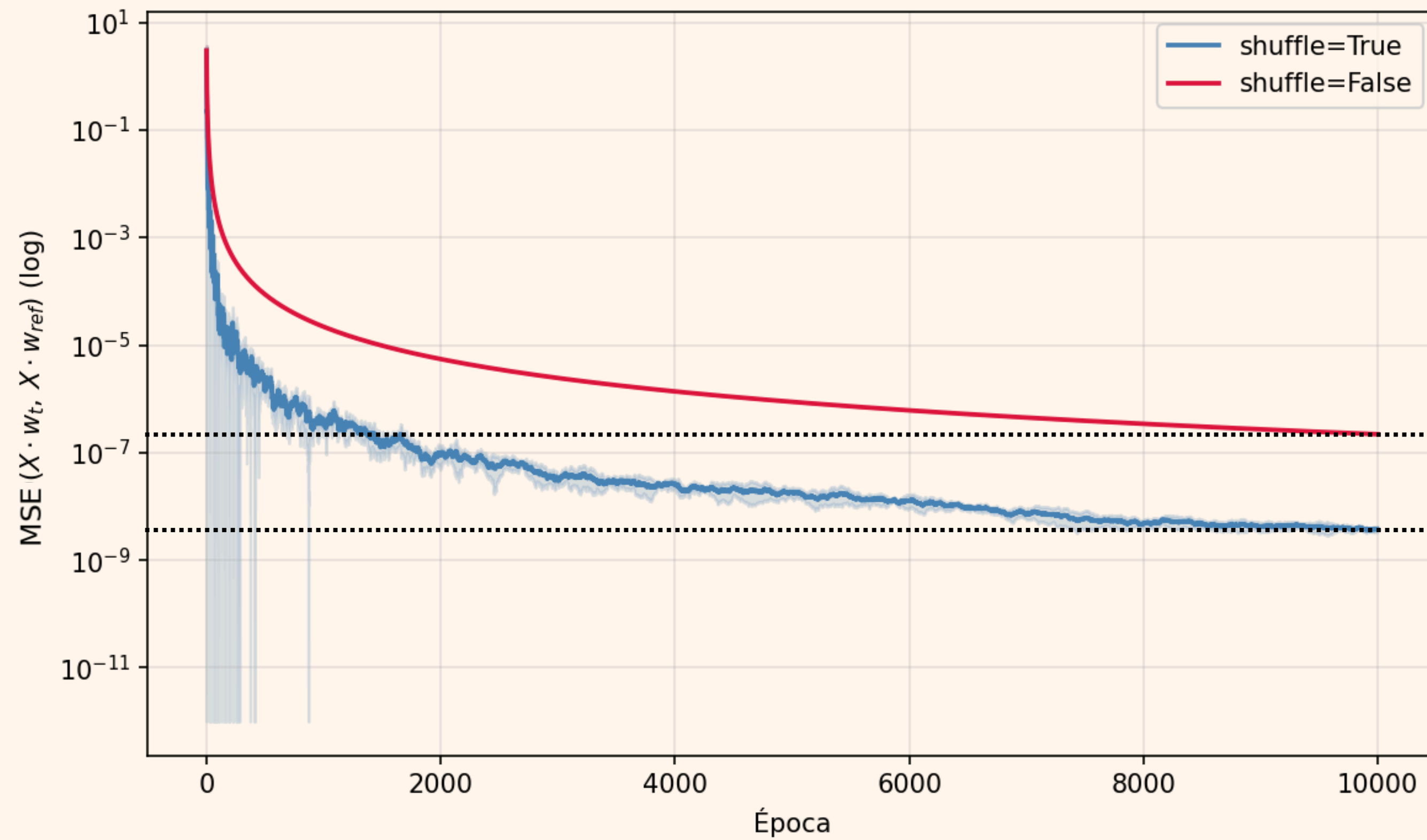
$\lambda_1 = 3.3467$

$1/(1.2 \times \lambda_1) = 0.249$

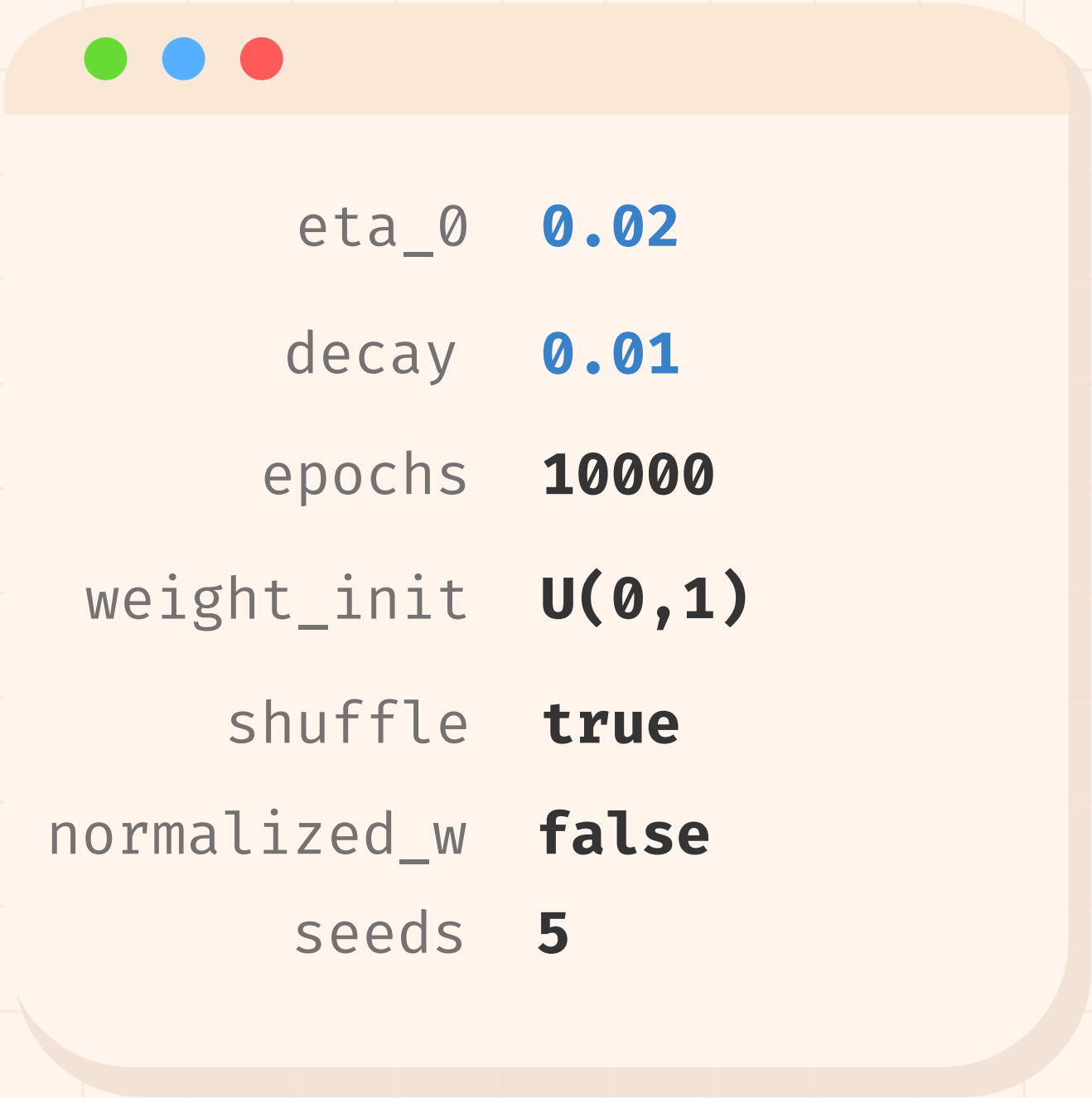
si $\eta(t) > 1/1.2 \times \lambda_1 \approx 0.25$ inestable



Efecto del shuffle de muestras



NUEVOS HIPERPARÁMETROS



eta_0 **0.02**

decay **0.01**

epochs **10000**

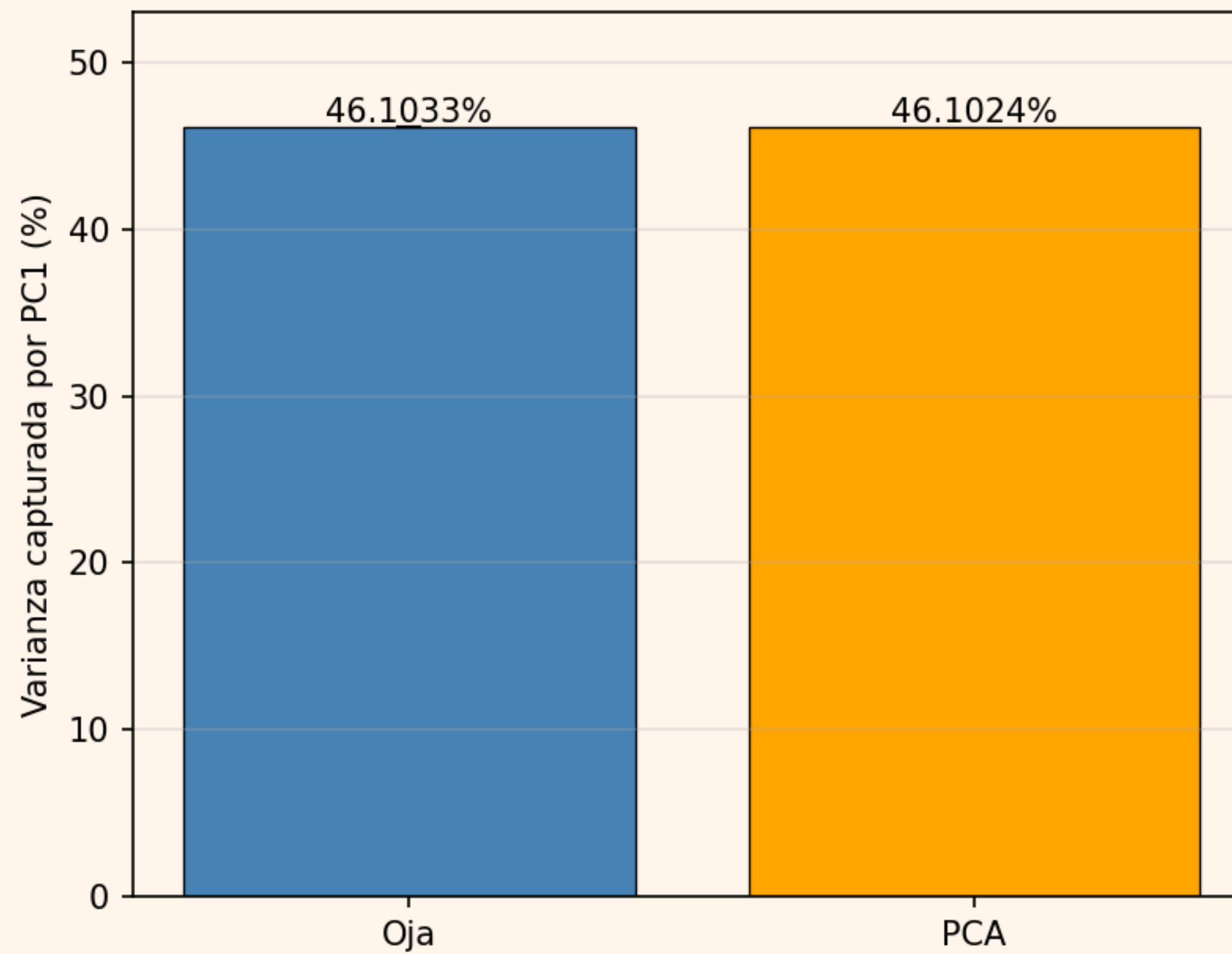
weight_init **U(0,1)**

shuffle **true**

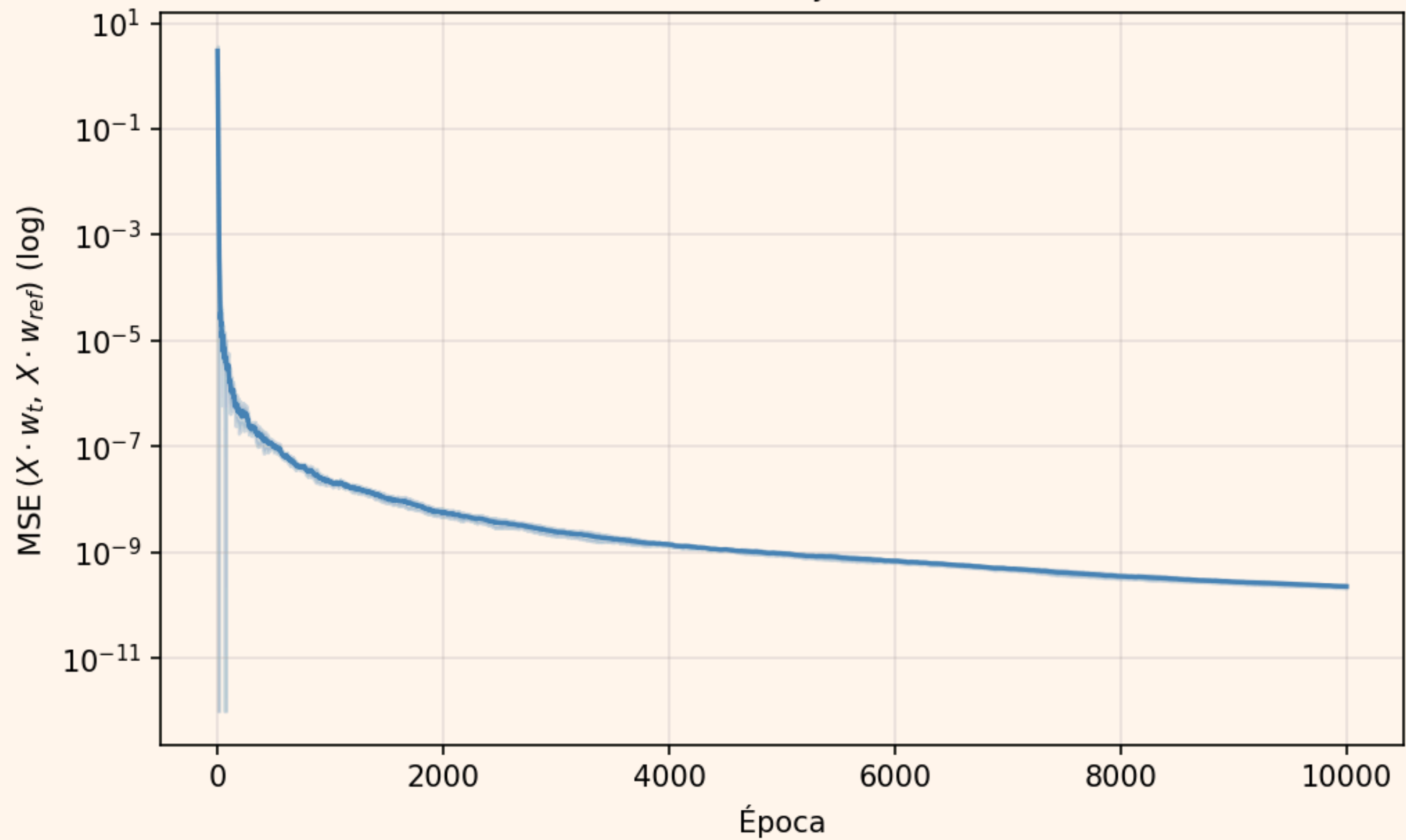
normalized_w **false**

seeds **5**

Varianza Capturada: Oja vs PCA

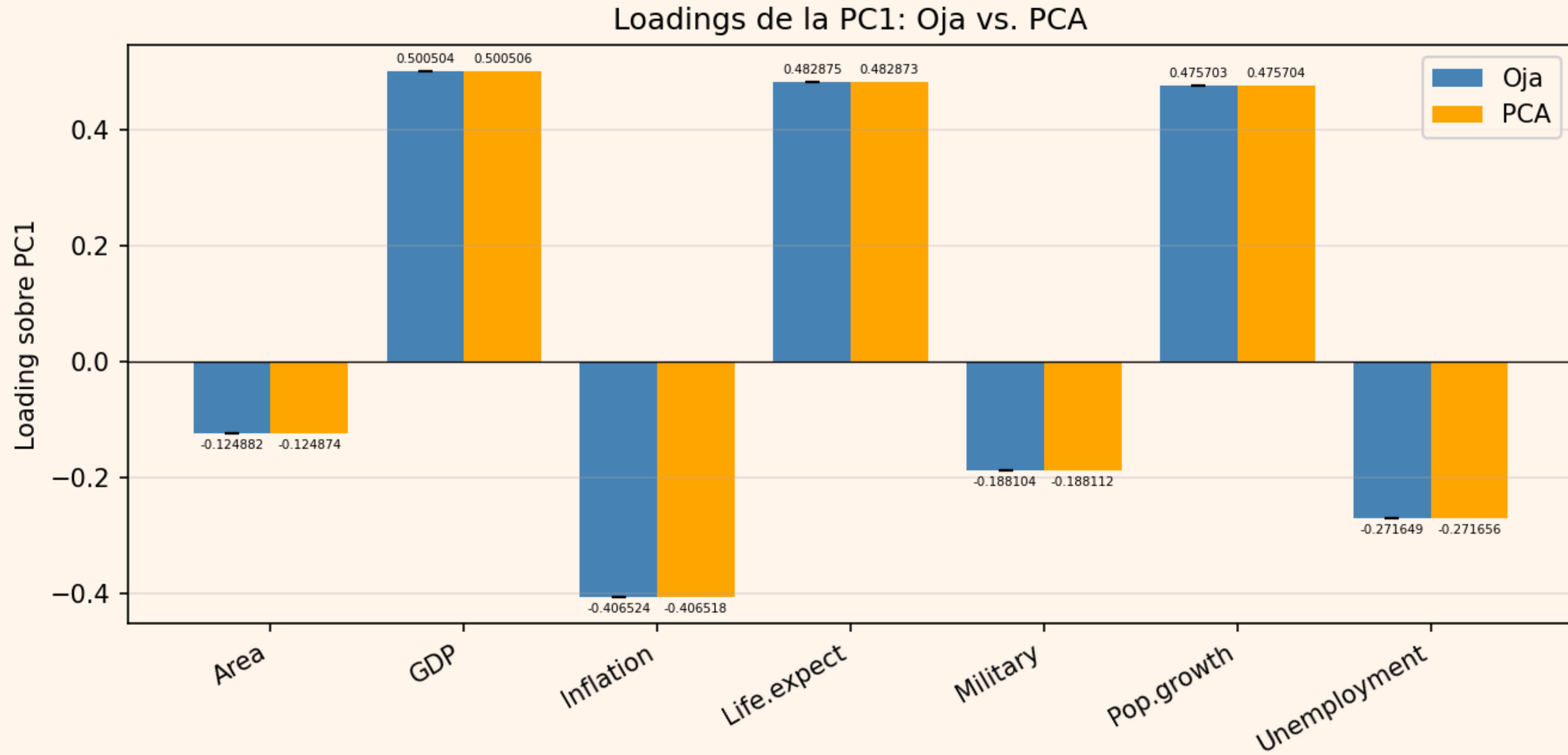


MSD de scores Oja vs. sklearn

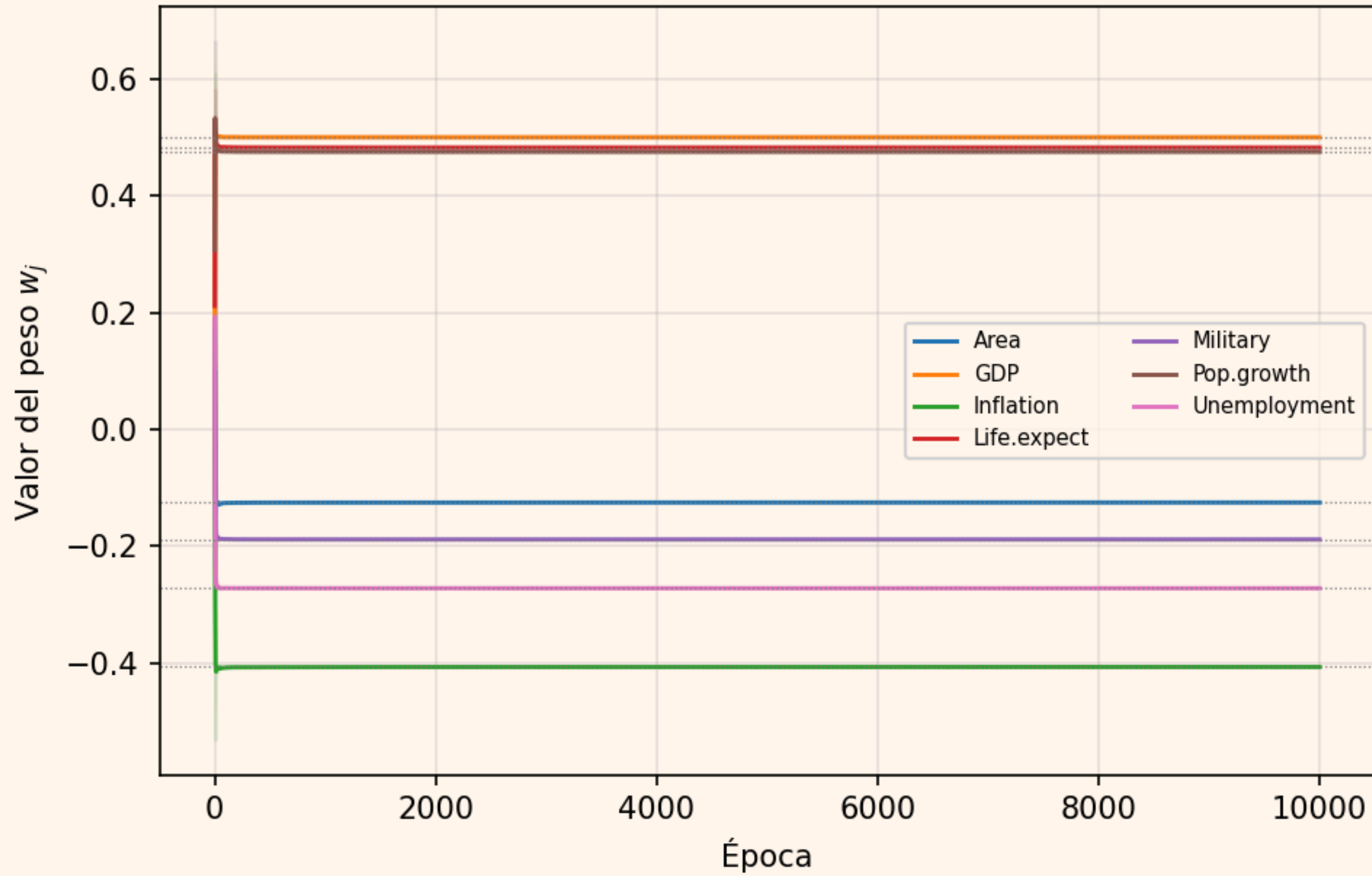


Varianza Capturada: **46.1033% (Oja)** vs. **46.1024% (librería PCA)**

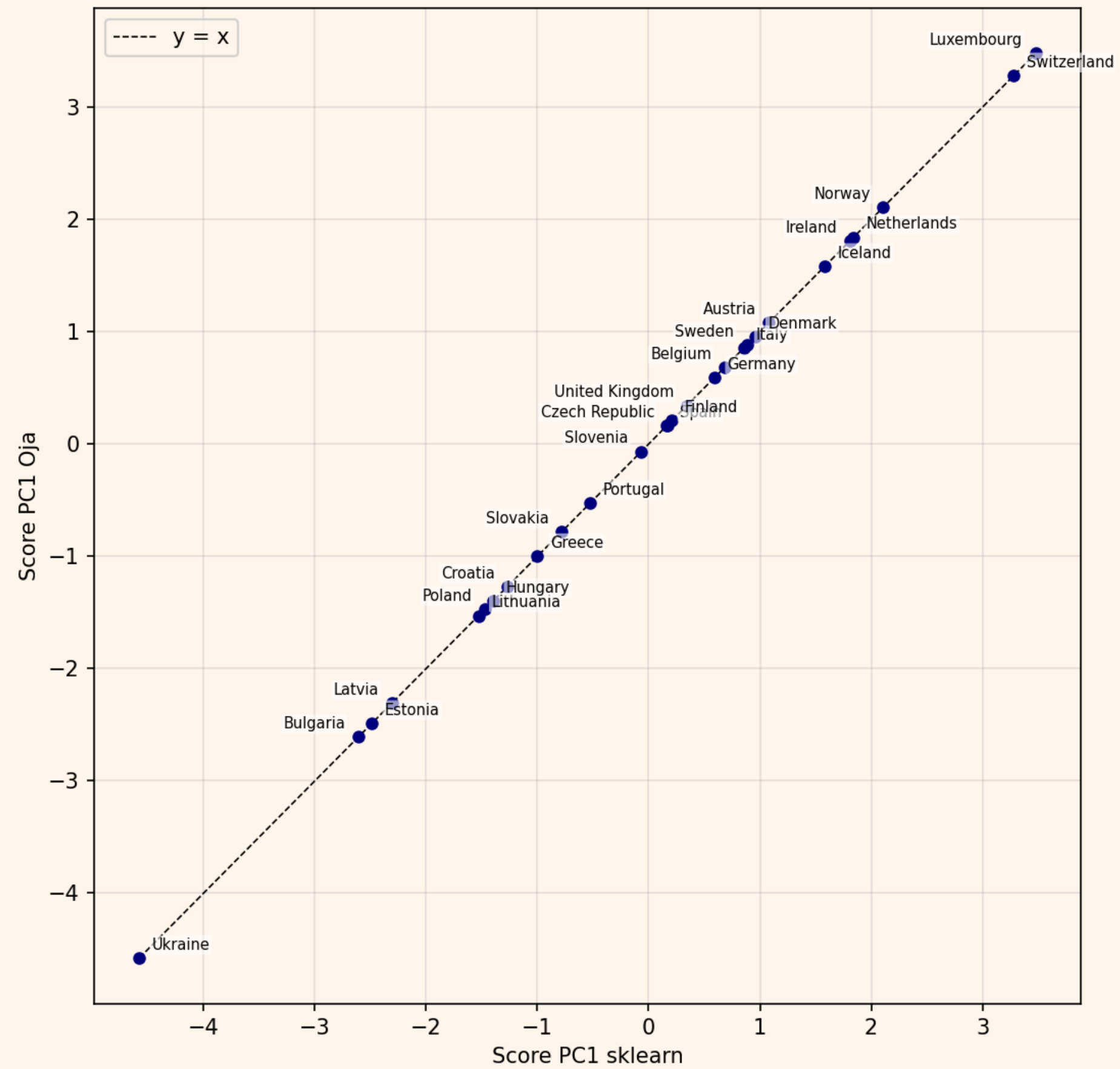
Países: **MSE= 5.9×10^{-10} $R^2=1.0000$**



Convergencia de los pesos de Oja

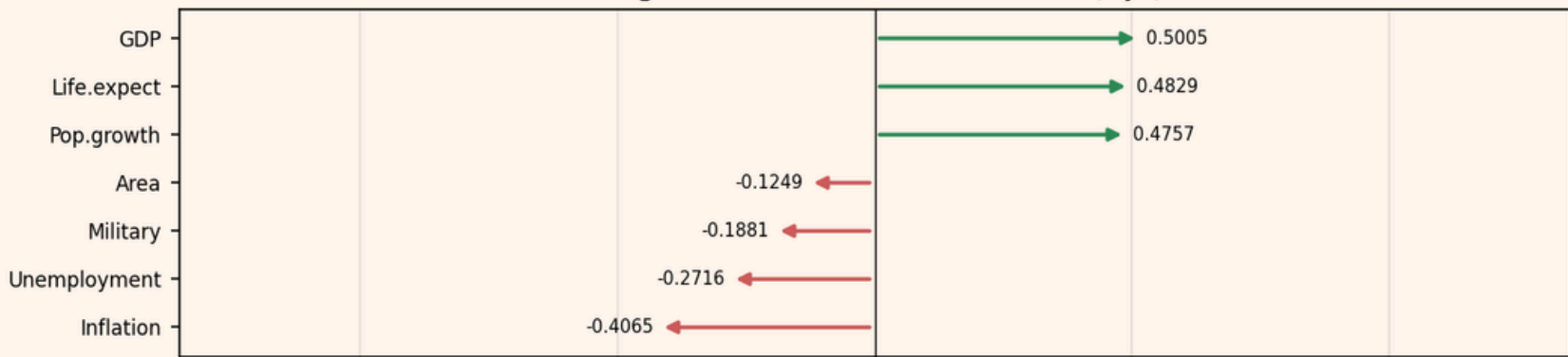


Scores Oja vs. sklearn (MSE = 5.90×10^{-10} , $R^2=1.0000$)

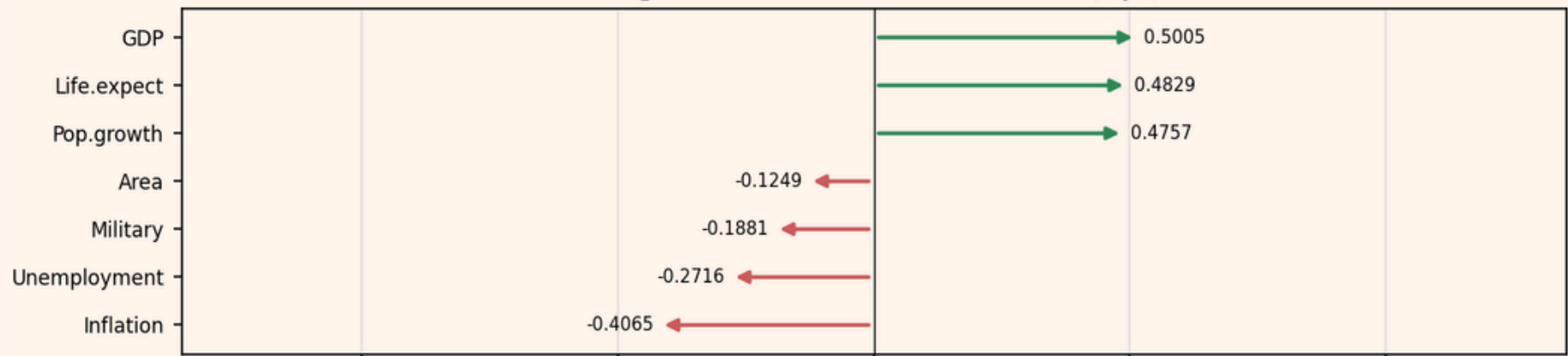


+

Cargas de variables sobre la PC1 (Oja)

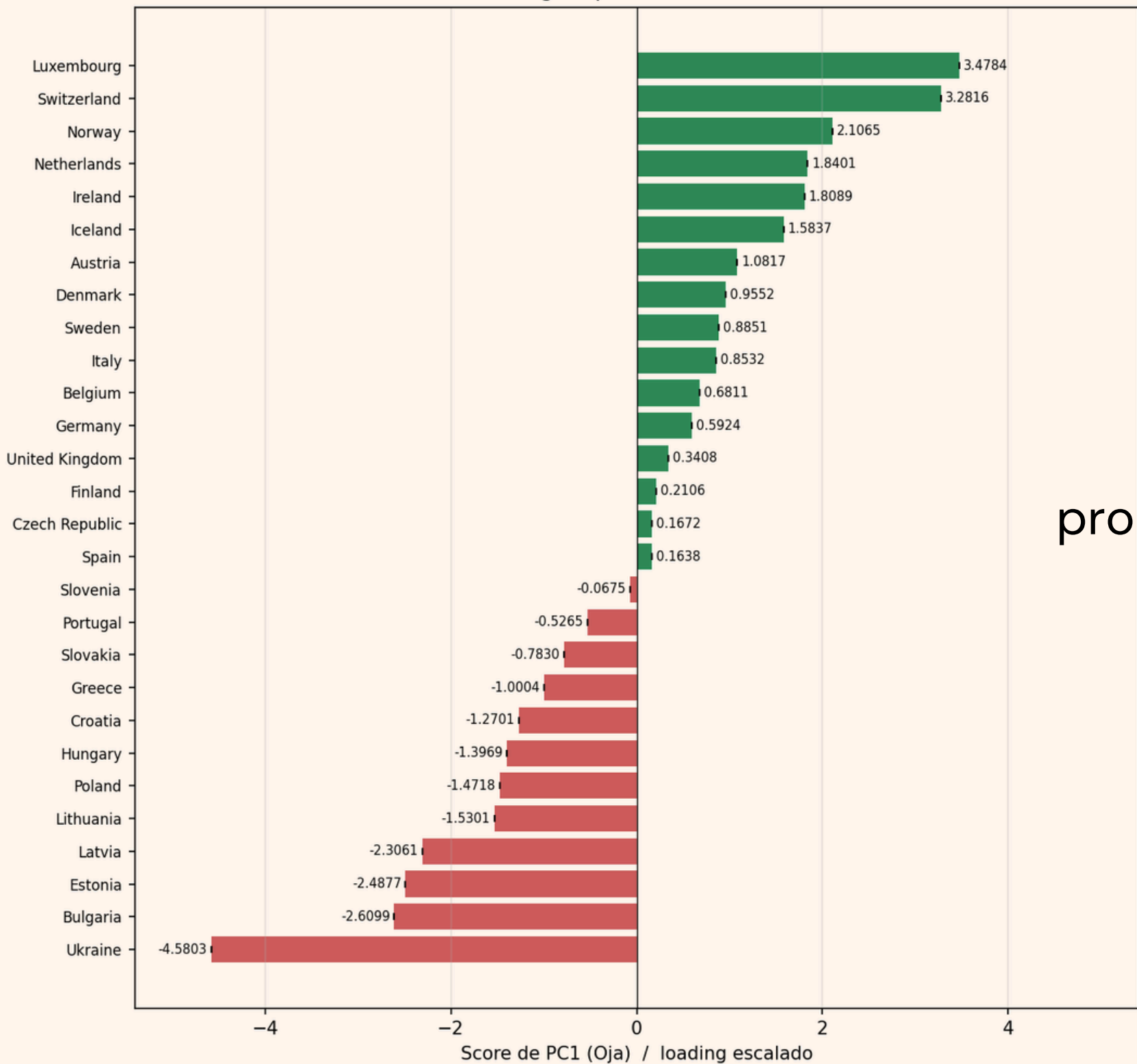


Cargas de variables sobre la PC1 (Oja)



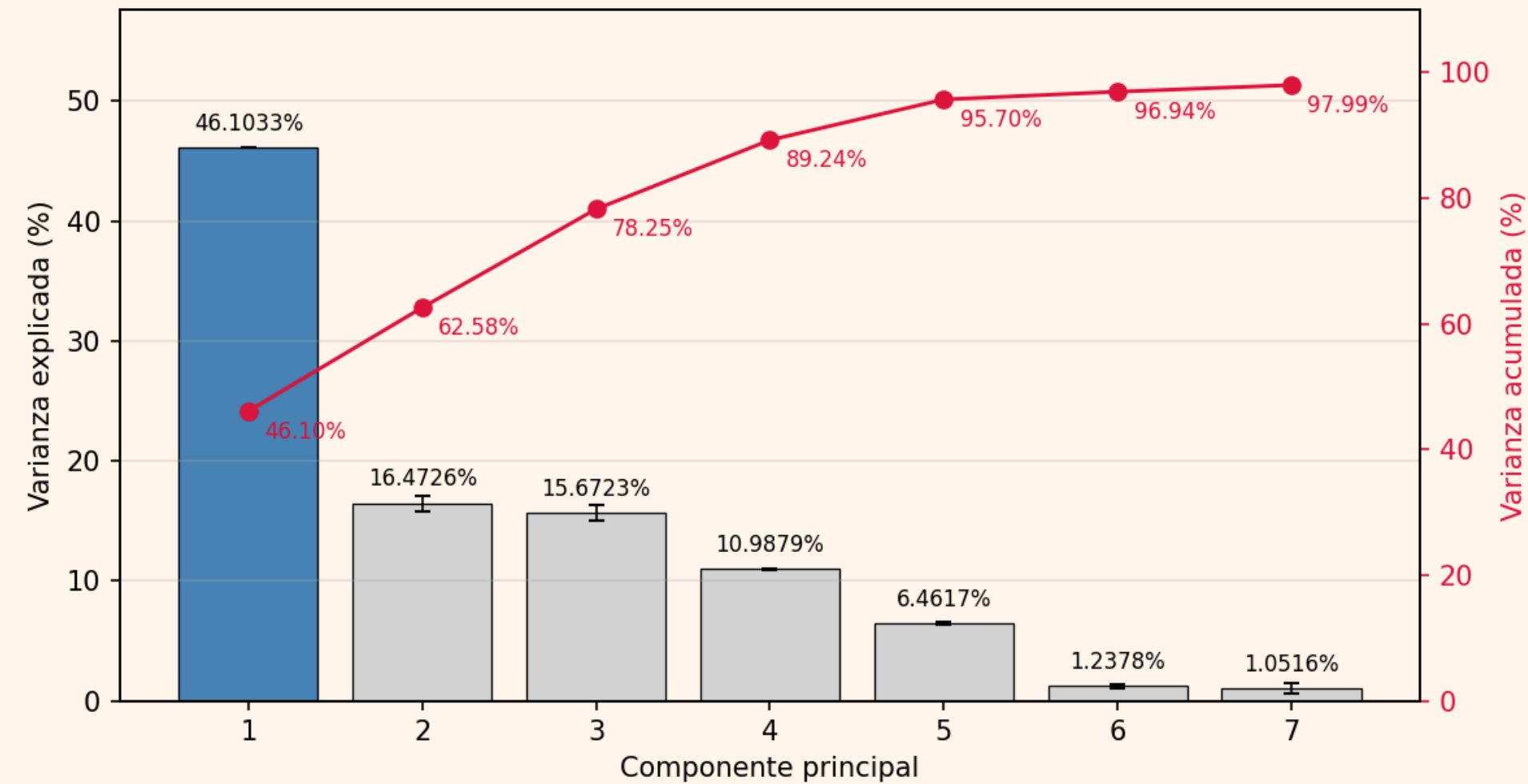
-

Ranking de países sobre la PC1



prosperidad

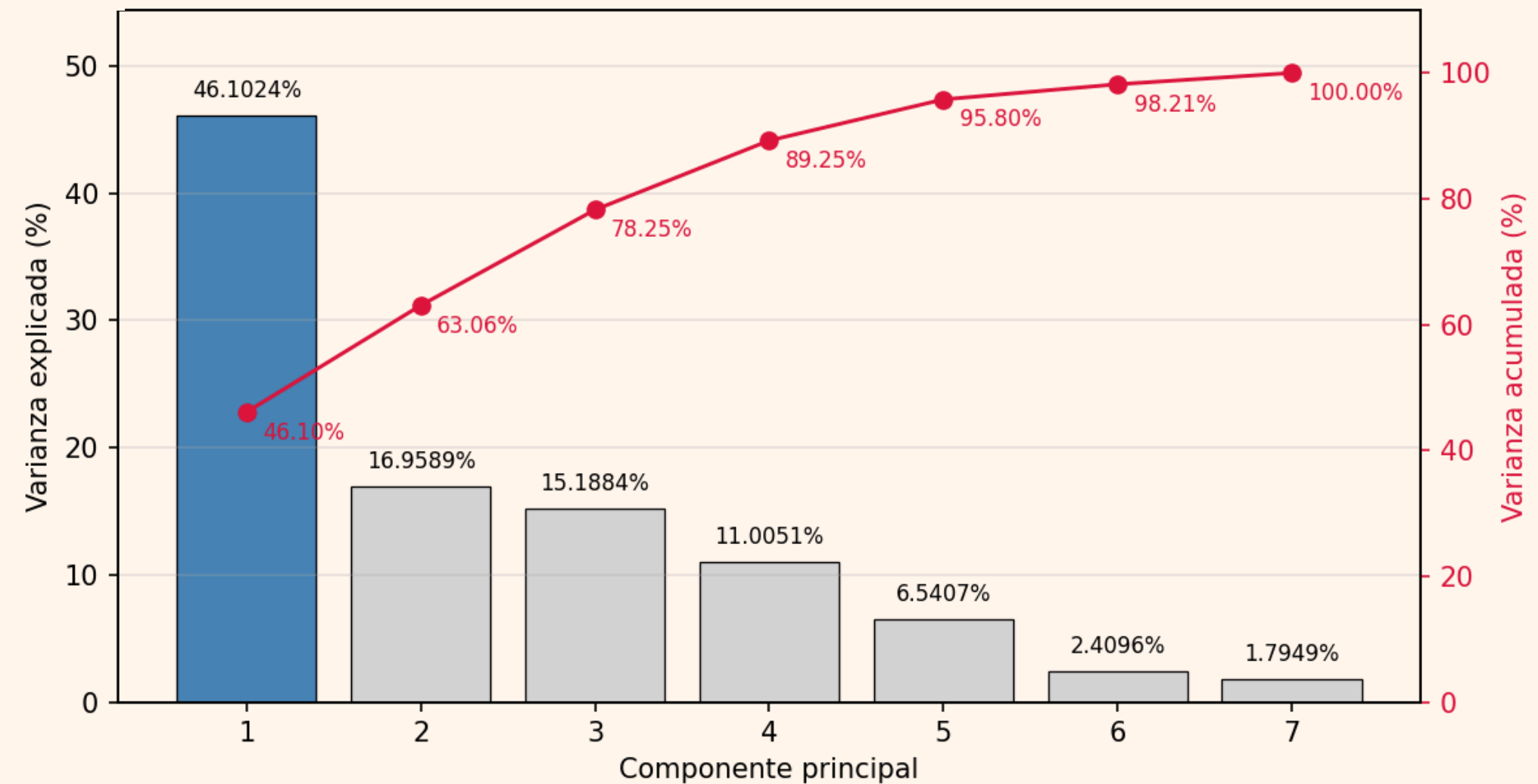
Componentes principales y varianza explicada (Sanger)



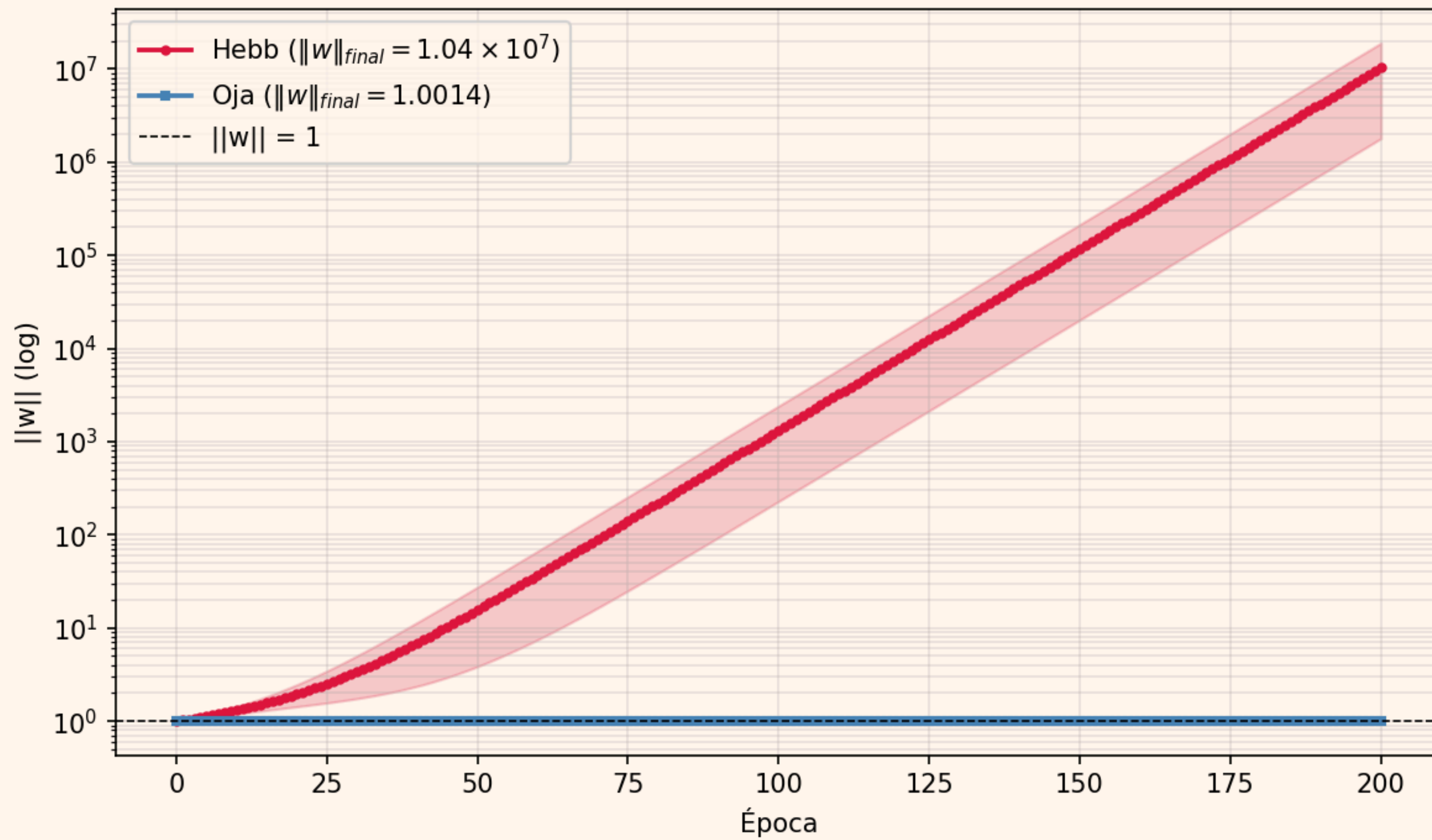
Sanger funciona muy bien para las primeras componentes, pero la acumulación de error degrada la calidad de las componentes posteriores.

Limitación conocida del método secuencial respecto a PCA clásico, que descompone la matriz de covarianza de una sola vez y no tiene este problema.

Componentes principales y varianza explicada



Hebb vs Oja



CONCLUSIONES

- Oja logra recuperar la PC1 con una precisión prácticamente exacta respecto a la solución analítica de *sklearn*.
 - Con los hiperparámetros optimizados ($\eta_0=0.02$, $\text{decay}=0.01$), se obtiene un MSE de 5.9×10^{-10} y un $R^2=1$ entre los scores de Oja y los de PCA, demuestra que el método neuronal converge al autovector dominante de la matriz de covarianza.
 - La varianza capturada también coincide: 46.1033% (Oja) vs 46.1024% (PCA).
 - El learning rate inicial tiene un límite teórico de estabilidad: cuando $\eta(t) > 1/(1.2 \cdot \lambda_1) \approx 0.25$, el algoritmo puede divergir.
 - Hacer un shuffle de las muestras en cada época mejora la convergencia en 2 órdenes de magnitud.
-
- La PC1 funciona como un índice de prosperidad socioeconómica. Las tres variables con carga positiva fuerte son GDP (0.50), expectativa de vida (0.48) y crecimiento poblacional (0.48), mientras que inflación (-0.41) y desempleo (-0.27) cargan negativamente.
 - La norma de los pesos con Hebb puro crece exponencialmente hasta $\sim 10^7$ en 200 épocas, mientras que Oja queda en ≈ 1.0014 .
 - Sanger recupera las 7 componentes principales. Las primeras 4 acumulan $\sim 90\%$ de la varianza, por lo que el dataset europeo tiene una dimensionalidad ≈ 4 .

Oja demuestra que un perceptrón simple con aprendizaje hebbiano es capaz de extraer la componente principal de un dataset de forma online, sin calcular la matriz de covarianza ni los autovectores.

CONCLUSIONES

KOHONEN vs. OJA

Similitudes:

- Eje principal de Oja (PC1) coincide con el eje dominante que Kohonen organiza en la grilla: bienestar (GDP, LifeExp, PopGrowth) con Inflation en sentido opuesto.
- Ucrania aparece como extremo en ambos: valor más negativo de PC1 en Oja, país consistentemente solo en una neurona y valor más alto en la U-matrix (4x4)

Diferencias:

- Oja produce un ordenamiento lineal: cada país recibe un valor escalar sobre PC1
 - Kohonen produce una partición: cada país se asigna a una neurona, y los que comparten neurona son tratados como equivalentes
- Oja permite cuantificar diferencias entre países; Kohonen permite identificar grupos discretos pero no diferenciar dentro de cada uno.

WOLFELD

ESTRATEGIA

1

Letras

Abecedario y patrones elegidos

2

Paso a paso

Encontrando la estructura de una letra

3

Ruido

Influencia del ruido en la performance de la red

4

Ortogonalidad y clusters

Cambiando letras y patrones

5

Alternativas

Incrementando la capacidad del modelo y variaciones

6

Conclusiones

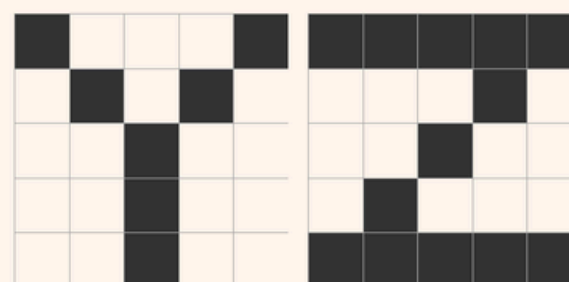
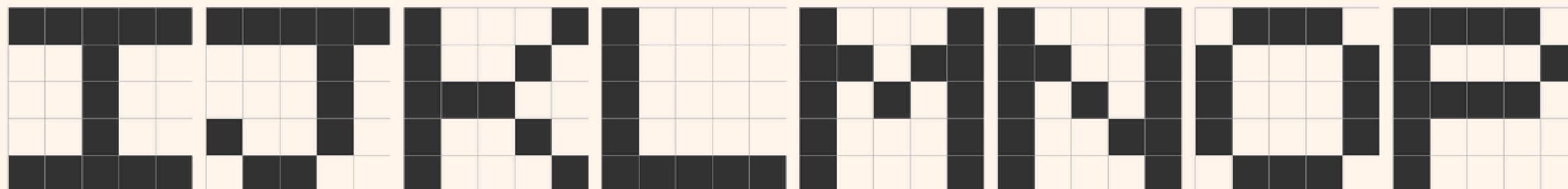
Capacidad del modelo y alternativas

1

Letras

Abecedario

Abecedario



1

Letras

Patrones elegidos

Max DP	Max DP Count	Avg DP	Group
3.0	1	1.33	(I,R,W,X)
3.0	1	1.33	(O,R,T,V)
3.0	2	1.67	(F,I,W,X)
3.0	2	1,67	(G,R,T,V)
3.0	2	1,67	(J,L,R,X)
3.0	2	1,67	(L,R,T,X)

MEJORES CANDIDATOS

En un espacio vectorial con producto interior V , dos vectores $x \in V$ e $y \in V$ son ortogonales si el producto escalar de $\langle x,y \rangle$ es 0.

Producto 0 entre A y B implica que, por cada pixel "encendido" de una letra A, ese pixel de la letra B es igual de probable de ser -1 o 1.

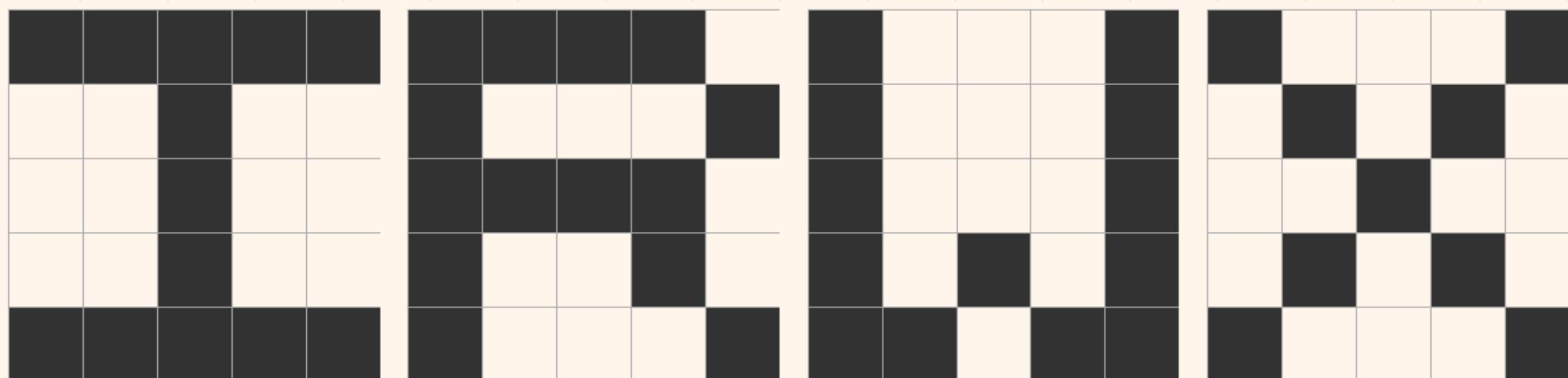
No hay estructura compartida.

Para el análisis de grupos se utilizó la estrategia vista en clase.

1

Letras

Patrones elegidos



1

Letras

Elecciones de parámetros

Max DP	Max DP Count	Avg DP	Group
21.0	2	17.67	(A,F,P,R)
21.0	2	17.00	(B,F,P,R)
21.0	2	17.00	(D,O,Q,X)
21.0	2	17.00	(E,F,P,R)
21.0	2	16.67	(C,D,O,Q)
21.0	2	16.67	(D,G,O,Q)

PEORES CANDIDATOS

ESTRATEGIA



Letras

~~Abecedario y patrones elegidos~~



Paso a paso

Encontrando la estructura de una letra



Ruido

Influencia del ruido en la performance de la red



Ortogonalidad y clusters

Cambiando letras y patrones



Alternativas

Incrementando la capacidad del modelo y variaciones



Conclusiones

Capacidad del modelo y alternativas

2

Paso a paso

Modo sincrónico

Se implementó la versión **sincrónica** de Hopfield como modo por defecto.

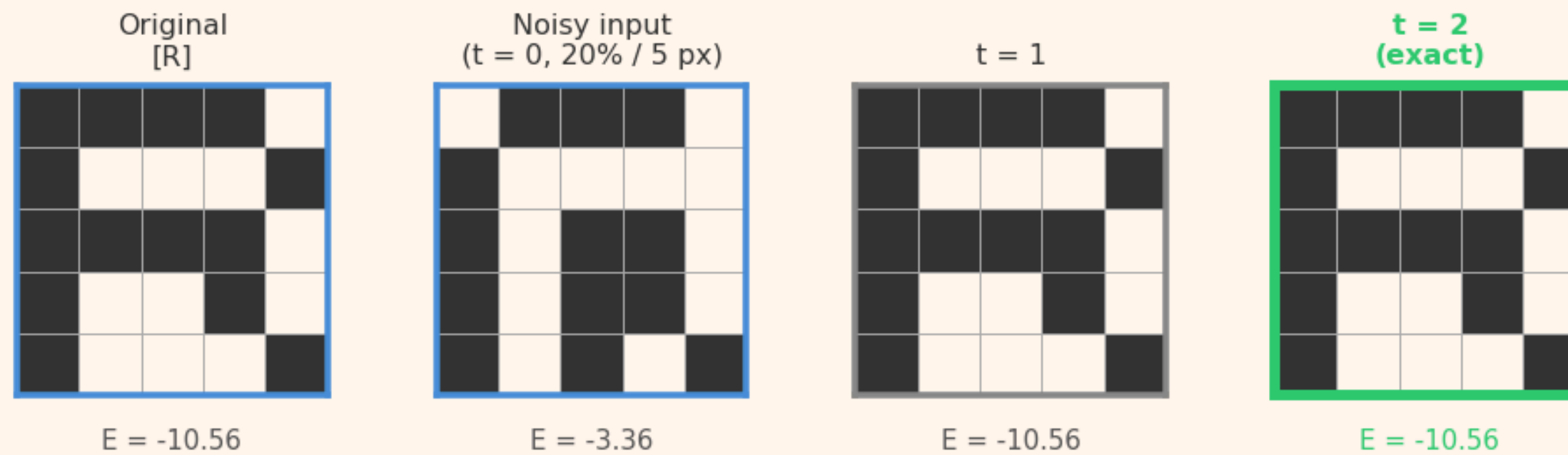
En cada paso, calcula el estado de todas las neuronas y las actualiza **al mismo tiempo**.

Si bien puede quedar atrapado en ciclos, se eligió por ser más **rápido**.

2

Paso a paso

Convergencia exacta



OBSERVACIONES

- Con un 20% de ruido, cae en el atractor del patrón original en un paso.
- El segundo paso (repetido) indica que se convergió.

2

Paso a paso

Convergencia exacta

campo de ξ
(original [R])

0.7	0.9	0.9	0.9	-1.0
0.9	-0.8	-0.8	-0.8	0.9
0.9	1.0	0.8	1.0	-0.8
0.9	-0.8	-0.9	0.9	-0.8
0.7	-0.9	-0.8	-0.9	0.7

 $h(0) = W \cdot s(0)$
(noisy, 20% / 5 px)

0.7	0.9	0.9	0.9	-0.5
0.2	-0.6	0.0	-0.6	0.6
0.2	0.8	0.6	0.5	-0.6
0.2	-0.6	-0.6	0.2	-0.6
0.4	-0.2	-0.3	-0.2	0.4

 $h(1) = W \cdot s(1)$
(t = 1)

0.7	0.9	0.9	0.9	-1.0
0.9	-0.8	-0.8	-0.8	0.9
0.9	1.0	0.8	1.0	-0.8
0.9	-0.8	-0.9	0.9	-0.8
0.7	-0.9	-0.8	-0.9	0.7

 $h(2) = W \cdot s(2)$
(exact, t = 2)

0.7	0.9	0.9	0.9	-1.0
0.9	-0.8	-0.8	-0.8	0.9
0.9	1.0	0.8	1.0	-0.8
0.9	-0.8	-0.9	0.9	-0.8
0.7	-0.9	-0.8	-0.9	0.7

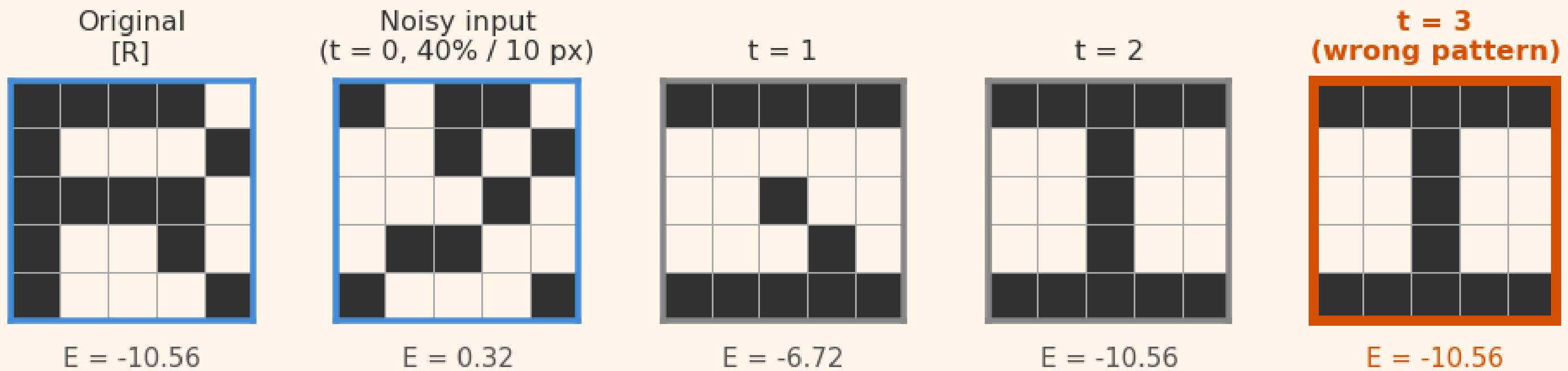
OBSERVACIONES

- El primer cuadrado es el campo de referencia (pesos multiplicados por el patrón **R**).
- La red llega al patrón correcto en el primer cálculo (mutiplicando los pesos por el estado ruidoso inicial), pero no con un “alto grado de seguridad”.
- Entre el paso 0 y el 1, el campo local de las neuronas se acerca al campo de referencia correspondiente a la **R**.

2

Paso a paso

Convergencia a otro patrón



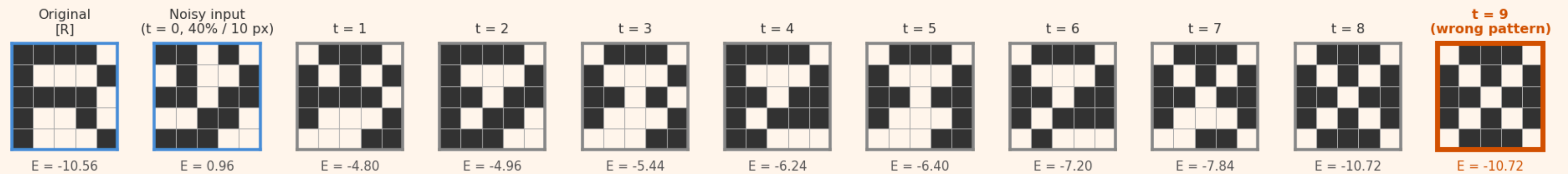
OBSERVACIONES

- Con un 40% de ruido, la entrada se asemeja más a otro patrón, el de la I.
- La red converge correctamente, pero al atractor equivocado.

2

Paso a paso

Convergencia a otro patrón (diferente seed)



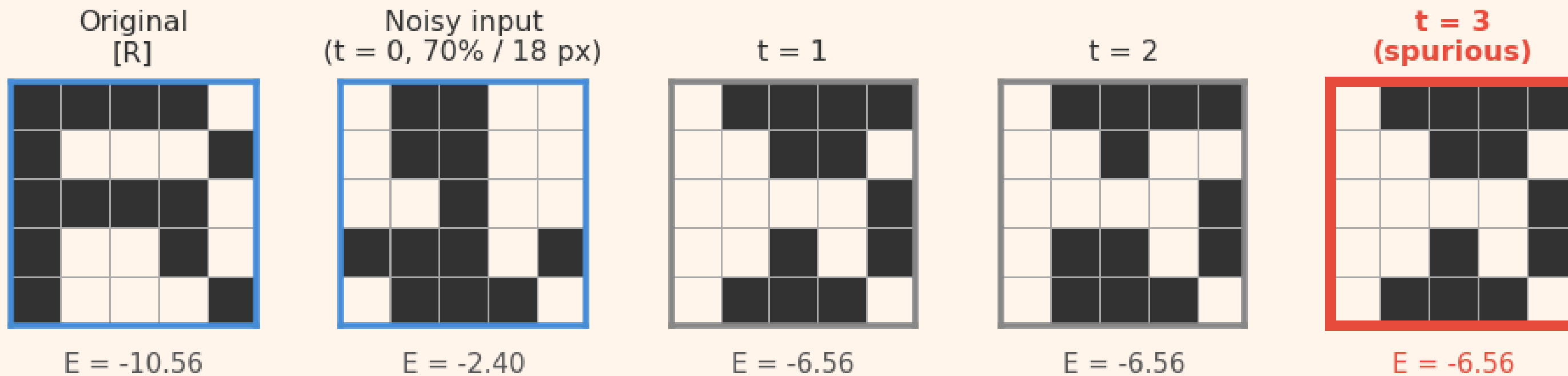
OBSERVACIONES

- La seed genera variaciones notables en la cantidad de pasos requeridos y en el resultado final.

2

Paso a paso

Convergencia a patrón espurioso



OBSERVACIONES

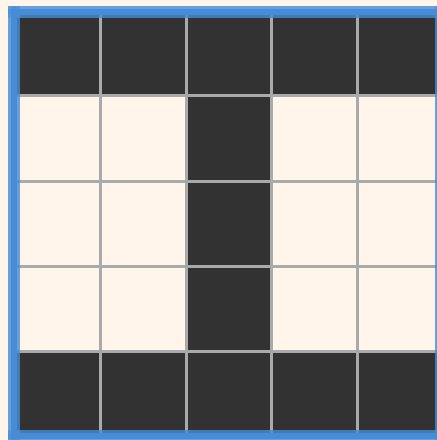
- Con un 70% de ruido, la red llega a un mínimo local: un estado espúreo.
- La energía es de menor magnitud que cuando se llega a uno de los patrones cargados.

2

Paso a paso

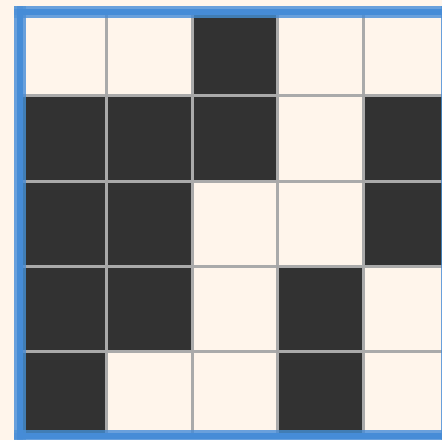
Convergencia a inverso

Original
[I]



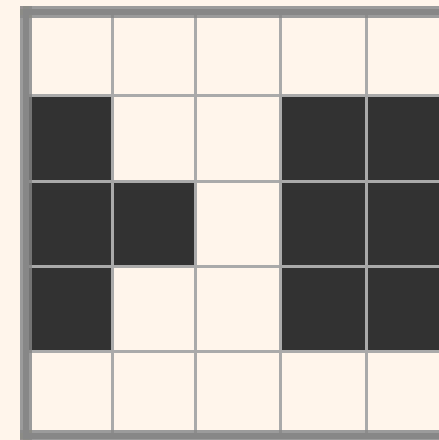
$E = -10.56$

Noisy input
($t = 0$, 70% / 18 px)



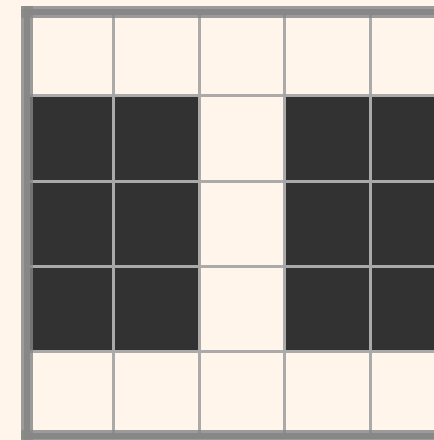
$E = -0.80$

$t = 1$



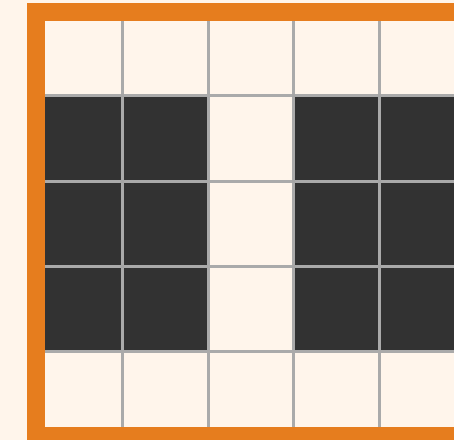
$E = -8.00$

$t = 2$



$E = -10.56$

$t = 3$
(inverse)



$E = -10.56$

OBSERVACIONES

- Con un 70% de ruido y otra letra, la red llega al patrón inverso al esperado.

ESTRATEGIA



~~Letras~~

~~Abecedario y patrones elegidos~~



~~Paso a paso~~

~~Encontrando la estructura de una letra~~



Ruido

Influencia del ruido en la performance de la red



Ortogonalidad y clusters

Cambiando letras y patrones



Alternativas

Incrementando la capacidad del modelo y variaciones



Conclusiones

Capacidad del modelo y alternativas

3

Ruido

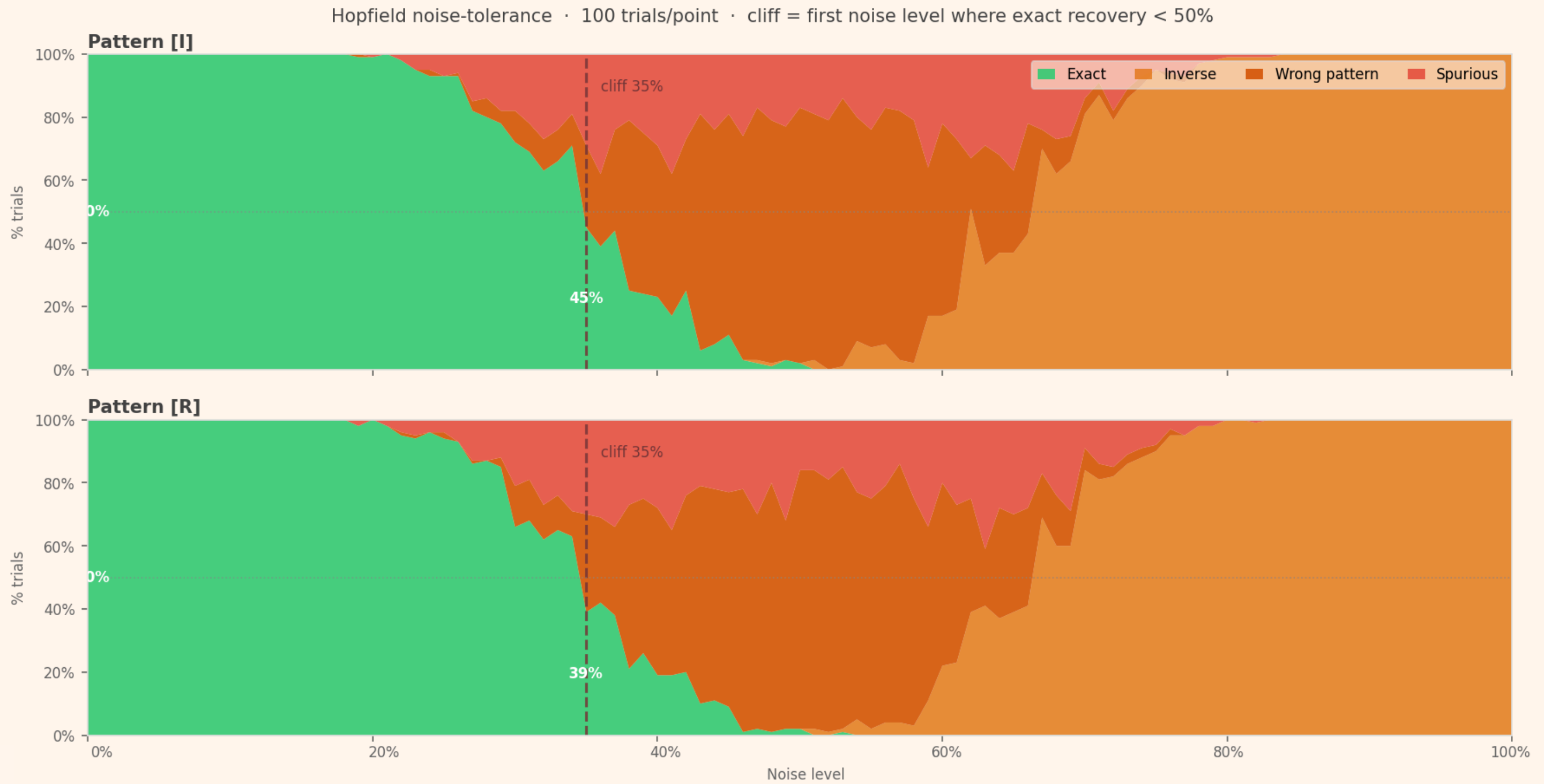
Influencia del ruido en la performance de la red



3

Ruido

¿Cuál es el límite de ruido que puede tolerar cada patrón?

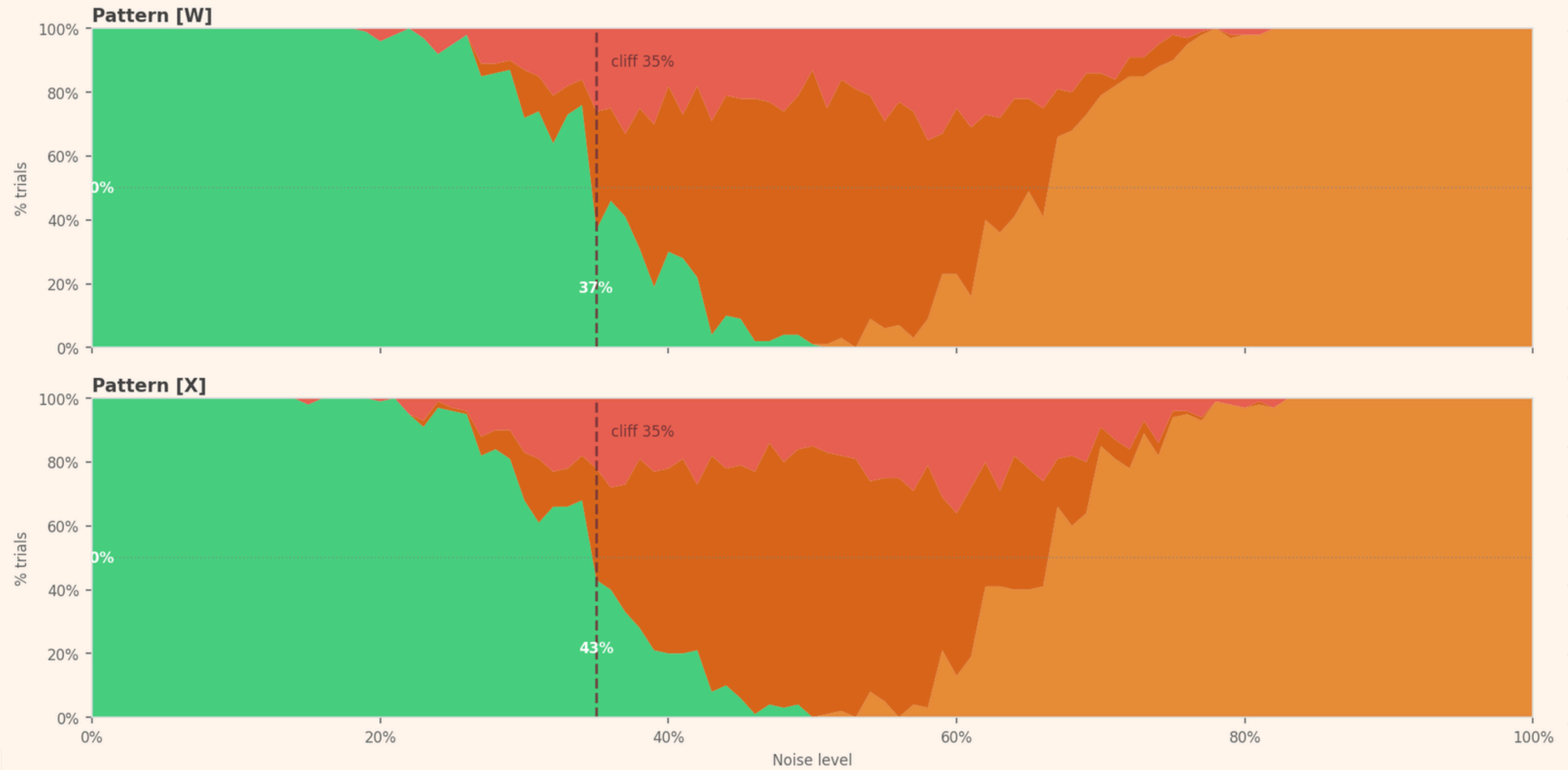


3

Ruido

¿Cuál es el límite de ruido que puede tolerar cada patrón?

Hopfield noise-tolerance · 100 trials/point · cliff = first noise level where correct recovery < 50%

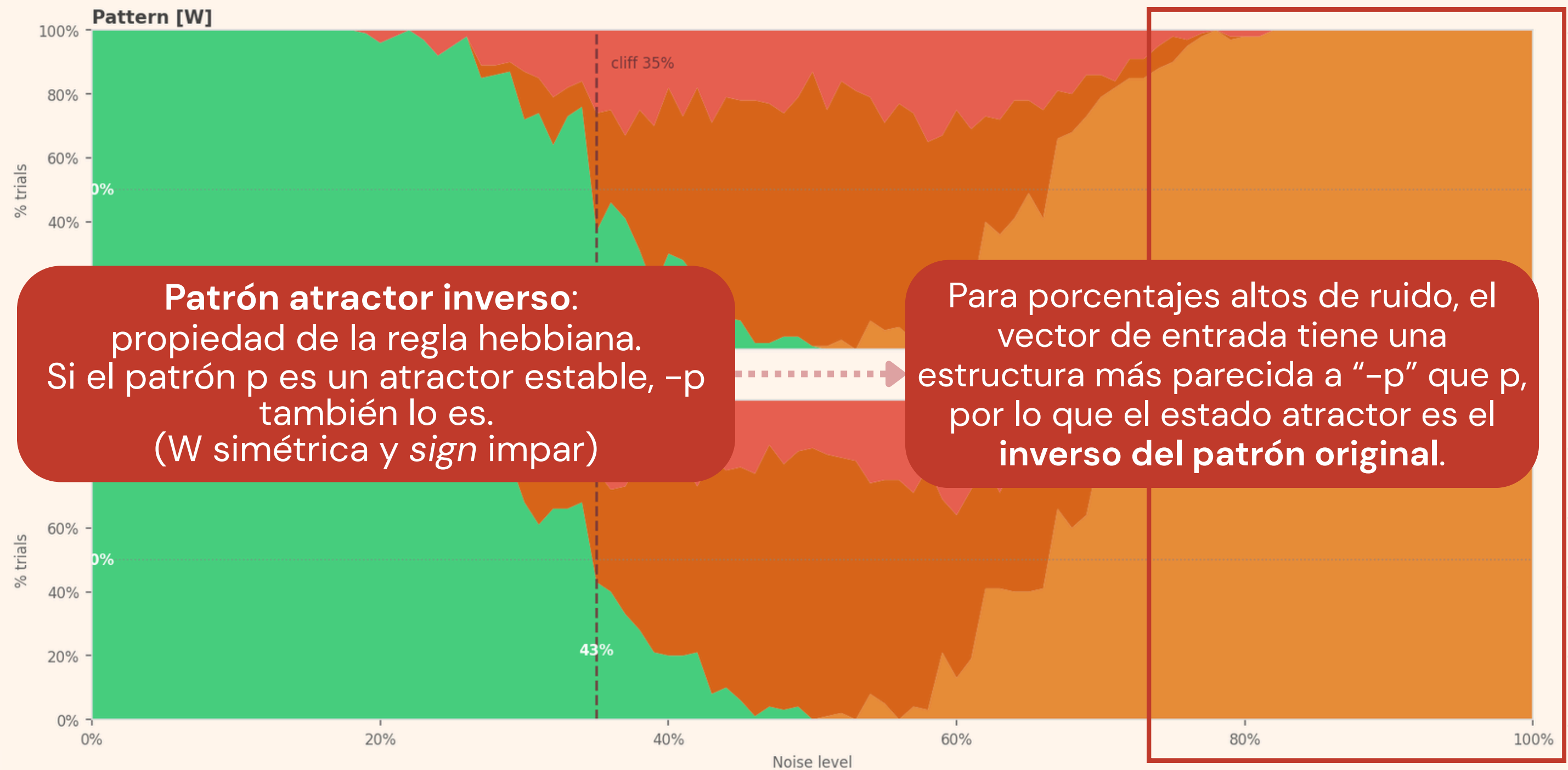


3

Ruido

¿Cuál es el límite de ruido que puede tolerar cada patrón?

Hopfield noise-tolerance · 100 trials/point · cliff = first noise level where correct recovery < 50%

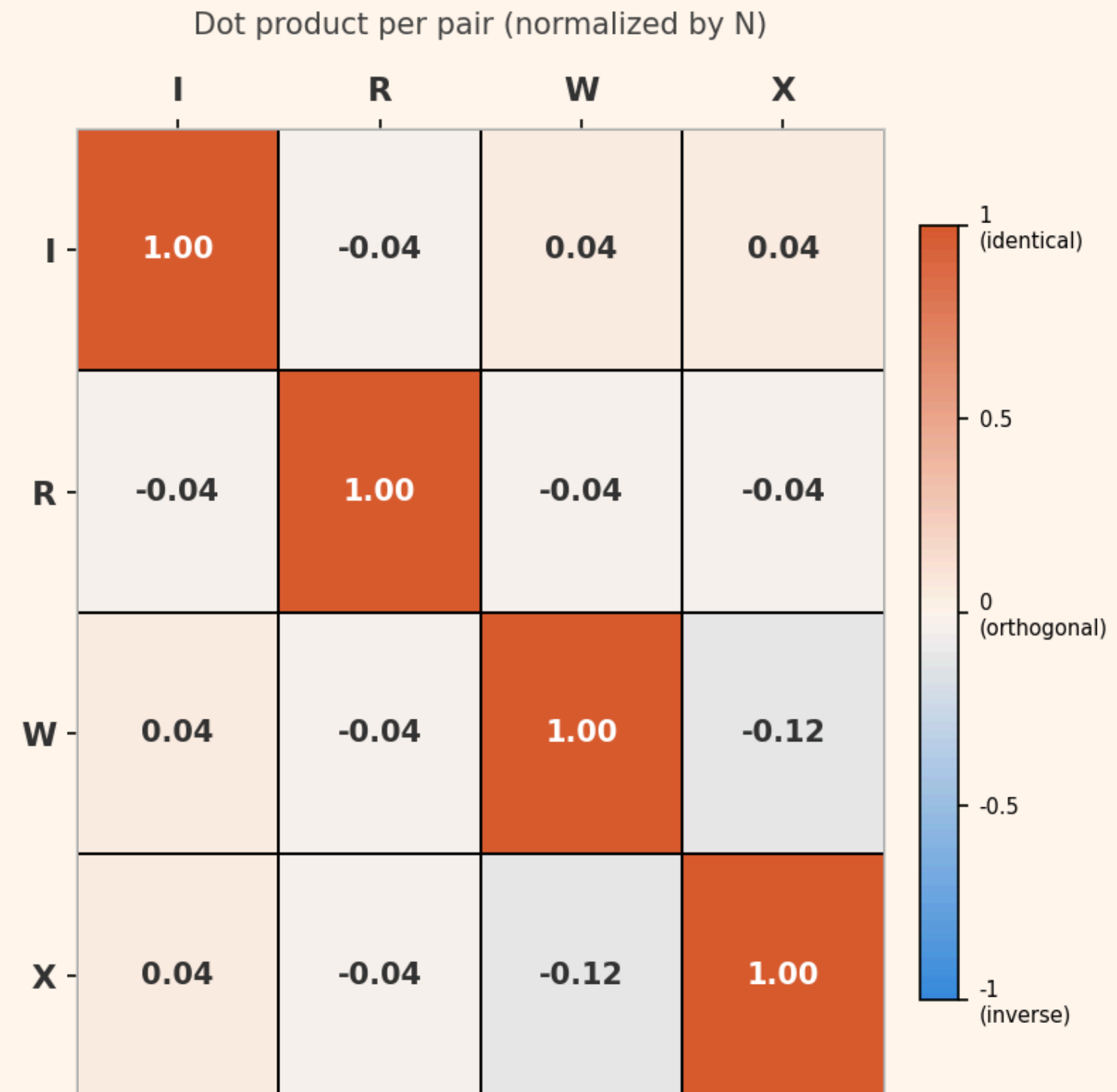


3

Ruido

Tolerancia a ruido por patrón: similitud entre ellos

Pattern	Cliff	Correct trials
I	36%	39%
R	36%	42%
W	36%	46%
X	36%	40%



ESTRATEGIA



Letras

~~Abecedario y patrones elegidos~~



Paso a paso

~~Encontrando la estructura de una letra~~



Ruido

~~Influencia del ruido en la performance de la red~~



Ortogonalidad y clusters

Cambiando letras y patrones



Alternativas

Incrementando la capacidad del modelo y variaciones



Conclusiones

Capacidad del modelo y alternativas

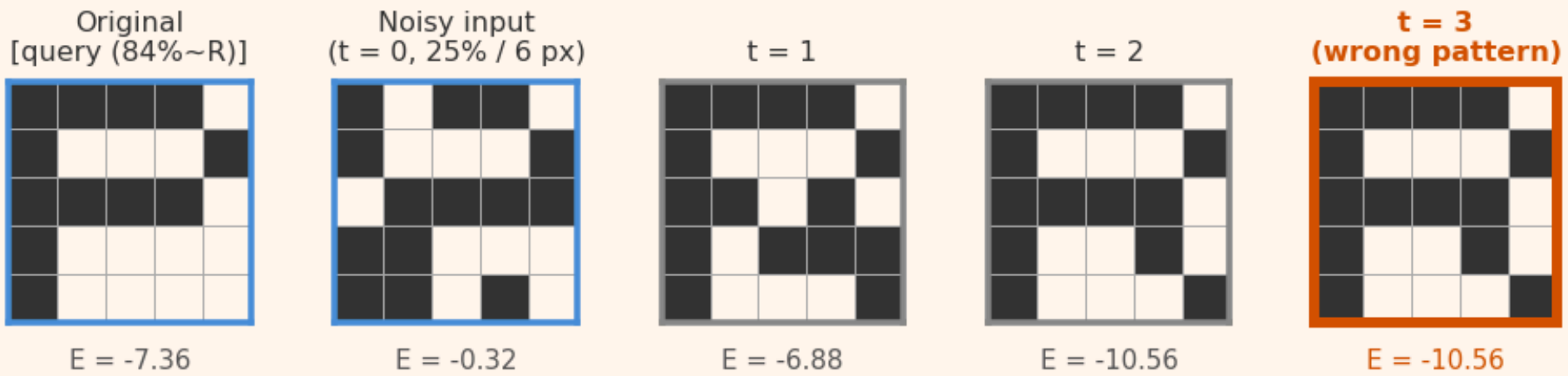
4

Ortogonalidad y clusters

Cambiando las letras elegidas

Avg DP	Group
21.00	(D,O)
21.00	(F,P)
21.00	(M,N)
21.00	(O,Q)
21.00	(P,R)
17.00	(A,P)

Hopfield convergence · query [query (84%~R)] · noise 25% · seed 42 · converged in 2 step(s) → exact recovery of [R]



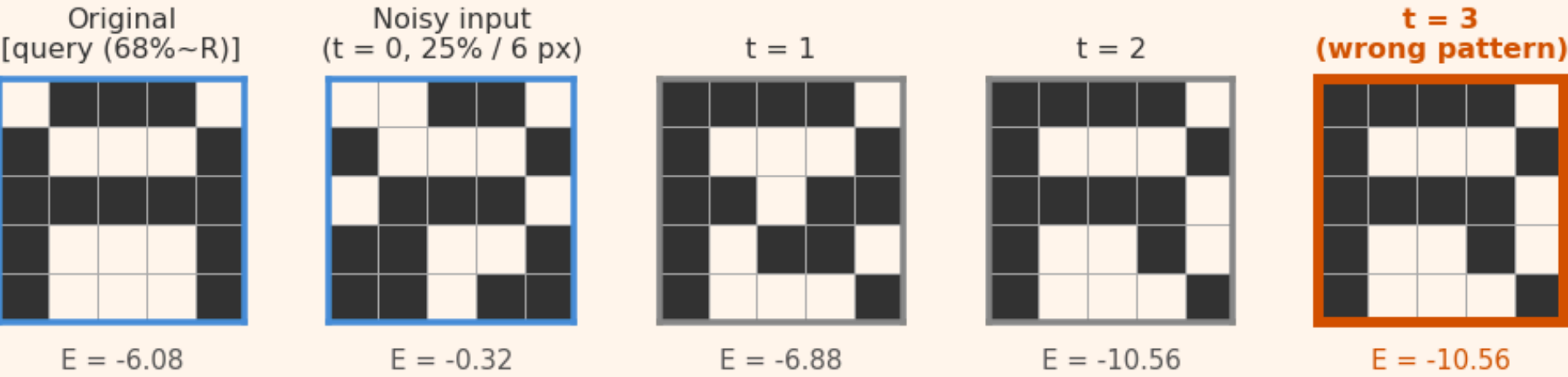
4

Ortogonalidad y clusters

Cambiando las letras elegidas

Avg DP	Group
21.00	(D,O)
21.00	(F,P)
21.00	(M,N)
21.00	(O,Q)
21.00	(P,R)
17.00	(A,P)

Hopfield convergence · query [query (68%~R)] · noise 25% · seed 42 · converged in 2 step(s) → exact recovery of [R]

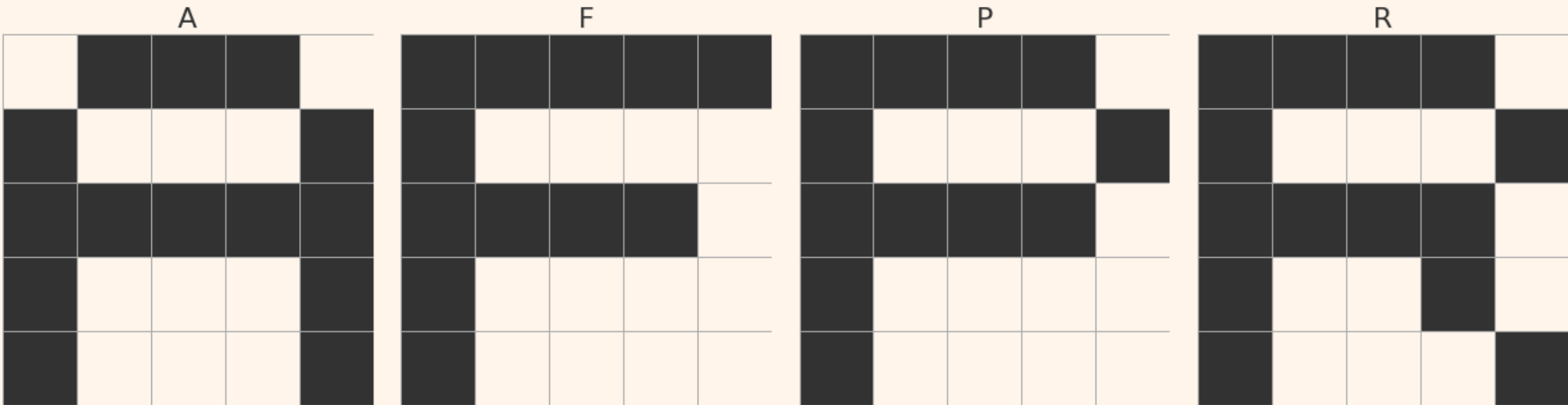


4

Ortogonalidad y clusters

Cambiando las letras elegidas

Stored Patterns

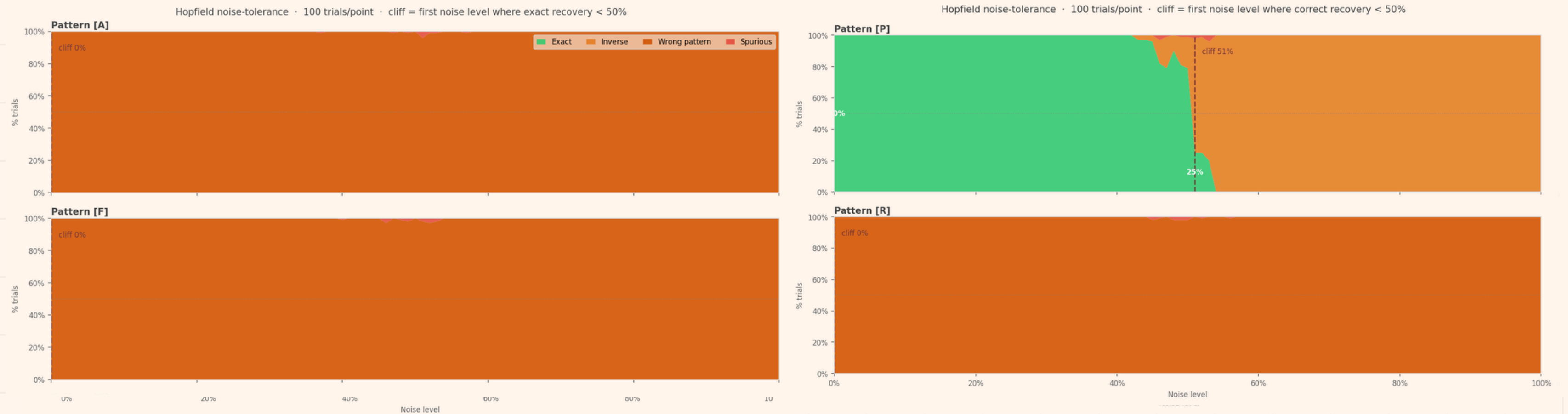


Max DP	Max DP Count	Avg DP	Group
21.0	2	17.67	(A,F,P,R)

4

Ortogonalidad y clusters

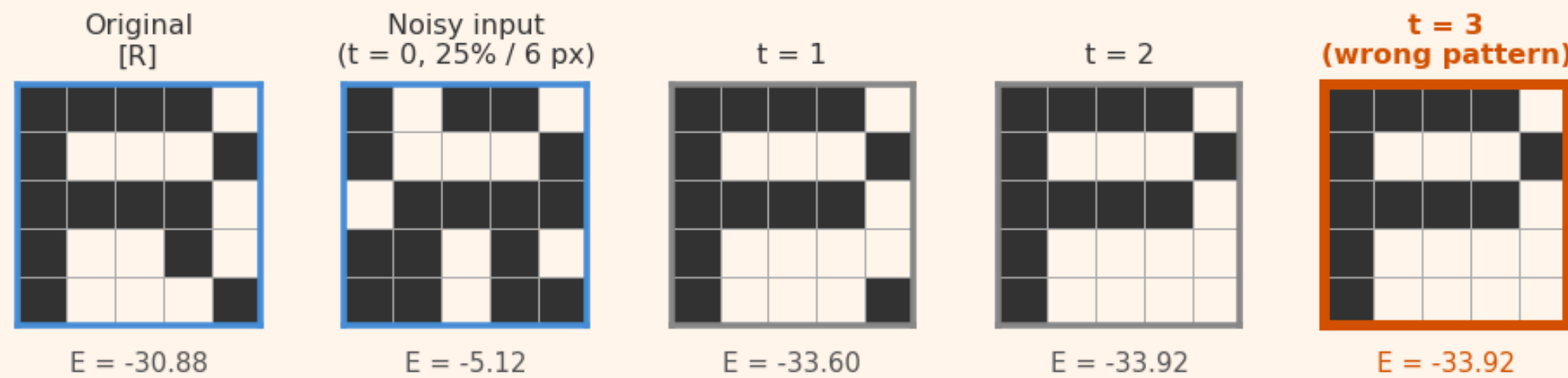
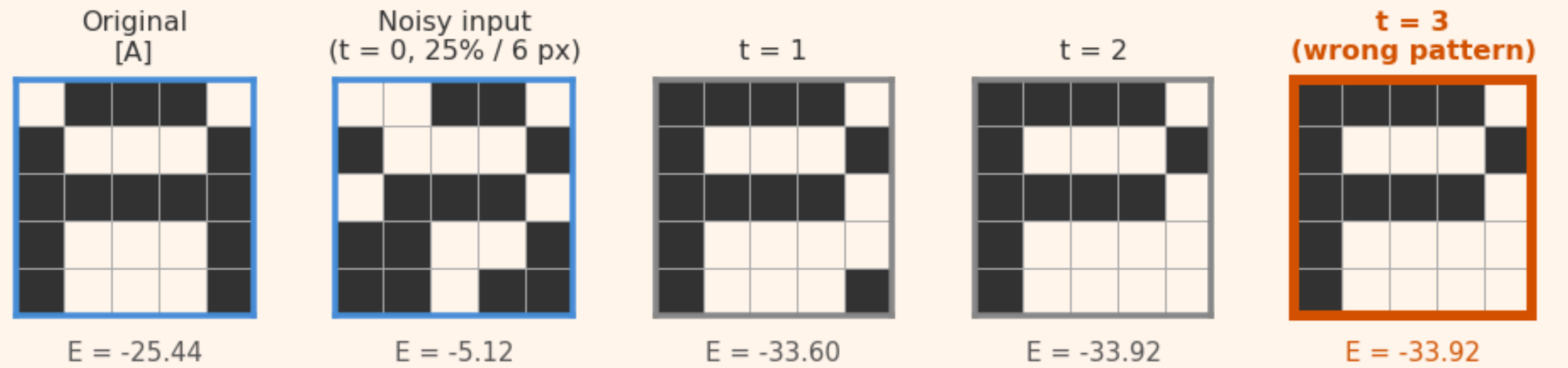
Cambiando las letras elegidas



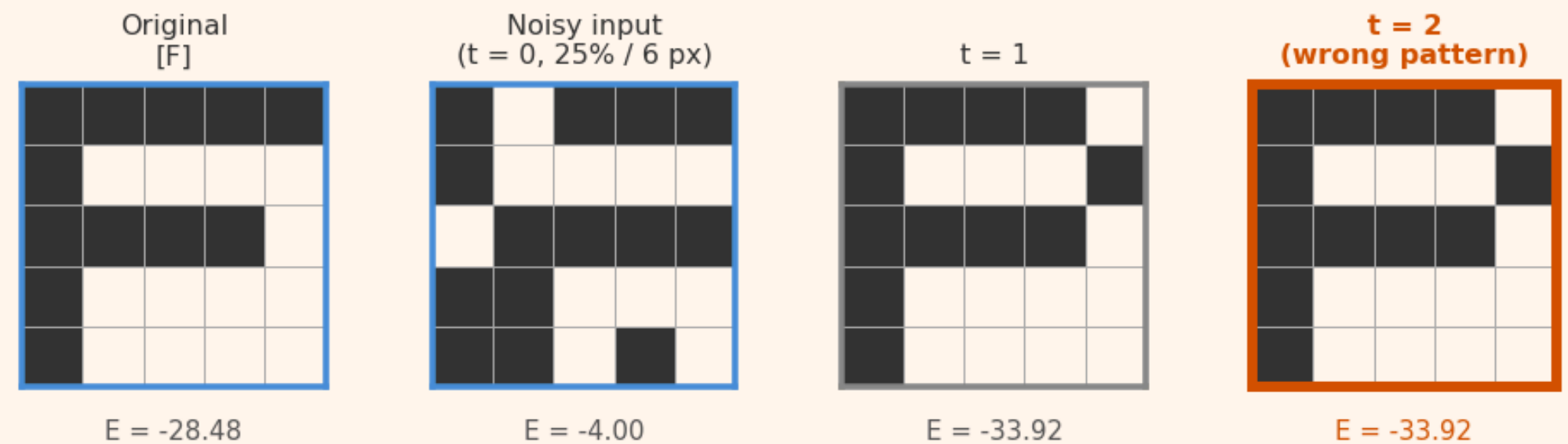
Todas las consultas convergen a la letra P
(o su inverso)

4 Ortogonalidad y clusters

Cambiando las letras elegidas



Todas las consultas convergen a la letra P.

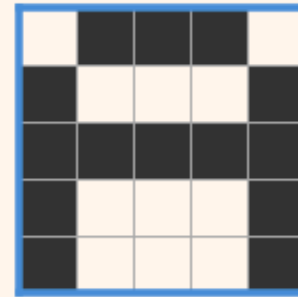


4

Ortogonalidad y clusters

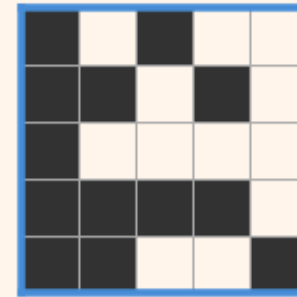
Cambiando las letras elegidas

Original
[A]



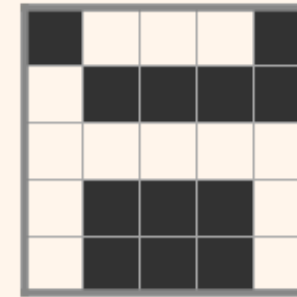
$E = -25.44$

Noisy input
($t = 0$, 60% / 15 px)



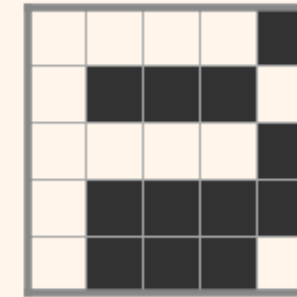
$E = 1.28$

$t = 1$



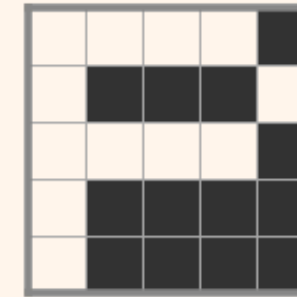
$E = -22.08$

$t = 2$



$E = -33.60$

$t = 3$



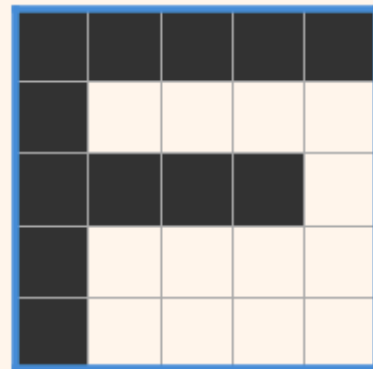
$E = -33.92$

$t = 4$
(wrong pattern)



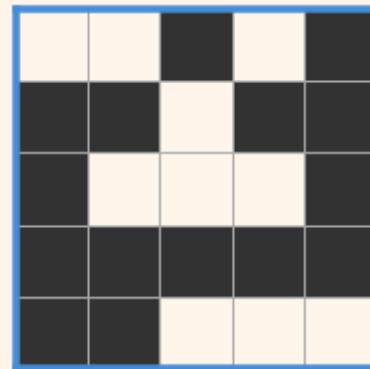
$E = -33.92$

Original
[F]



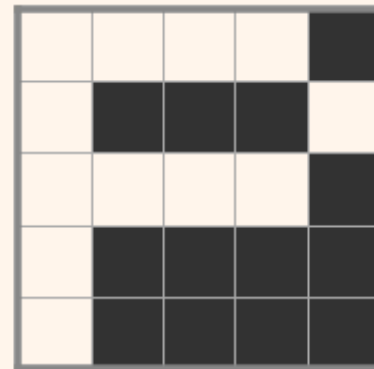
$E = -28.48$

Noisy input
($t = 0$, 60% / 15 px)



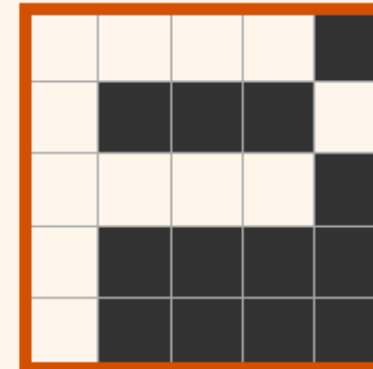
$E = 0.48$

$t = 1$



$E = -33.92$

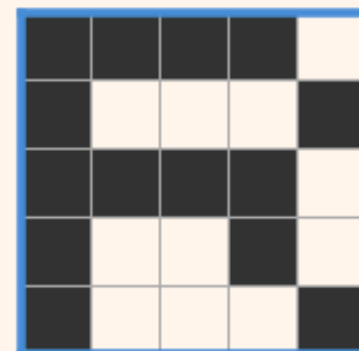
$t = 2$
(wrong pattern)



$E = -33.92$

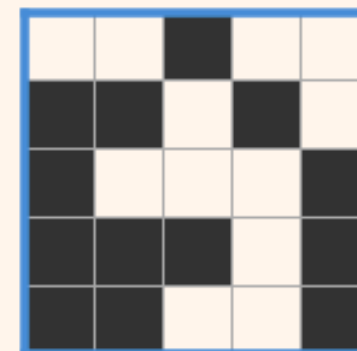
O su inverso.

Original
[R]



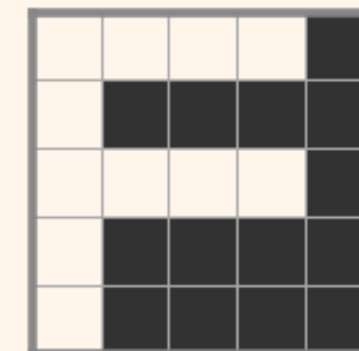
$E = -30.88$

Noisy input
($t = 0$, 60% / 15 px)



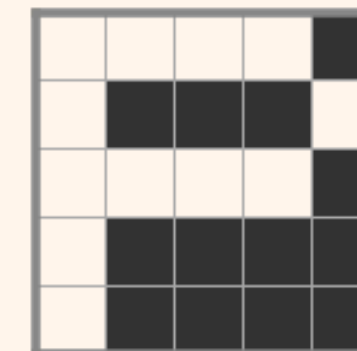
$E = 0.32$

$t = 1$



$E = -30.88$

$t = 2$



$E = -33.92$

$t = 3$
(wrong pattern)

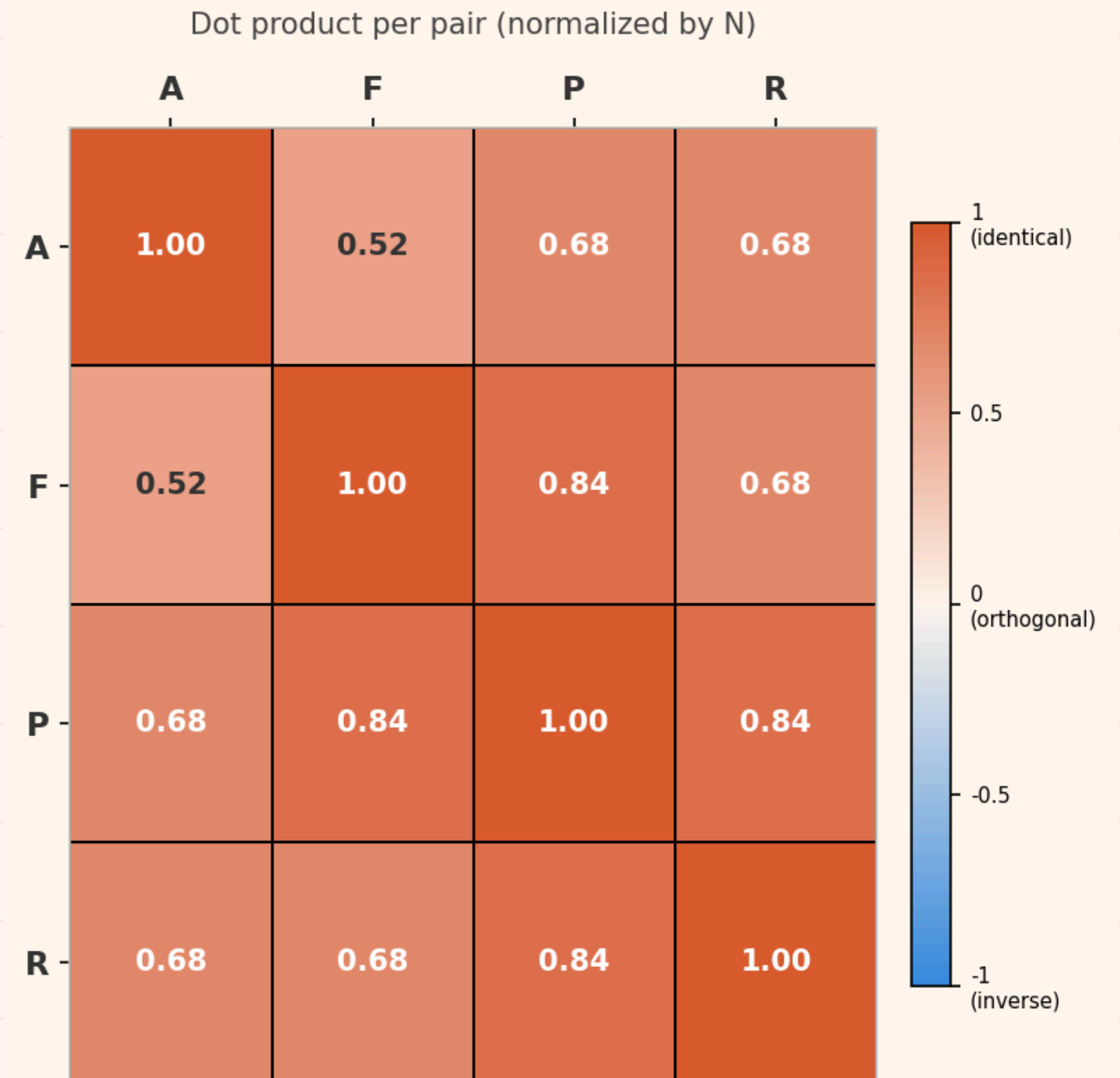
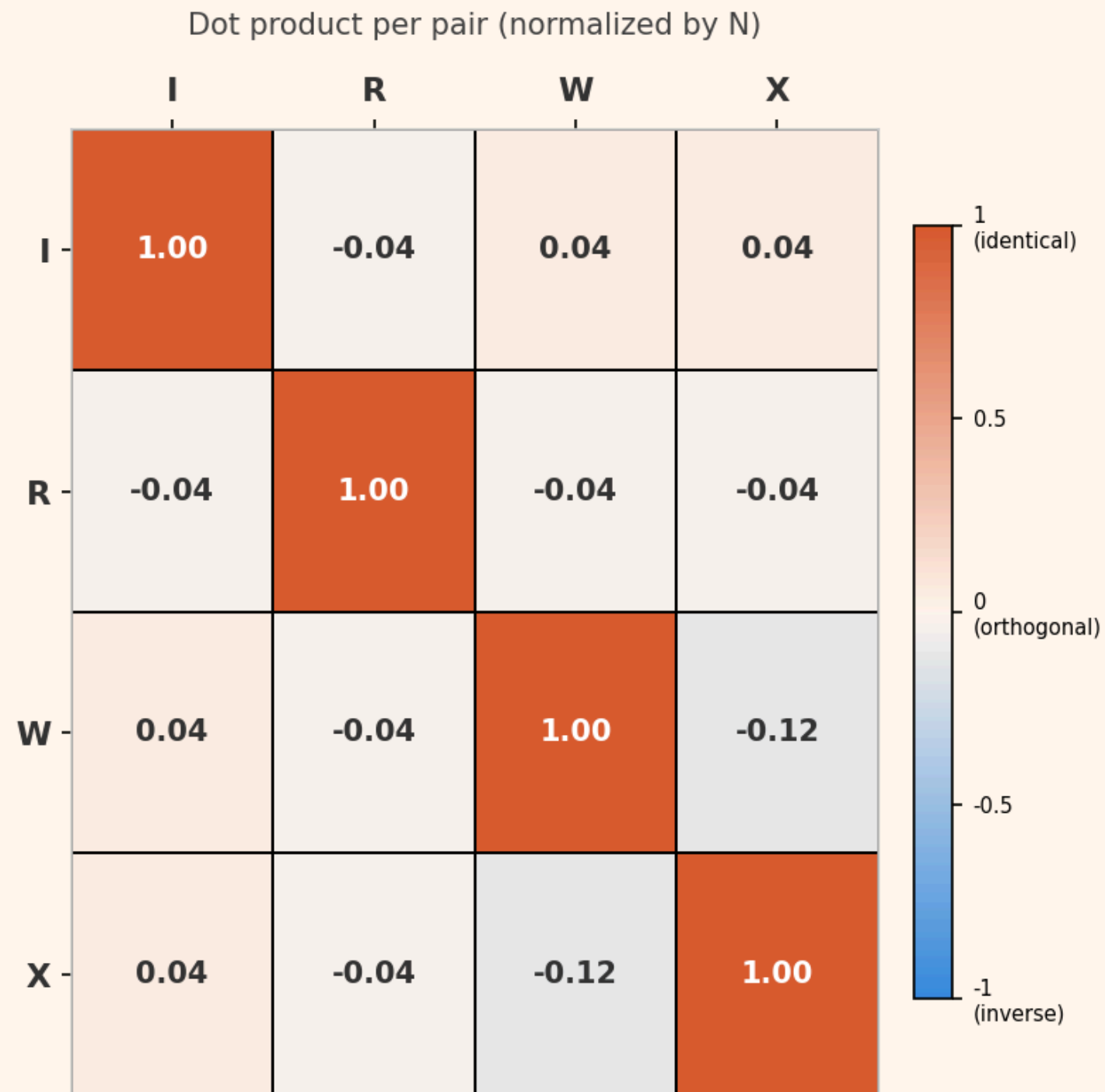


$E = -33.92$

4

Ortogonalidad y clusters

Cambiando las letras elegidas



4

Ortogonalidad y clusters

Más limitaciones del modelo

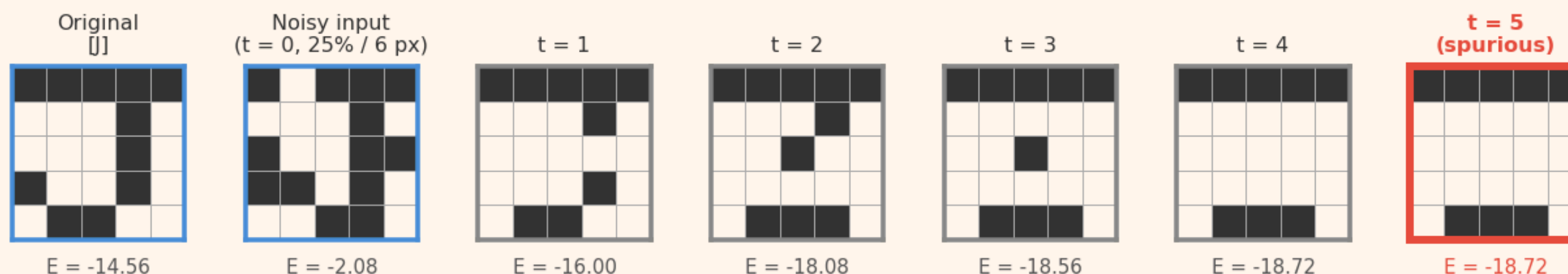
La capacidad del modelo es tal que

$$p \leq 0.15 \times N$$

El término de crosstalk crece con P. Si P es grande la interferencia puede superar la señal:

$$h_i = \sum_j W_{ij} \xi_j^\mu = \xi_i^\mu + \frac{1}{N} \sum_{v \neq \mu} \sum_j \xi_i^v \xi_j^v \xi_j^\mu$$

Se toman los 10 patrones con mayor diferencia entre sí: (A,C,D,J,K,S,T,V,W,Z)



No solo hay interferencia por la cantidad de patrones sino por la naturaleza del "dataset".

La red no recupera los patrones correctamente.

ESTRATEGIA



Letras

~~Abecedario y patrones elegidos~~



Paso a paso

~~Encontrando la estructura de una letra~~



Ruido

~~Influencia del ruido en la performance de la red~~



Ortogonalidad y clusters

~~Cambiando letras y patrones~~



Alternativas

Incrementando la capacidad del modelo y variaciones



Conclusiones

Capacidad del modelo y alternativas

5

Alternativas

Alternativa al modo sincrónico

Decidió implementarse el modo **asincrónico** de Hopfield, para comparar.

En cada paso, recorre las neuronas en orden aleatorio y las actualiza **una por una**.

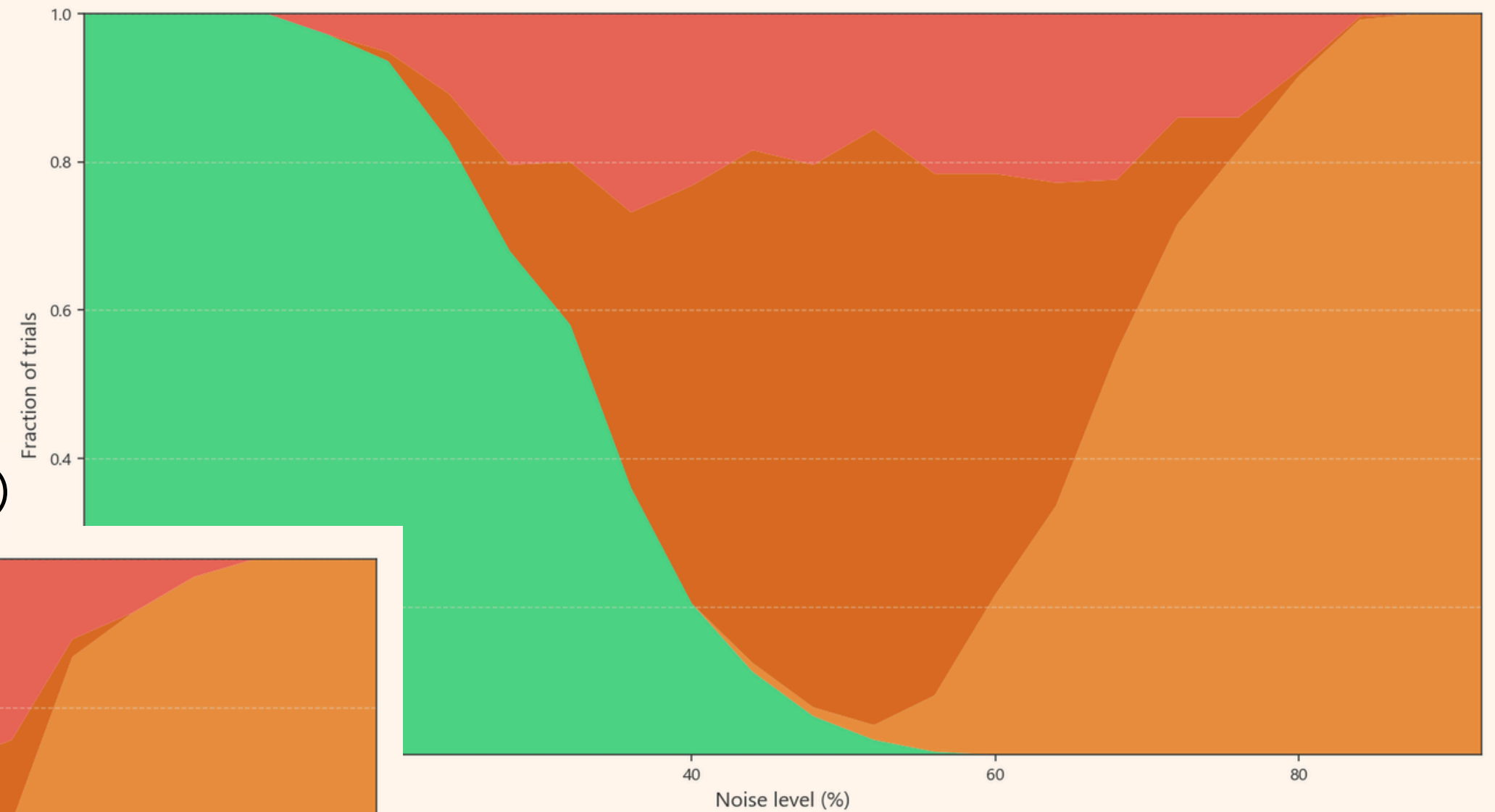
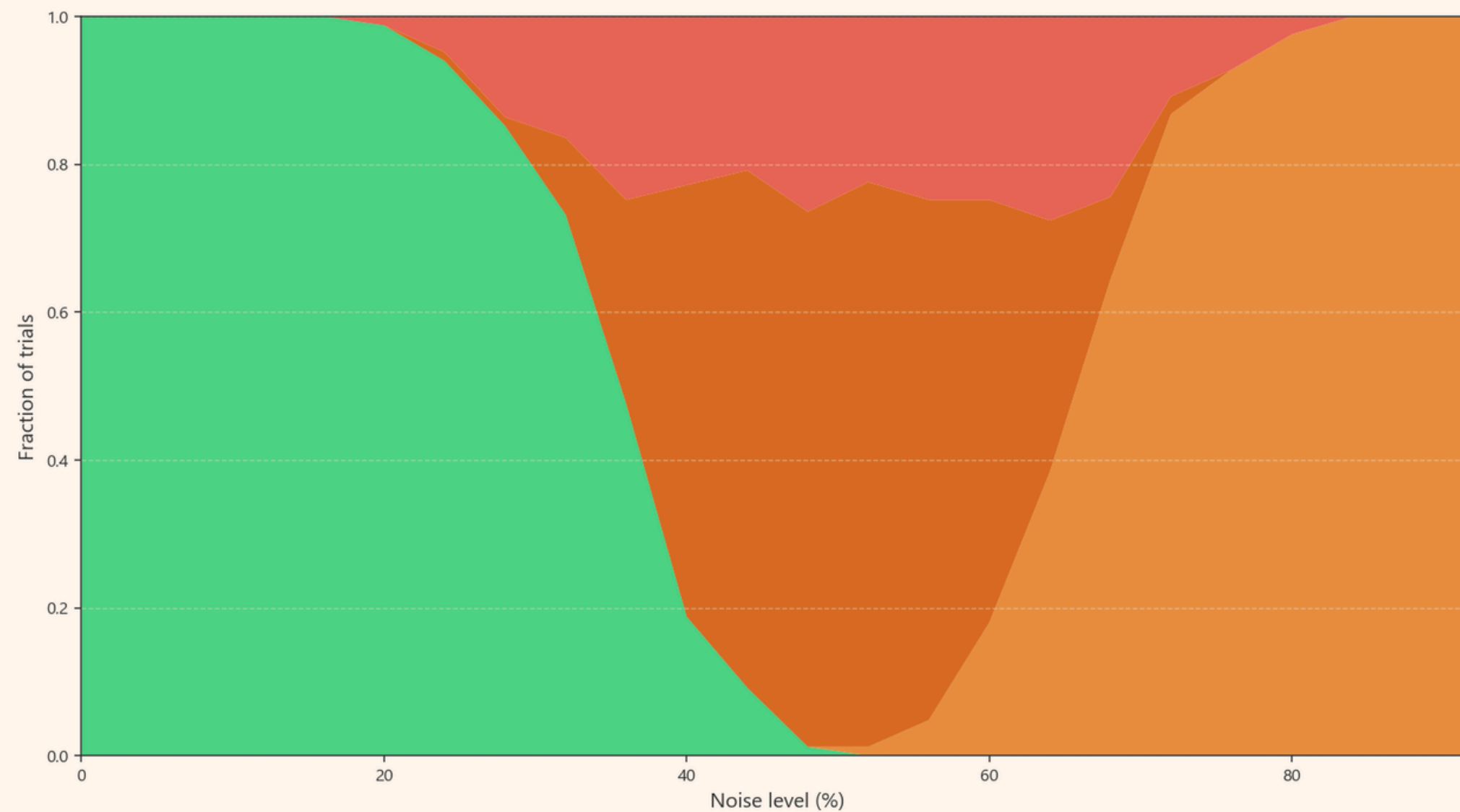
Garantiza la convergencia y que se bajará la energía de forma monótona.

5

Alternativas

Sync vs. async (resultado por ruido)

sync (debajo) vs. **async** (derecha)



OBSERVACIONES

- Los dos modos son similares con ruido bajo, llegando a un **36%** de ruido antes de fallar en la mayoría de los casos.
- Con ruido bajo, **sync** falla menos (correcto en el **99%** de casos), cayendo **async** (con **93%**) en más patrones incorrectos. Esto también puede apreciarse en los casos inversos.

5

Alternativas

Sync vs. async (resultado por ruido, 1000 runs por nivel de ruido)

Noise	sync				async			
	Exact	Wrong	Inverse	Spurious	Exact	Wrong	Inverse	Spurious
4%	100.0	—	—	—	100.0	—	—	—
12%	100.0	—	—	—	99.5	—	—	0.5
20%	98.9	0.1	—	1.0	93.6	1.0	—	5.4
28%	85.3	3.6	—	11.1	69.1	11.8	—	19.1
36%	46.4	29.0	—	24.6	37.6	36.2	—	26.2
44%	8.1	68.0	—	23.9	12.7	69.6	0.1	17.6
52%	0.2	75.6	1.5	22.7	2.2	76.4	3.7	17.7
60%	—	52.1	20.6	27.3	—	55.4	21.9	22.7

OBSERVACIONES

- Poca diferencia a poco ruido, con **async** empezando a caer antes.
- Pasado el 44% de ruido, **async** se vuelve mejor.
- Tiene lógica que los inversos empiecen a aparecer después del 50% de ruido.

5

Alternativas

Sync vs. *async* (teoría)

Async es víctima de su propio éxito; al garantizar que la energía decrecerá de forma monótona, no saldrá de mínimos locales para mantener a esto cierto.

Sync, al cambiar de forma más radical, se saltea gran parte de los mínimos locales, llegando al patrón correcto con mayor frecuencia.

5

Alternativas

Sync vs. *async* (ciclos)

Sync llega a un **ciclo** cuando todas las neuronas reaccionan al estado anterior a la vez, generando un cambio coordinado que las lleva a una configuración inestable de la misma forma que lo era antes. Desde ahí, el mismo mecanismo las devuelve al estado original, y así hasta la perpetuidad.

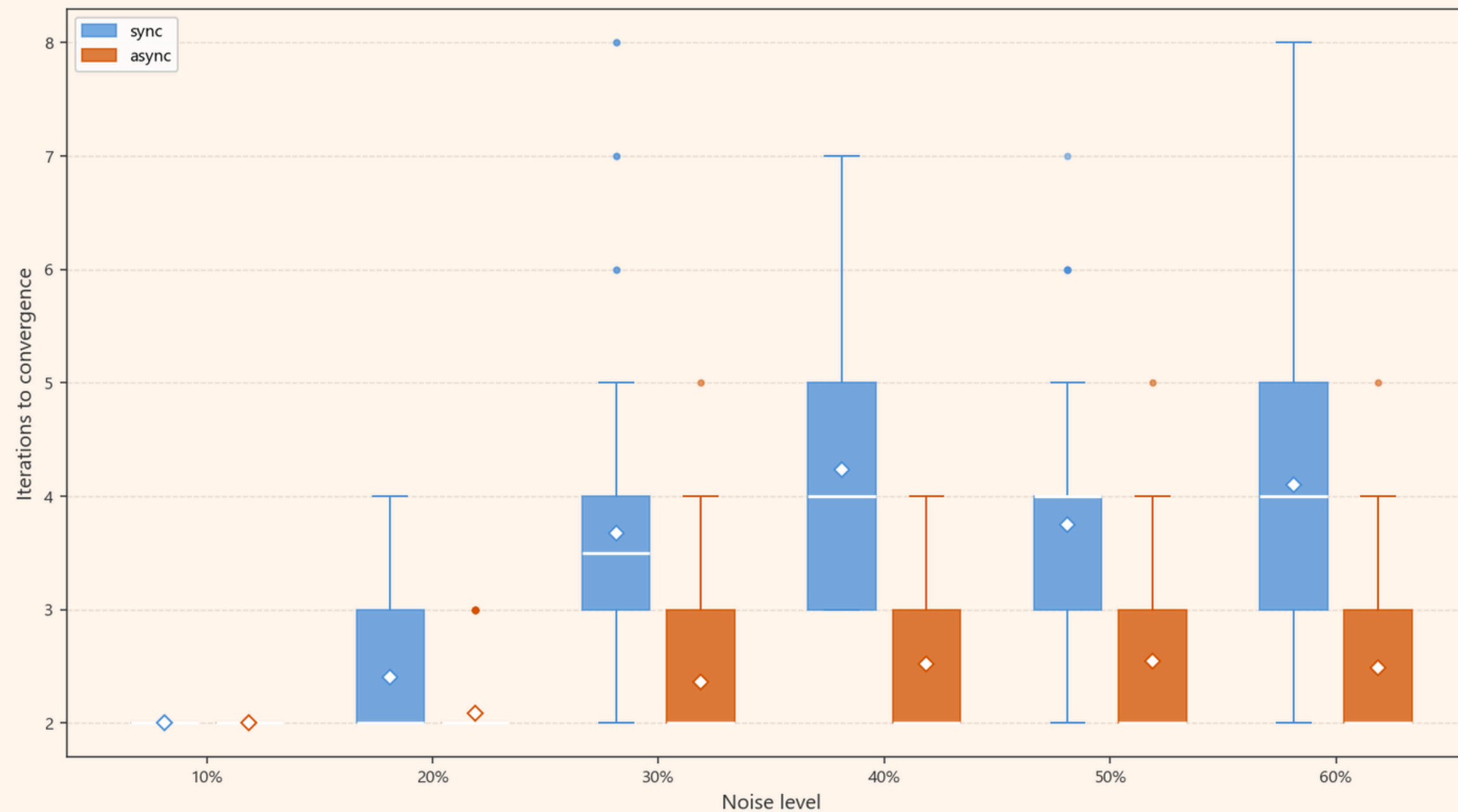
Suele suceder cuando se está equidistante de dos atractores, cosa más común con altos niveles de ruido.

En nuestro caso particular, los ciclos se cortan en favor de marcarlos como estado espúreo.

5

Alternativas

Sync vs. *async* (cantidad de iteraciones, 80 seeds)



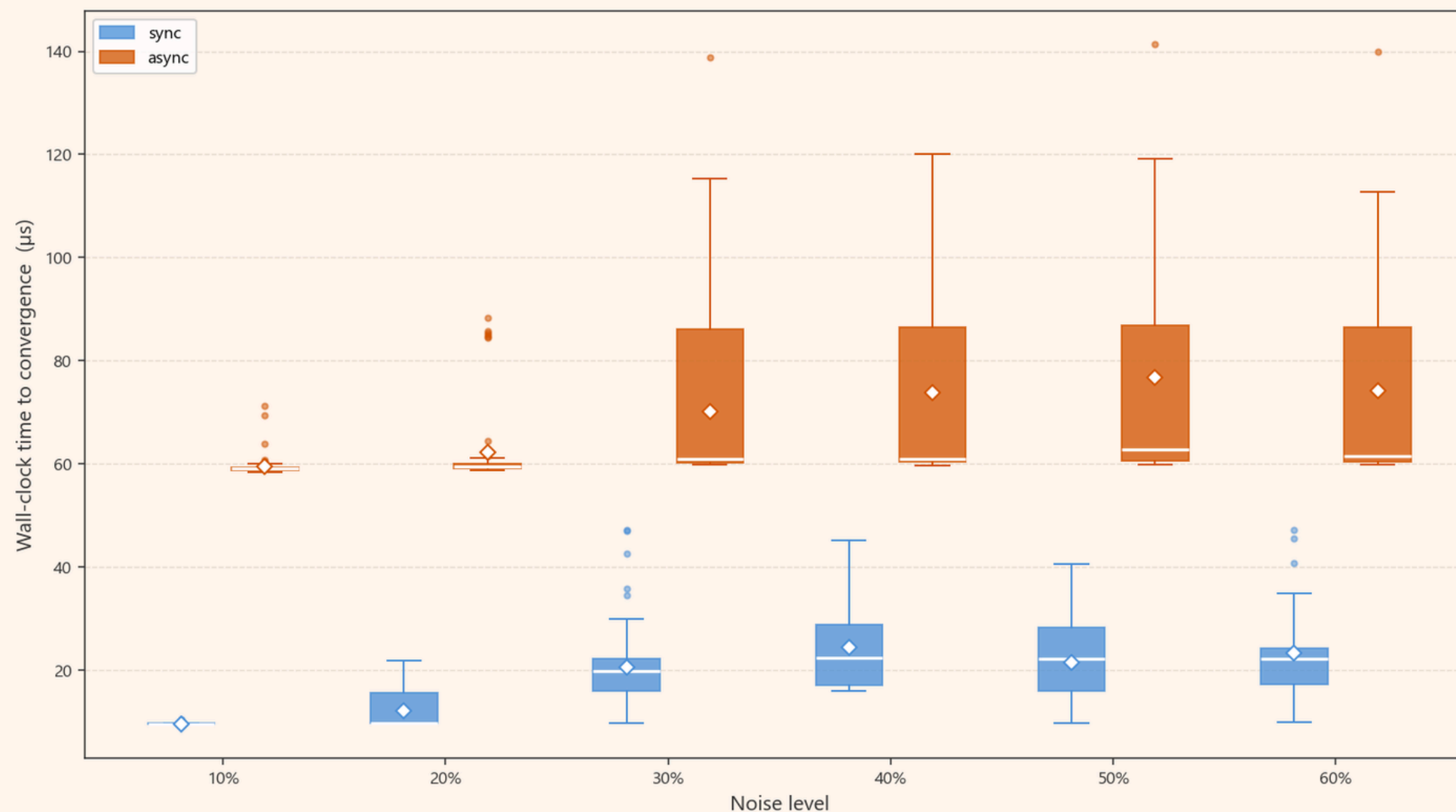
OBSERVACIONES

- **Async** termina en menos iteraciones, pero cada una implica 25 actualizaciones secuenciales (a diferencia de **sync**, que hace todas a la vez).

5

Alternativas

Sync vs. *async* (tiempo, 60 seeds)



OBSERVACIONES

- **Async** toma entre tres y seis veces más tiempo que **sync**.
- En este gráfico sí puede apreciarse la ventaja de **async** en términos de tiempo.

5

Alternativas

Sync vs. async (resumen)

- El modo **asincrónico** demora más tiempo en converger, y cuando lo hace suele ser peor que su contraparte. Esto es a cambio de garantizar que no haya ciclos.
- El modo **sincrónico** termina en menos tiempo y llega al patrón correcto (o su inverso) una mayor parte de las veces, a coste de –con ruido alto– caer en ciclos.

5

Alternativas

Hopfield Moderno: Continuous Hopfield Networks

“Hopfield Networks Is All You Need” Generalización del modelo moderno de Hopfield

¿Cómo incrementar la capacidad del modelo para incluir más letras?

Se rompe la relación lineal entre neuronas y cantidad de patrones y minimizar estados espúreos.

Cambio de la función de energía

En el paper se propone principalmente para incorporar a deep learning con backprop

Se amplifican las diferencias entre patrones en la recuperación y se garantiza convergencia.

Cambio de la regla de update

5

Alternativas

Hopfield Moderno: Continuous Hopfield Networks

“Hopfield Networks Is All You Need”
Generalización del modelo moderno de Hopfield

Cambio de la función de energía

Clásico $E = -\frac{1}{2} \times \xi^T \times W \times \xi$

Versión Moderna $E = -e^{lse(1, \xi^T X)}$

$$E = \underbrace{-lse(\beta, X^T \xi)}_{\text{Valles cerca de los patrones X}} + \underbrace{\frac{1}{2} \xi^T \xi}_{\text{Reg. para acotar estado}} + \underbrace{\frac{1}{N} \times \log(N)}_{\text{Corrección por \# de patrones}} + \underbrace{\frac{1}{2} \times M^2}_{\text{Corrección por tamaño de patrones}}$$

Cambio de la regla de update

$$S(t+1) = \text{sign}(W \times S(t))$$

$$S(t+1) = X \times \text{softmax}(\beta \times X^T S(t))$$

Diferencia con los otros patrones

Para potenciar diferencias:

$$\beta = 4$$

5

Alternativas

Hopfield Moderno: Continuous Hopfield Networks

Resultados

El nuevo modelo permite incrementar la capacidad



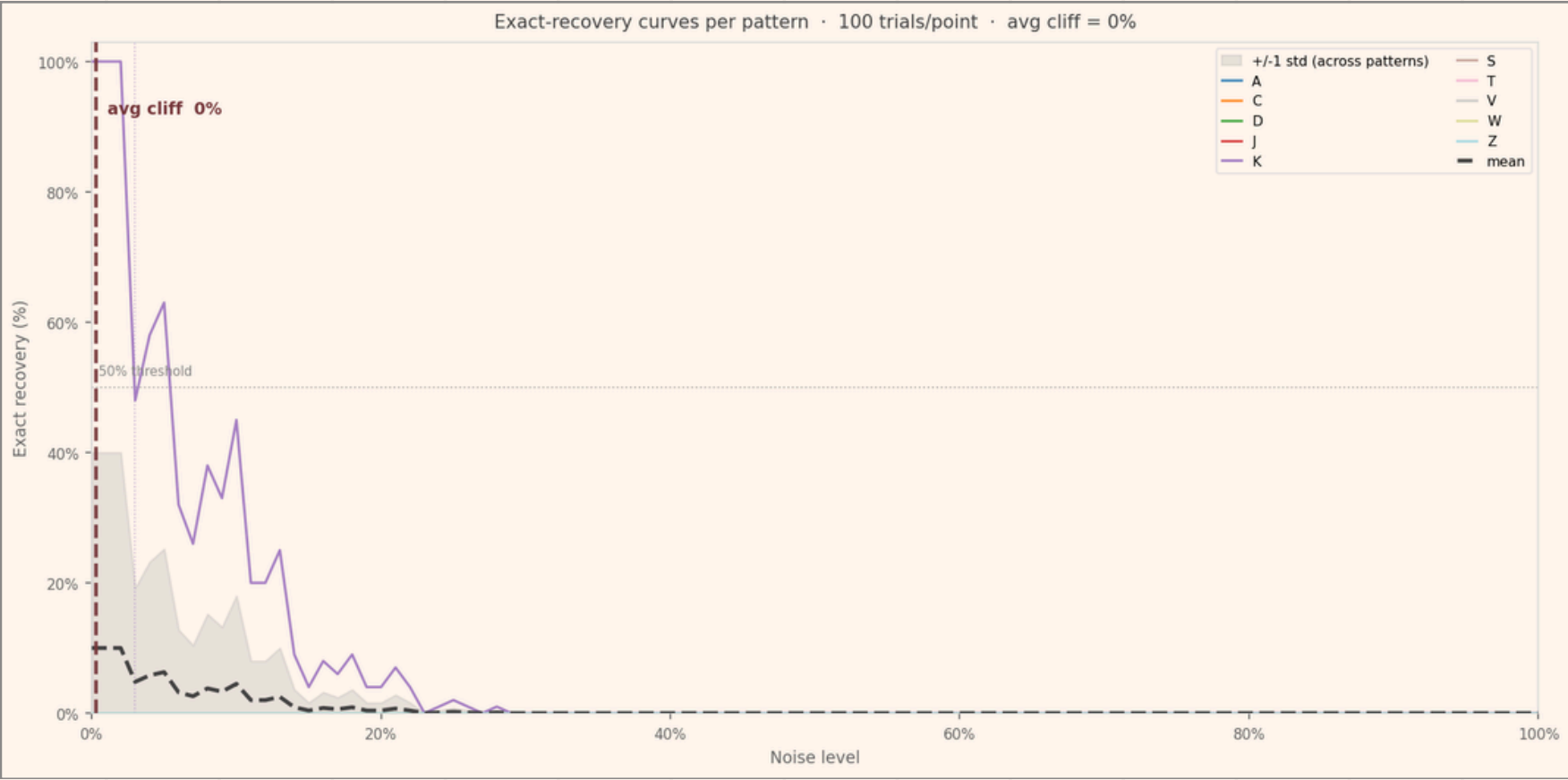
5

Alternativas

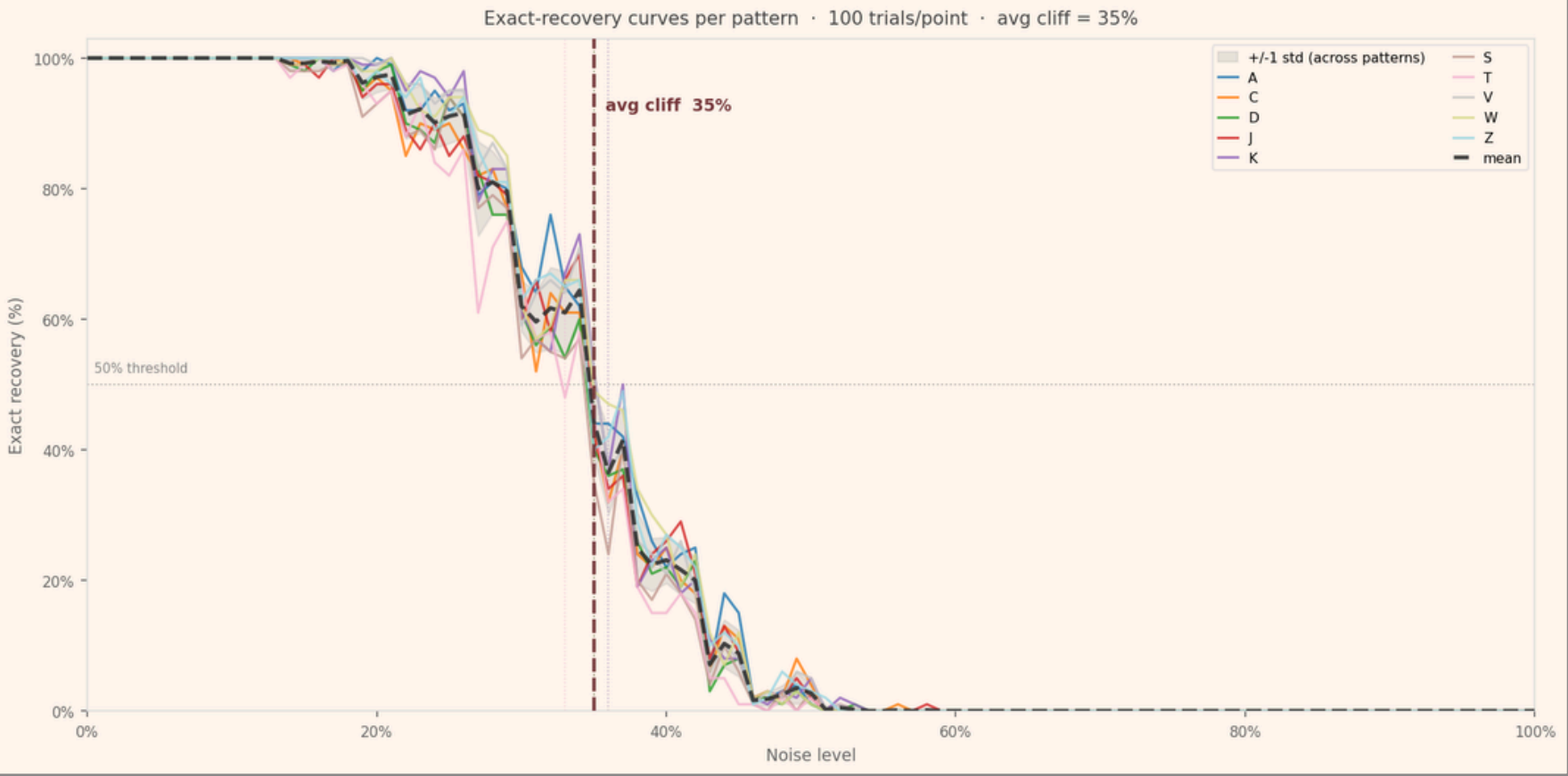
Hopfield Moderno: Continuous Hopfield Networks

Resultados

El nuevo modelo permite incrementar la capacidad



Clásico



Nuevo Modelo

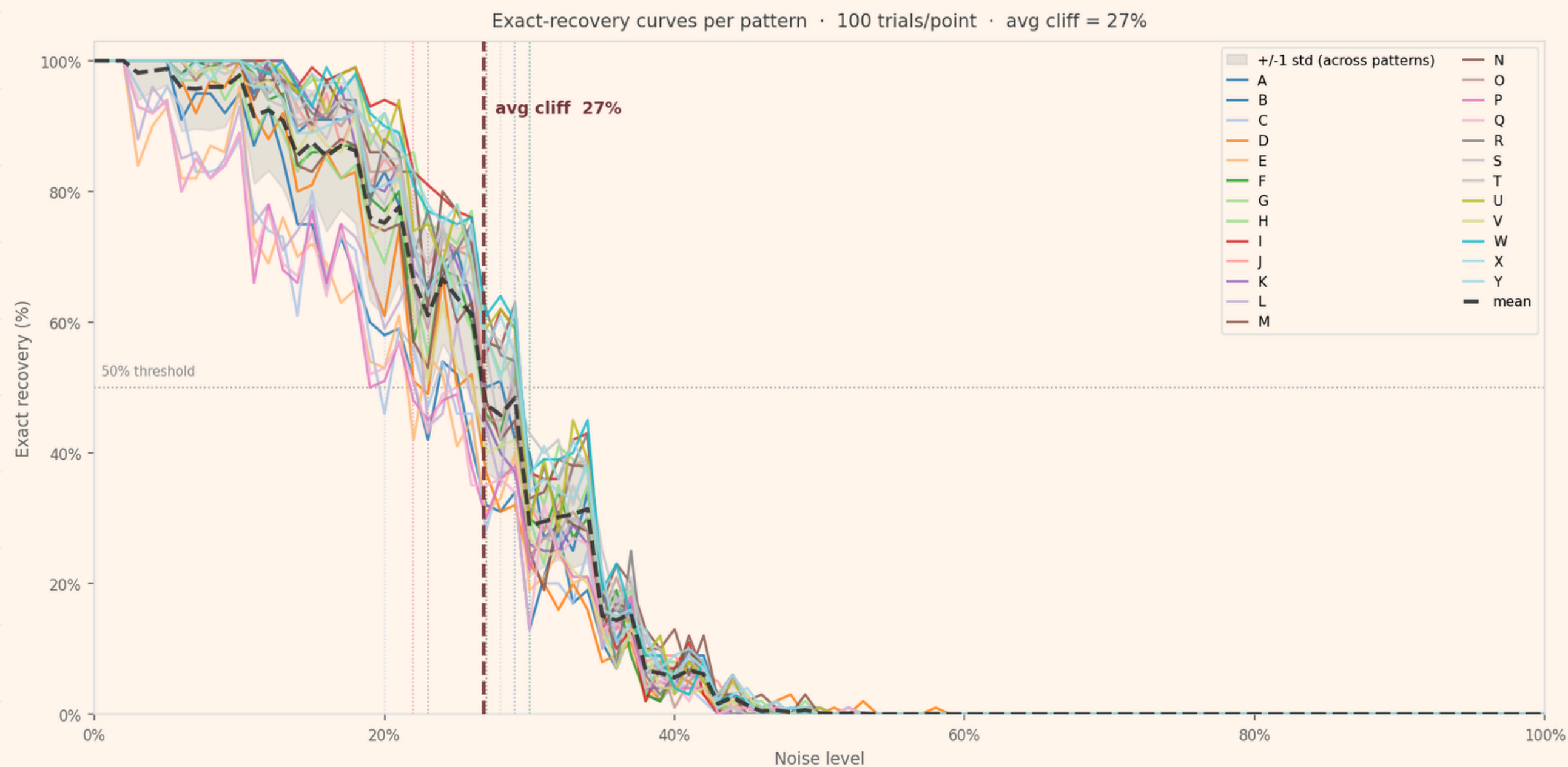
5

Alternativas

Hopfield Moderno: Continuous Hopfield Networks

Resultados

El nuevo modelo permite almacenar el alfabeto completo de forma funcional

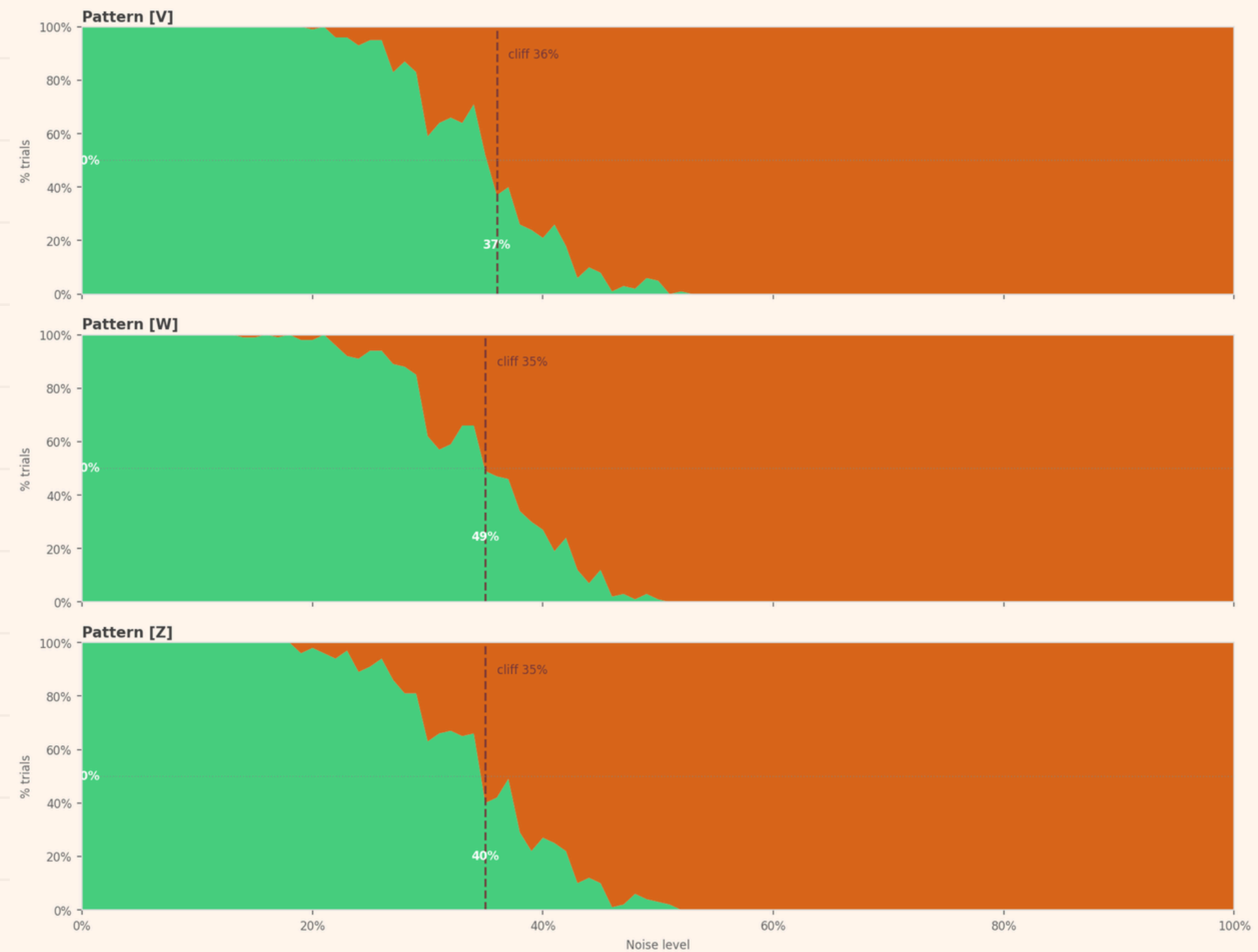
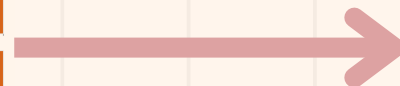
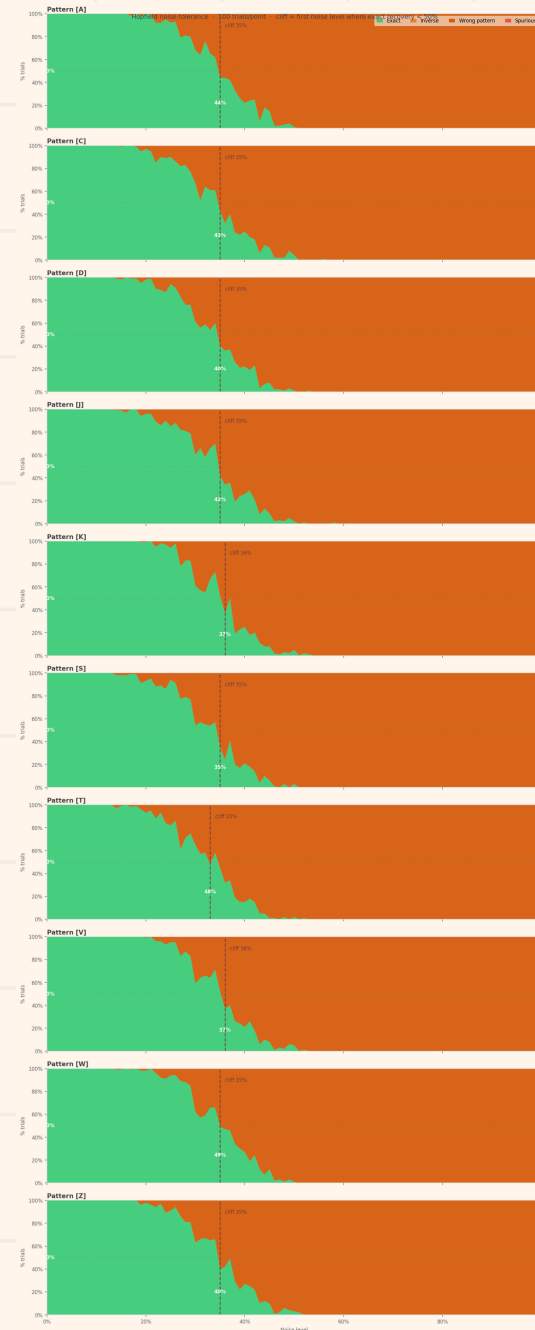


5

Alternativas

Hopfield Moderno: Continuous Hopfield Networks

Desaparecen los inversos y espúreos.
Esto es una propiedad de la nueva función de actualización



ESTRATEGIA



~~Letras~~

~~Abecedario y patrones elegidos~~



~~Paso a paso~~

~~Encontrando la estructura de una letra~~



~~Ruido~~

~~Influencia del ruido en la performance de la red~~



~~Ortogonalidad y clusters~~

~~Cambiando letras y patrones~~



~~Alternativas~~

~~Incrementando la capacidad del modelo y variaciones~~



Conclusiones

Capacidad del modelo y alternativas

6

Conclusiones

Capacidad del modelo y alternativas

- La red de **Hopfield clásica** es una memoria asociativa con capacidad limitada por su geometría: para este caso concreto su performance depende fuertemente del grupo elegido. Ventaja y desventaja a la vez.
- El modo **sincrónico** utilizado implica una mayor velocidad y porcentaje de aciertos, a coste de caer en ciclos con altos niveles de ruido.
- Punto notable: patrones inversos.
- Se pudo mejorar la capacidad enormemente incorporando el **modelo continuo**. Esto podría explorarse más para matrices no binarias (imagenes a color, por ejemplo.)
- Hay un tradeoff: si bien en general funciona mejor en capacidad de memoria, es más sensible al ruido por las características del dominio mismo.

REFERENCIAS

Hopfield Networks is All You Need

Ramsauer, H., Schäfl, B., Lehner, J., Seidl, P., Widrich, M., Adler, T., ... & Hochreiter, S. (2020). Hopfield networks is all you need. arXiv preprint arXiv:2008.02217.

Neural Networks and Learning Machines

Haykin, S. (2009). Neural Networks and Learning Machines (3rd ed.). Pearson.

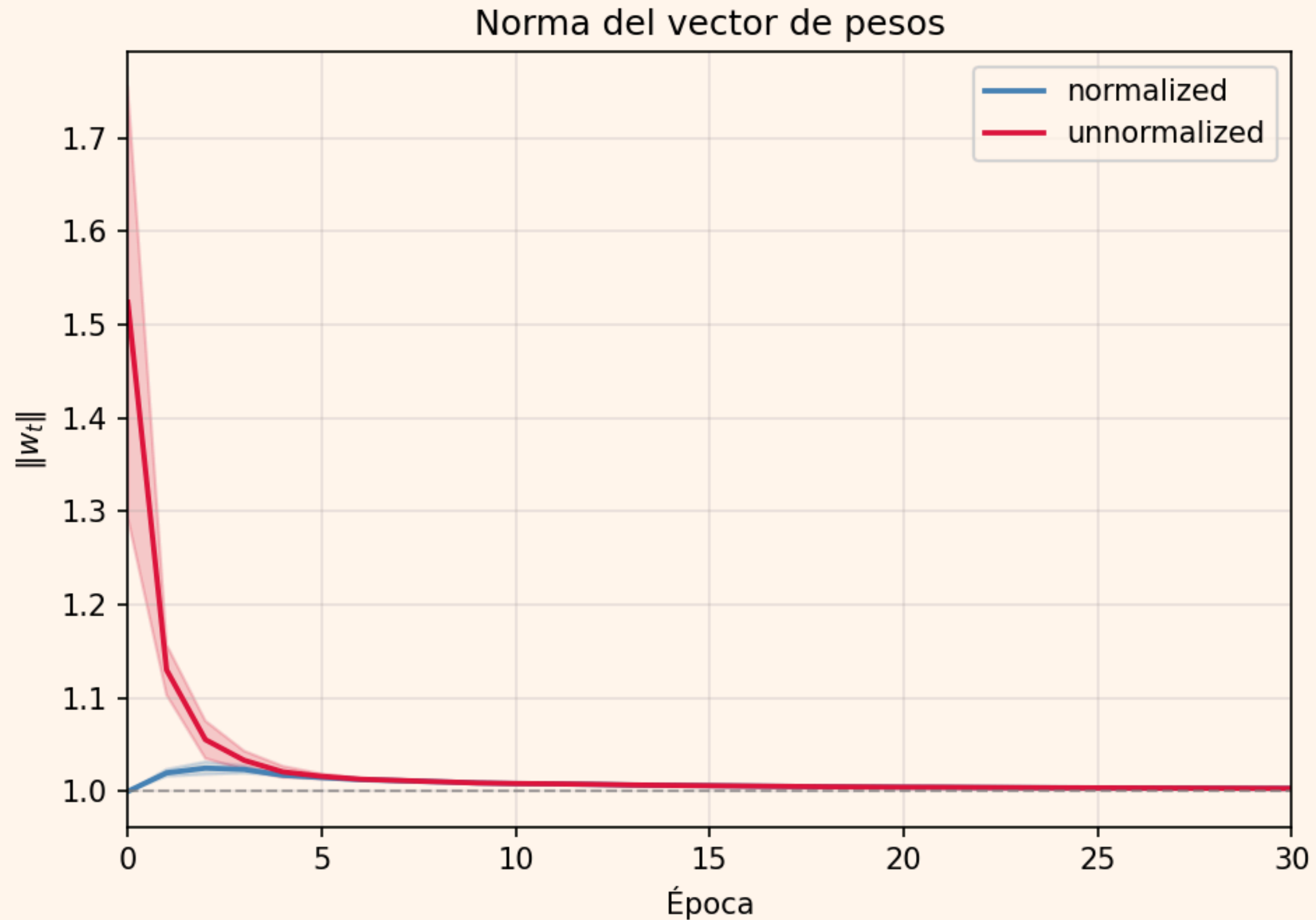
¡GRACIAS!

¿PREGUNTAS?



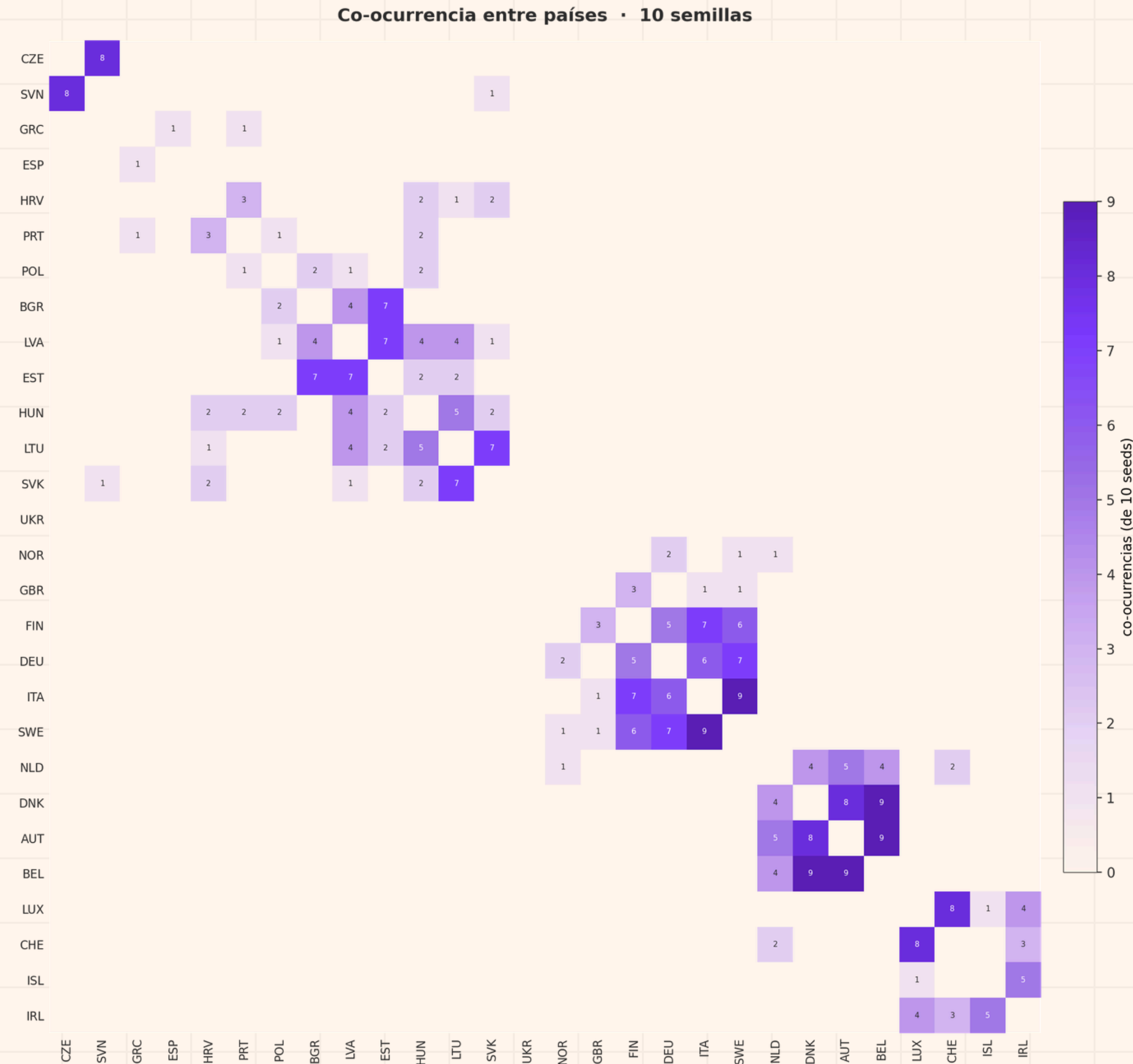
OJA - INICIALIZACIÓN

Inicialización: normalizada vs. sin normalizar (primeras 30 épocas)



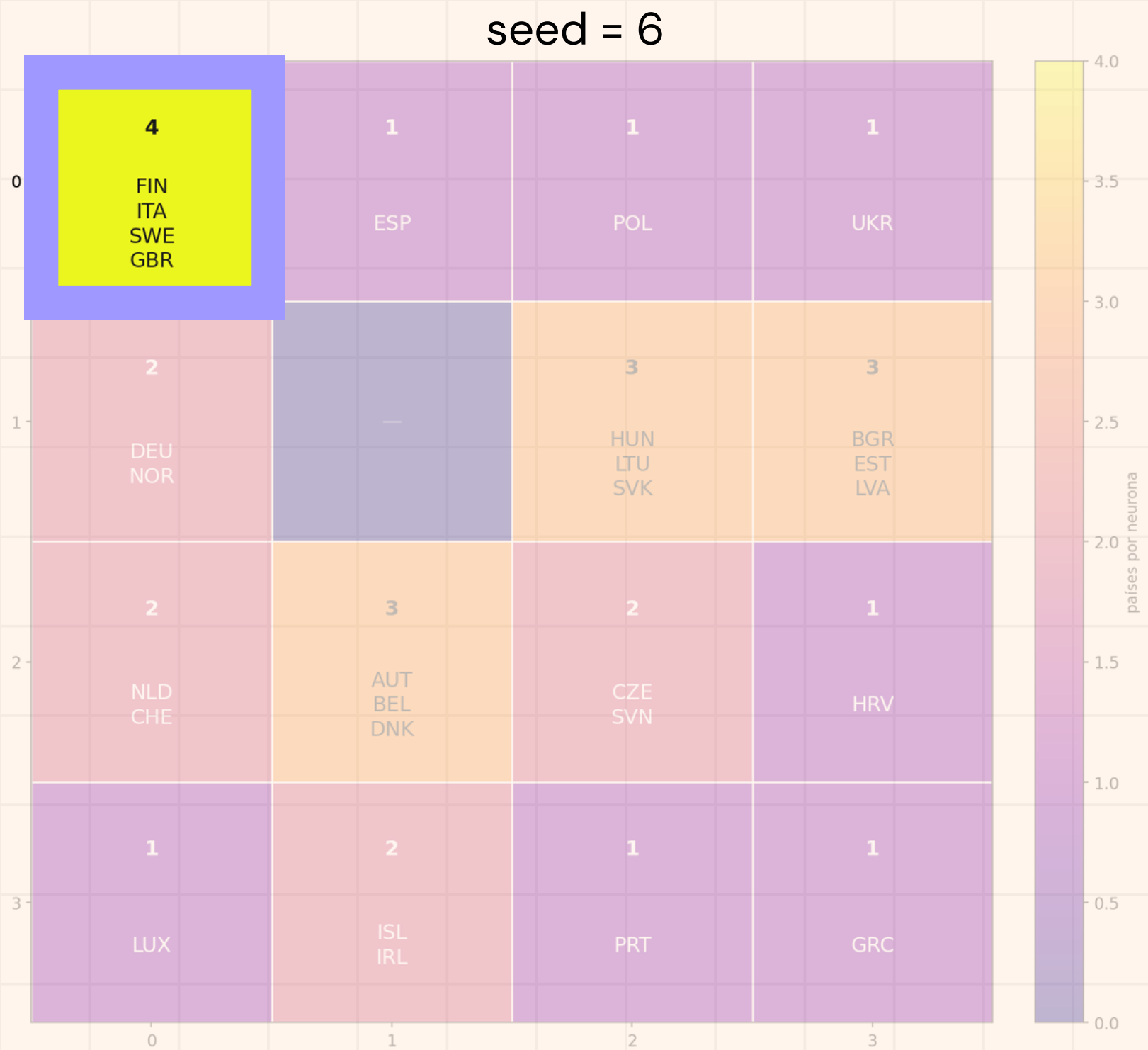
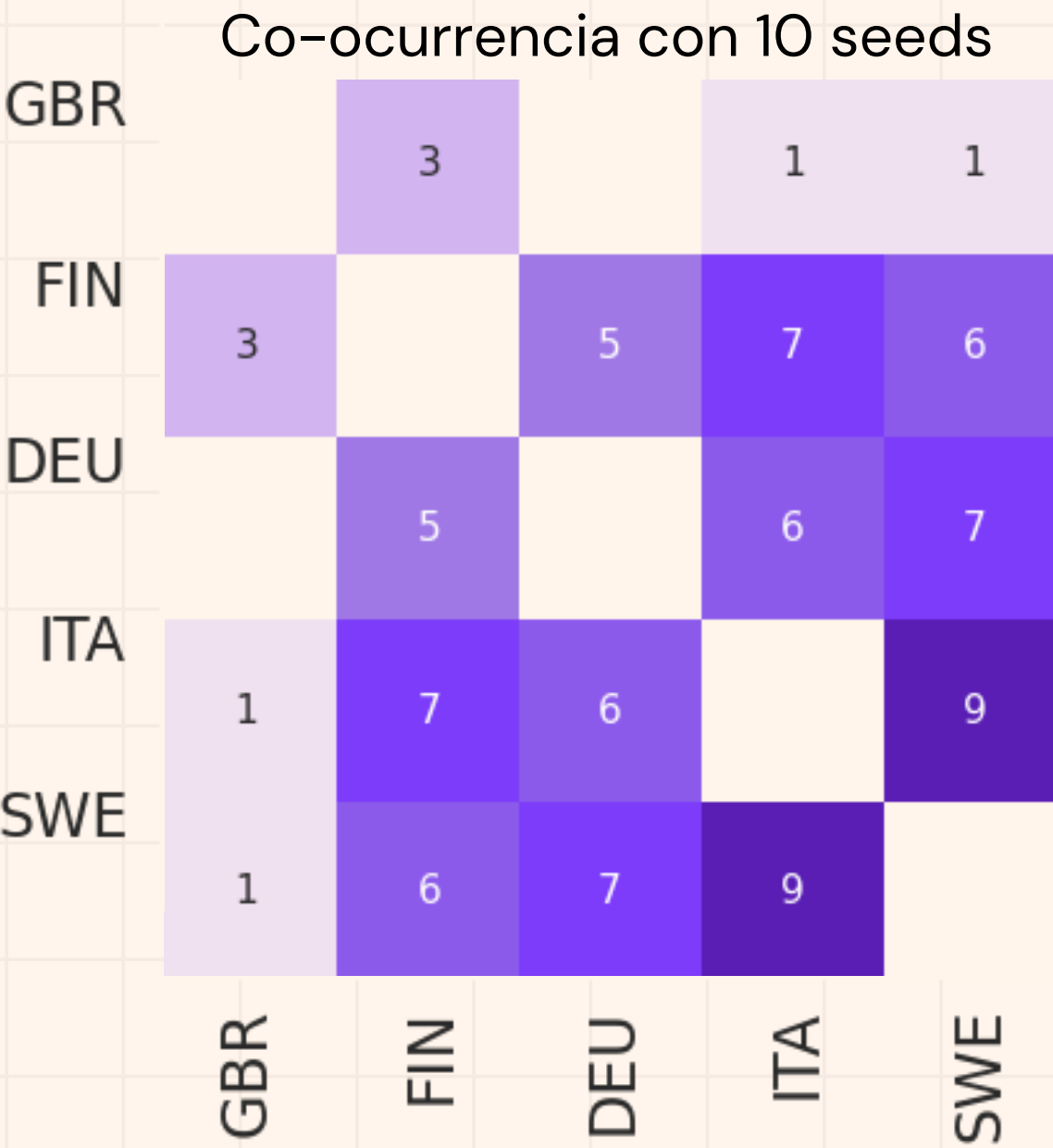
ANEXO A.0

MATRIZ DE CO-OCURRENCIA CON NÚMEROS





ANEXO A.0



A ANEXO A.1 QE Y TE

QUANTIZATION ERROR

- Distancia promedio entre cada país y su neurona ganadora.
- Mide qué tan bien se ajustan los pesos a los datos.
- QE bajo = los pesos representan bien las muestras.

$$QE = \frac{1}{N} \sum_{p=1}^N ||x_p - w_{\hat{k}(x_p)}||$$

N = numero de muestras

x_p = muestra p

$w_{\hat{k}(x_p)}$ = pesos de la neurona ganadora para esa muestra

$|| \cdot ||$ = distancia euclideana

TOPOGRAPHIC ERROR

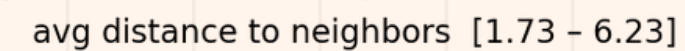
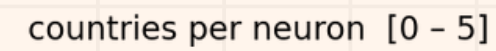
- Proporción de muestras cuya primera y segunda neurona más cercana no son vecinas en la grilla.
- Mide preservación de topología.
- TE bajo = grilla respeta las relaciones de cercanía del espacio original.

$$TE = \frac{1}{N} \sum_{p=1}^N u(x_p)$$

$u(x_p) = 1$ si las primeras y segundas neuronas ganadoras (BMU) de x_p no son vecinas en la grilla, 0 si lo son.

Vecindad: 8 vecinos (incluye diagonales)

ANEXO A.2 COMPARACIÓN DE TASA DE APRENDIZAJE INICIAL





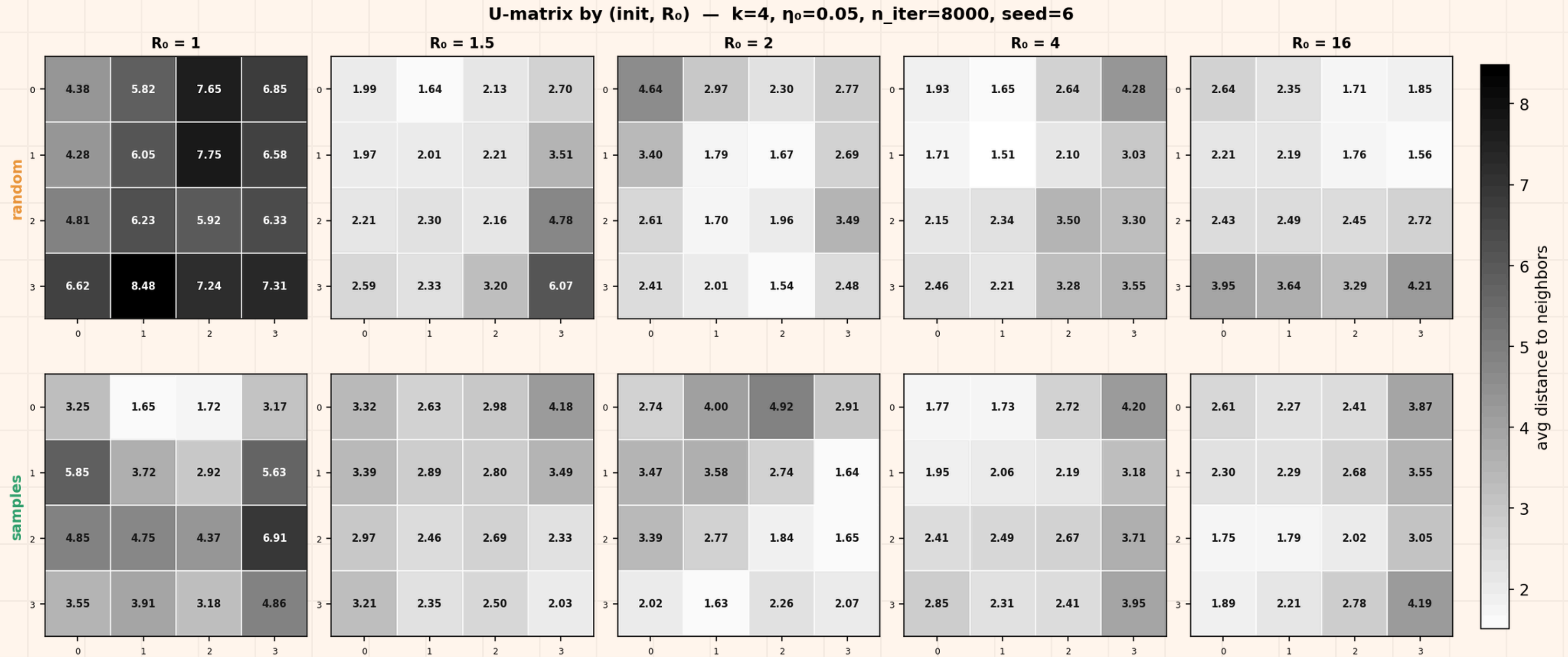
ANEXO A.3 COMPARACIÓN DE INICIALIZACIÓN Y RADIO INICIAL

Summary — k=4, $\eta_0=0.05$, n_iter=8000, seeds=[1, 2, 3, 4, 5, 6, 42, 69, 100, 200]

R ₀	Init	QE (mean ± std)	TE (mean ± std)	Dead neurons (mean ± std)	Max U-dist (mean ± std)	Max cluster (mean ± std)	Singleton % (mean ± std)
1	random	1.4466 ± 0.1405	0.7857 ± 0.2113	10.2 ± 0.9	7.969 ± 0.573	12.1 ± 1.5	47.5% ± 11.9%
1	samples	0.6396 ± 0.0352	0.6929 ± 0.1439	0.0 ± 0.0	6.358 ± 0.720	4.8 ± 0.6	58.8% ± 7.0%
1.5	random	0.7136 ± 0.0663	0.1250 ± 0.0624	0.9 ± 0.8	4.750 ± 0.584	4.0 ± 0.4	48.2% ± 5.9%
1.5	samples	0.7581 ± 0.0599	0.1893 ± 0.0946	1.0 ± 0.6	4.603 ± 0.367	3.8 ± 0.6	41.9% ± 6.8%
2	random	0.7868 ± 0.0808	0.0893 ± 0.0603	1.1 ± 0.8	4.467 ± 0.361	4.0 ± 0.6	42.7% ± 5.7%
2	samples	0.7661 ± 0.0739	0.1071 ± 0.0782	1.1 ± 0.9	4.478 ± 0.212	3.9 ± 0.9	42.5% ± 9.5%
4	random	0.7382 ± 0.0376	0.0750 ± 0.0628	0.5 ± 0.7	4.337 ± 0.088	3.8 ± 0.4	43.0% ± 6.3%
4	samples	0.7673 ± 0.0600	0.0750 ± 0.0406	0.8 ± 0.6	4.365 ± 0.105	3.8 ± 0.4	41.8% ± 8.9%
16	random	0.7341 ± 0.0313	0.0643 ± 0.0311	0.7 ± 0.6	4.295 ± 0.188	4.0 ± 0.4	47.0% ± 7.5%
16	samples	0.7411 ± 0.0547	0.0464 ± 0.0393	0.7 ± 0.8	4.284 ± 0.077	4.0 ± 0.4	48.0% ± 9.5%



ANEXO A.3 COMPARACIÓN DE INICIALIZACIÓN Y RADIO INICIAL

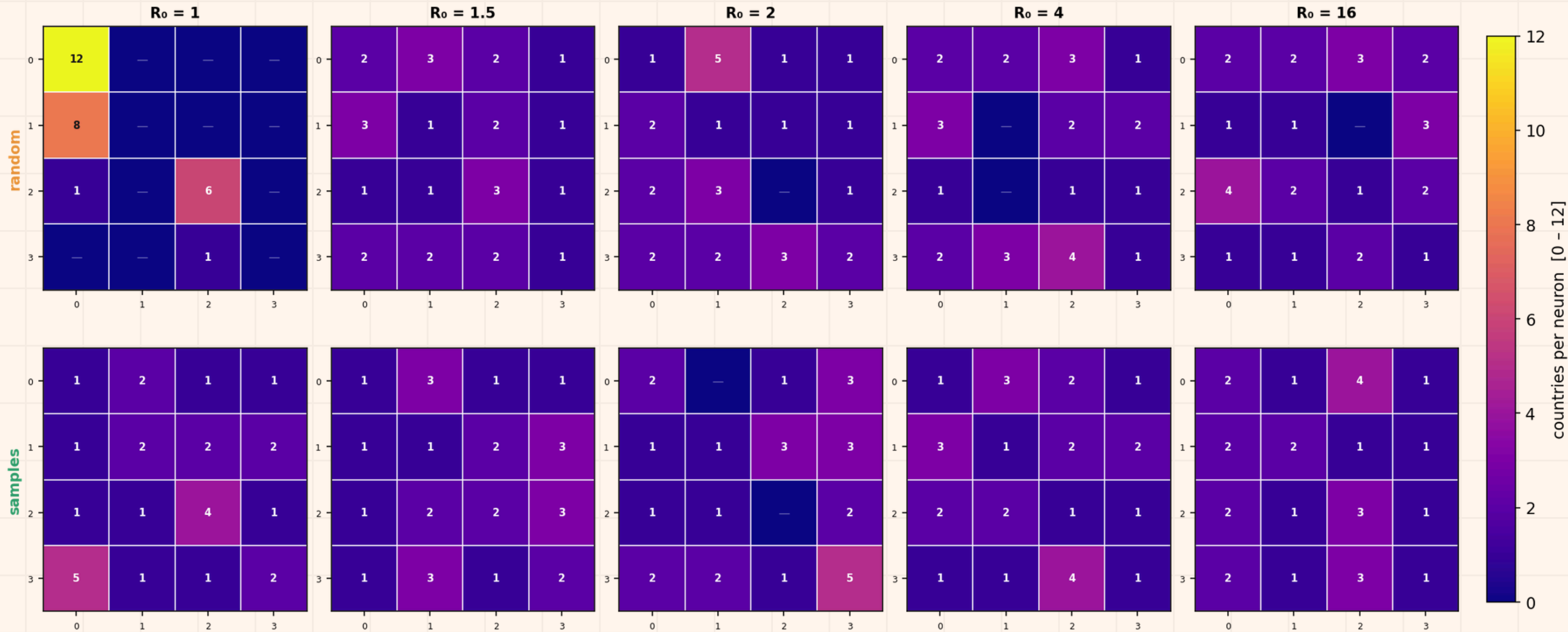




ANEXO A.3

COMPARACIÓN DE INICIALIZACIÓN Y RADIO INICIAL

Hit map by (init, R_0) — $k=4$, $\eta_0=0.05$, $n_iter=8000$, $seed=6$



CONFIGURACIÓN FINAL



`k` **4**
`eta_0` **0.05**
`radius_0` **k**
`n_iter` **500×k×k**
`weight_init` **samples**
`distance` **euclidean**





ANEXO A.4

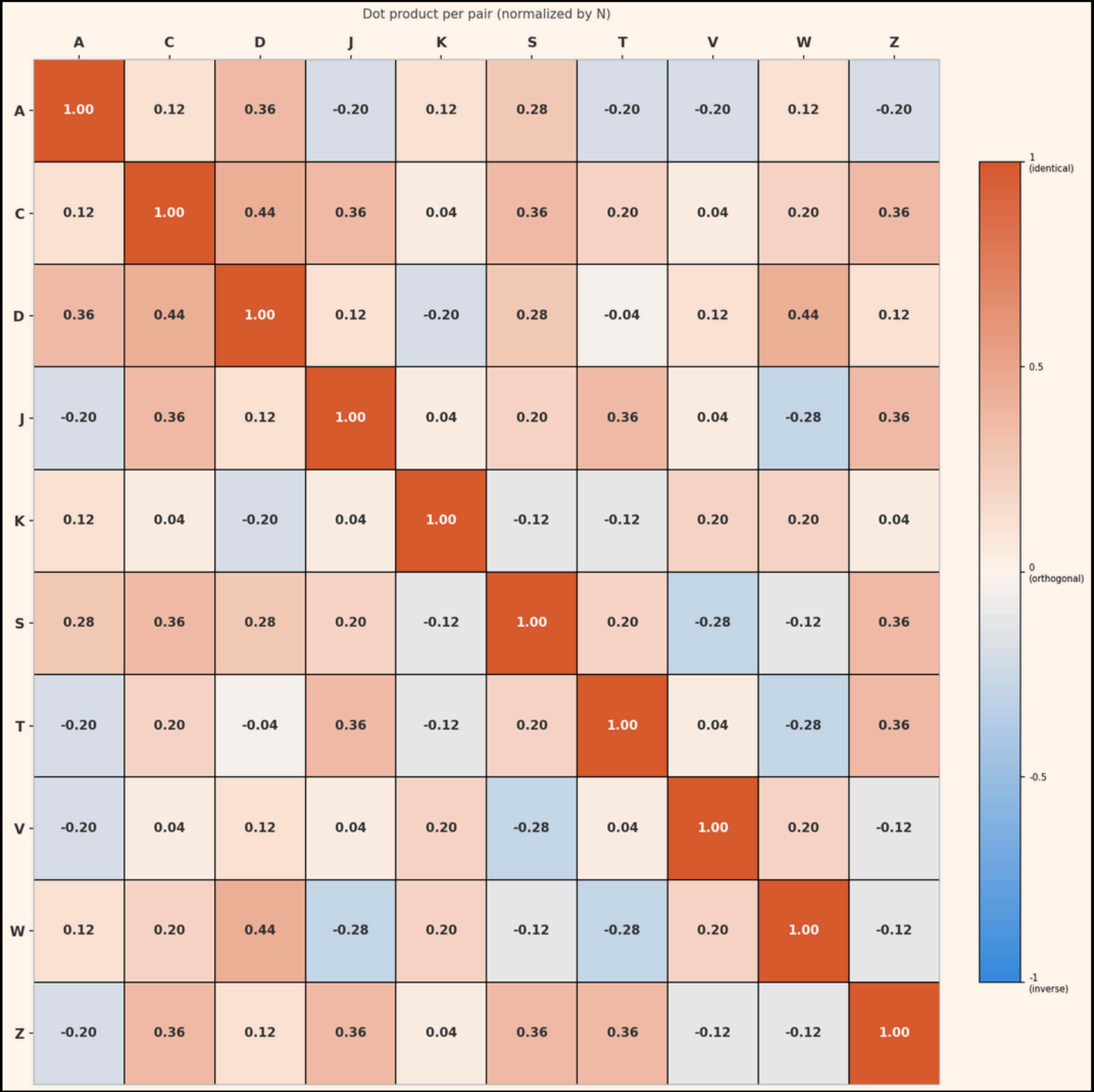
ELECCIÓN DE PATRONES (TOMADOS DE A 10)

Top 15 combined:		
<.> max	<.> medio	group
(11.0, 2.0)	5.09	(A, C, D, J, K, S, T, V, W, Z)
(11.0, 2.0)	4.87	(C, D, I, J, K, P, S, V, W, Y)
(11.0, 2.0)	4.87	(C, D, J, K, P, S, T, V, W, Z)
(11.0, 2.0)	4.87	(C, D, J, K, P, S, V, W, Y, Z)
(11.0, 3.0)	4.87	(C, D, I, J, K, R, S, V, W, Y)
(11.0, 3.0)	4.73	(C, D, J, K, R, S, T, V, W, Z)
(11.0, 3.0)	4.87	(C, D, J, K, R, S, V, W, Y, Z)
(11.0, 4.0)	5.09	(C, D, F, I, J, K, S, V, W, Y)
(11.0, 4.0)	5.27	(C, D, F, J, K, S, T, V, W, Z)
(11.0, 4.0)	5.09	(C, D, F, J, K, S, V, W, Y, Z)
(13.0, 1.0)	5.40	(A, C, D, F, I, J, K, S, V, W)
(13.0, 1.0)	5.36	(A, C, D, F, J, K, S, T, V, W)
(13.0, 1.0)	5.44	(A, C, D, F, J, K, S, T, V, Z)
(13.0, 1.0)	5.84	(A, C, D, F, J, K, S, T, W, Z)
(13.0, 1.0)	5.40	(A, C, D, F, J, K, S, V, W, Z)



ANEXO A.4

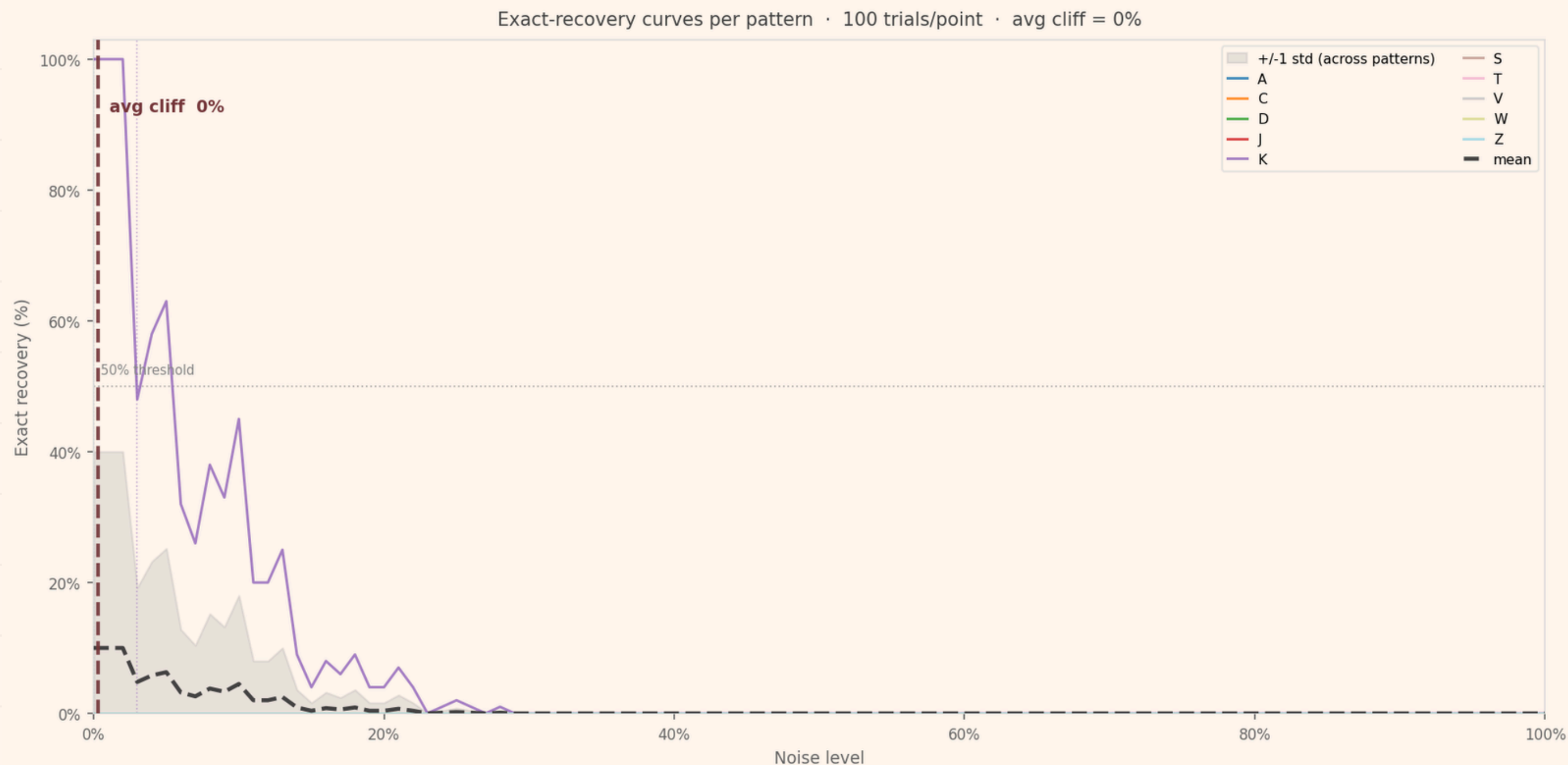
PRODUCTO ESCALAR NORMALIZADO PARA DATASET DE 10 PATRONES ELEGIDO





ANEXO A.4

RESULTADOS CON 10 PATRONES



Solo se puede recuperar parcialmente la letra k
(menor correlación en comparación con las otras letras)

The next theorem states that the update rule typically retrieves patterns after one update. Retrieval of a pattern $\mathbf{x}_i$ for fixed point $\mathbf{x}_i^*$ and query $\boldsymbol{\xi}$ is defined via an ϵ by $\|f(\boldsymbol{\xi}) - \mathbf{x}_i^*\| < \epsilon$, that is, the update is ϵ -close to the fixed point. Retrieval with one update is crucial to integrate modern Hopfield networks into deep learning architectures, where layers are activated only once. First we need the concept of separation of a pattern. For pattern $\mathbf{x}_i$ we define its separation Δ_i to other patterns by:

$$\Delta_i := \min_{j, j \neq i} (\mathbf{x}_i^T \mathbf{x}_i - \mathbf{x}_i^T \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_i - \max_{j, j \neq i} \mathbf{x}_i^T \mathbf{x}_j. \quad (5)$$

The update rule retrieves patterns with one update for well separated patterns, that is, patterns with large Δ_i .

Theorem 4. *With query $\boldsymbol{\xi}$, after one update the distance of the new point $f(\boldsymbol{\xi})$ to the fixed point $\mathbf{x}_i^*$ is exponentially small in the separation Δ_i . The precise bounds using the Jacobian $J = \frac{\partial f(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}}$ and its value J^m in the mean value theorem are:*

$$\|f(\boldsymbol{\xi}) - \mathbf{x}_i^*\| \leq \|J^m\|_2 \|\boldsymbol{\xi} - \mathbf{x}_i^*\|, \quad (6)$$

$$\|J^m\|_2 \leq 2 \beta N M^2 (N - 1) \exp(-\beta (\Delta_i - 2 \max\{\|\boldsymbol{\xi} - \mathbf{x}_i\|, \|\mathbf{x}_i^* - \mathbf{x}_i\|\} M)). \quad (7)$$

For given ϵ and sufficient large Δ_i , we have $\|f(\boldsymbol{\xi}) - \mathbf{x}_i^\| < \epsilon$, that is, retrieval with one update.*

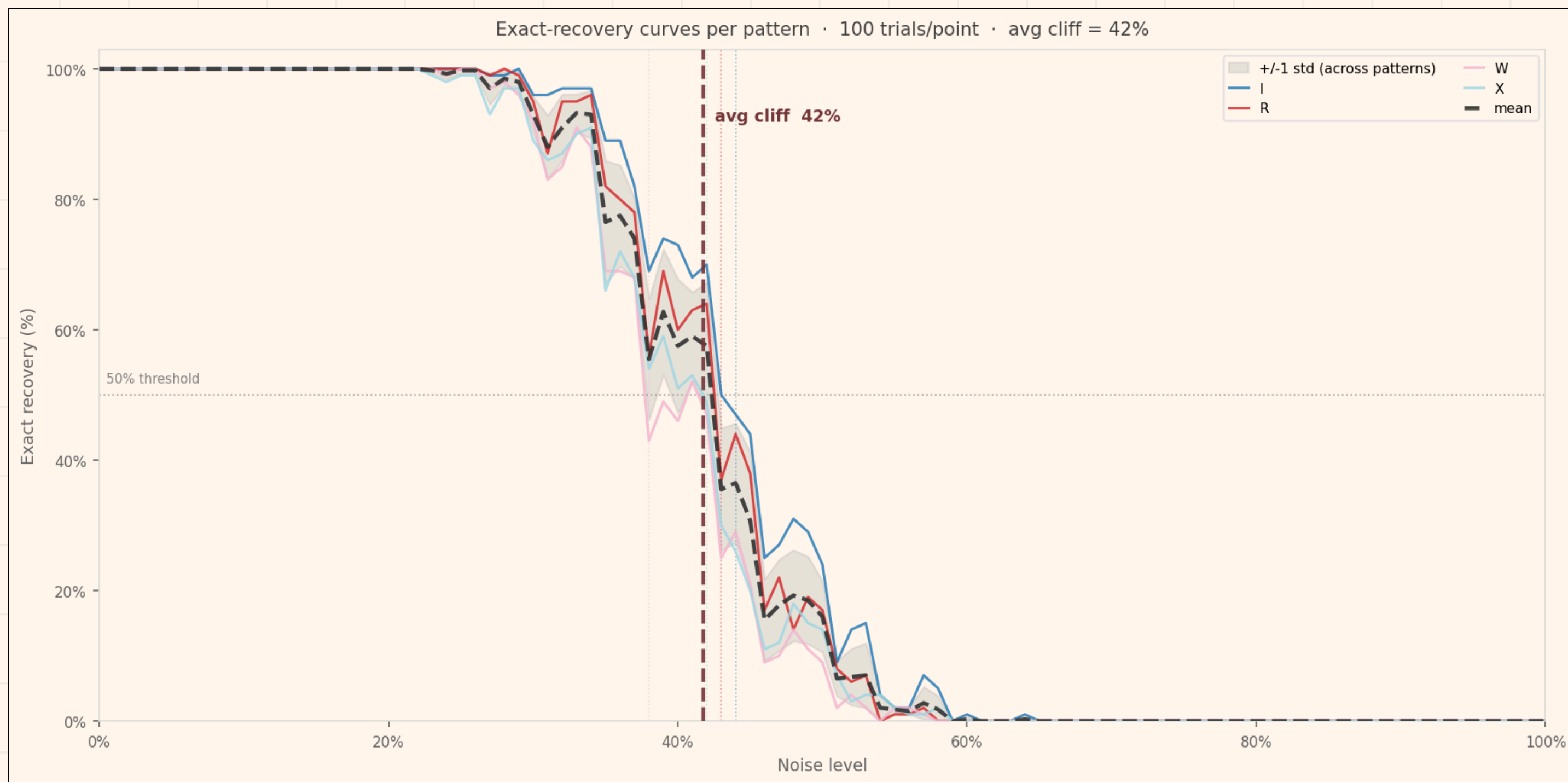
See proof in appendix Theorem [A8](#).

At the same time, the retrieval error decreases exponentially with the separation Δ_i .

A

ANEXO A.5

LUEGO PARA POCOS PATRONES
PERO BIEN SEPARADOS, MEJORA LA
PERFORMANCE!

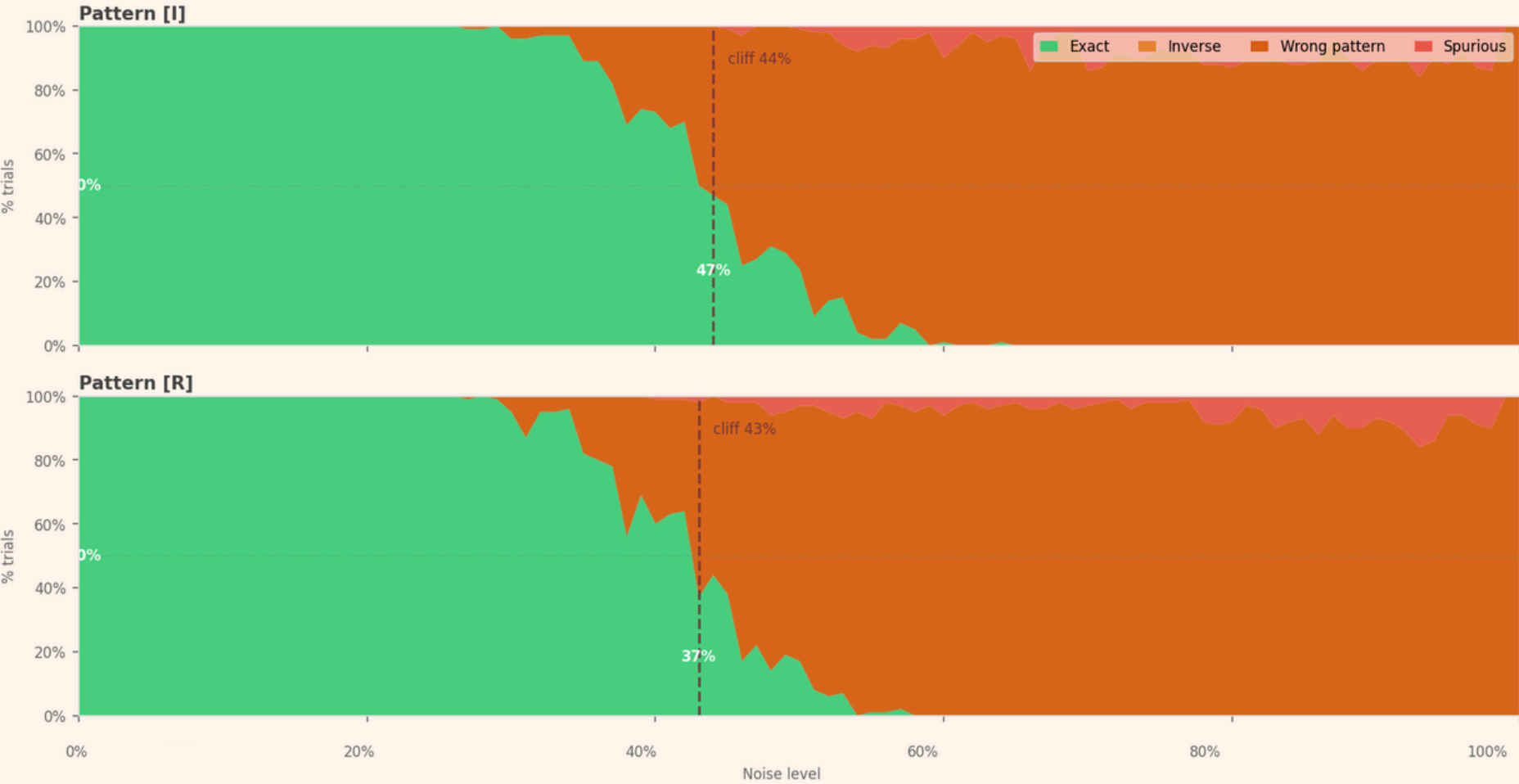


A

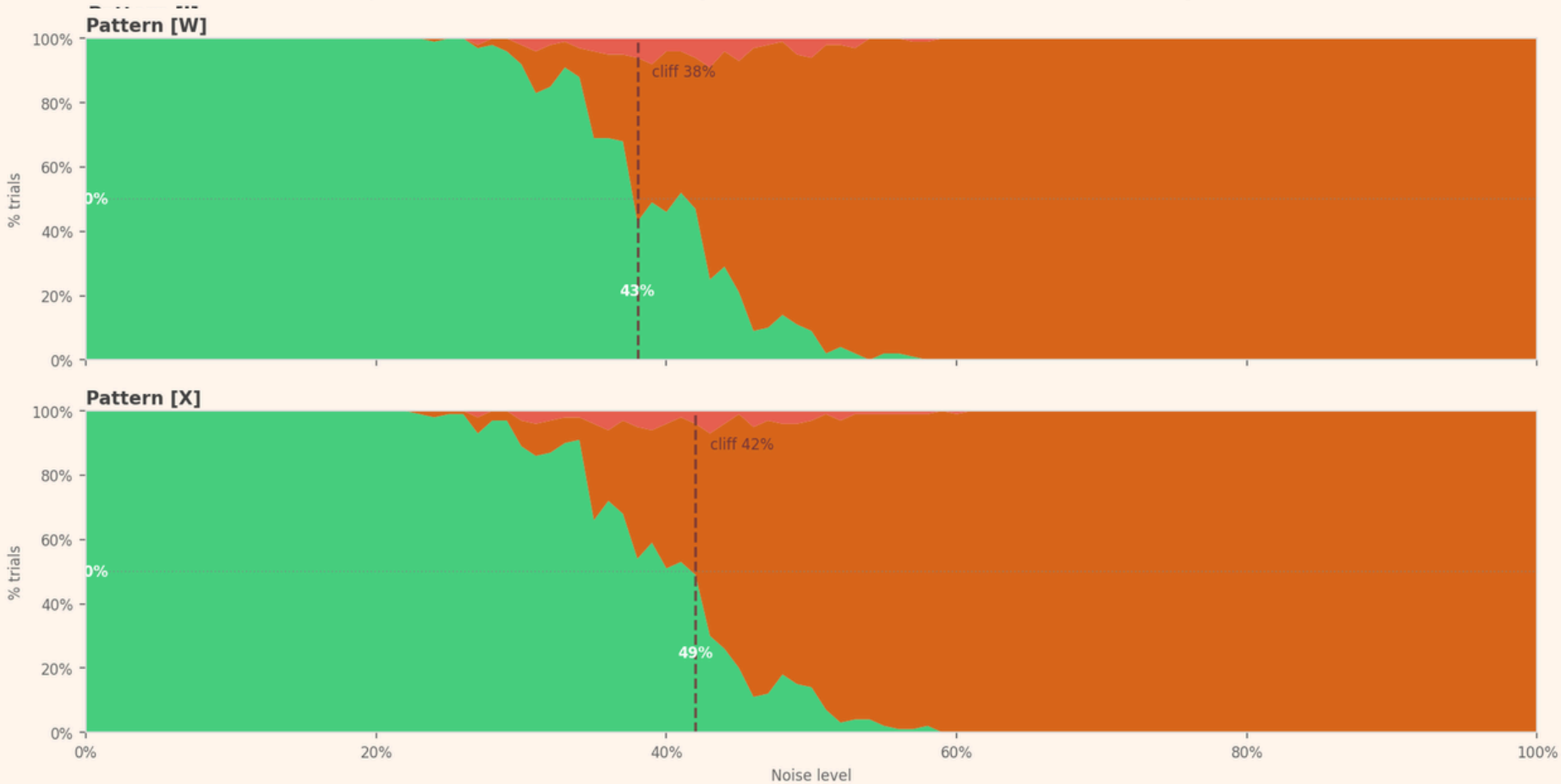
ANEXO A.5

LUEGO PARA POCOS PATRONES
PERO BIEN SEPARADOS, MEJORA LA
PERFORMANCE!

Hopfield noise-tolerance · 100 trials/point · cliff = first noise level where exact recovery < 50%



Hopfield noise-tolerance · 100 trials/point · cliff = first noise level where exact recovery < 50%



As with classical Hopfield networks, we consider patterns on the sphere, i.e. patterns with a fixed norm. For randomly chosen patterns, the number of patterns that can be stored is exponential in the dimension d of the space of the patterns ($\mathbf{x}_i \in \mathbb{R}^d$).

Theorem 3. *We assume a failure probability $0 < p \leq 1$ and randomly chosen patterns on the sphere with radius $M := K\sqrt{d-1}$. We define $a := \frac{2}{d-1}(1 + \ln(2\beta K^2 p(d-1)))$, $b := \frac{2K^2\beta}{5}$, and $c := \frac{b}{W_0(\exp(a+\ln(b)))}$, where W_0 is the upper branch of the Lambert W function ([Olver et al., 2010, \(4.13\)](#)), and ensure $c \geq \left(\frac{2}{\sqrt{p}}\right)^{\frac{4}{d-1}}$. Then with probability $1 - p$, the number of random patterns that can be stored is*

$$N \geq \sqrt{p} c^{\frac{d-1}{4}}. \quad (4)$$

Therefore it is proven for $c \geq 3.1546$ with $\beta = 1$, $K = 3$, $d = 20$ and $p = 0.001$ ($a + \ln(b) > 1.27$) and proven for $c \geq 1.3718$ with $\beta = 1$, $K = 1$, $d = 75$, and $p = 0.001$ ($a + \ln(b) < -0.94$).

For a proof, see appendix Theorem [A5](#).